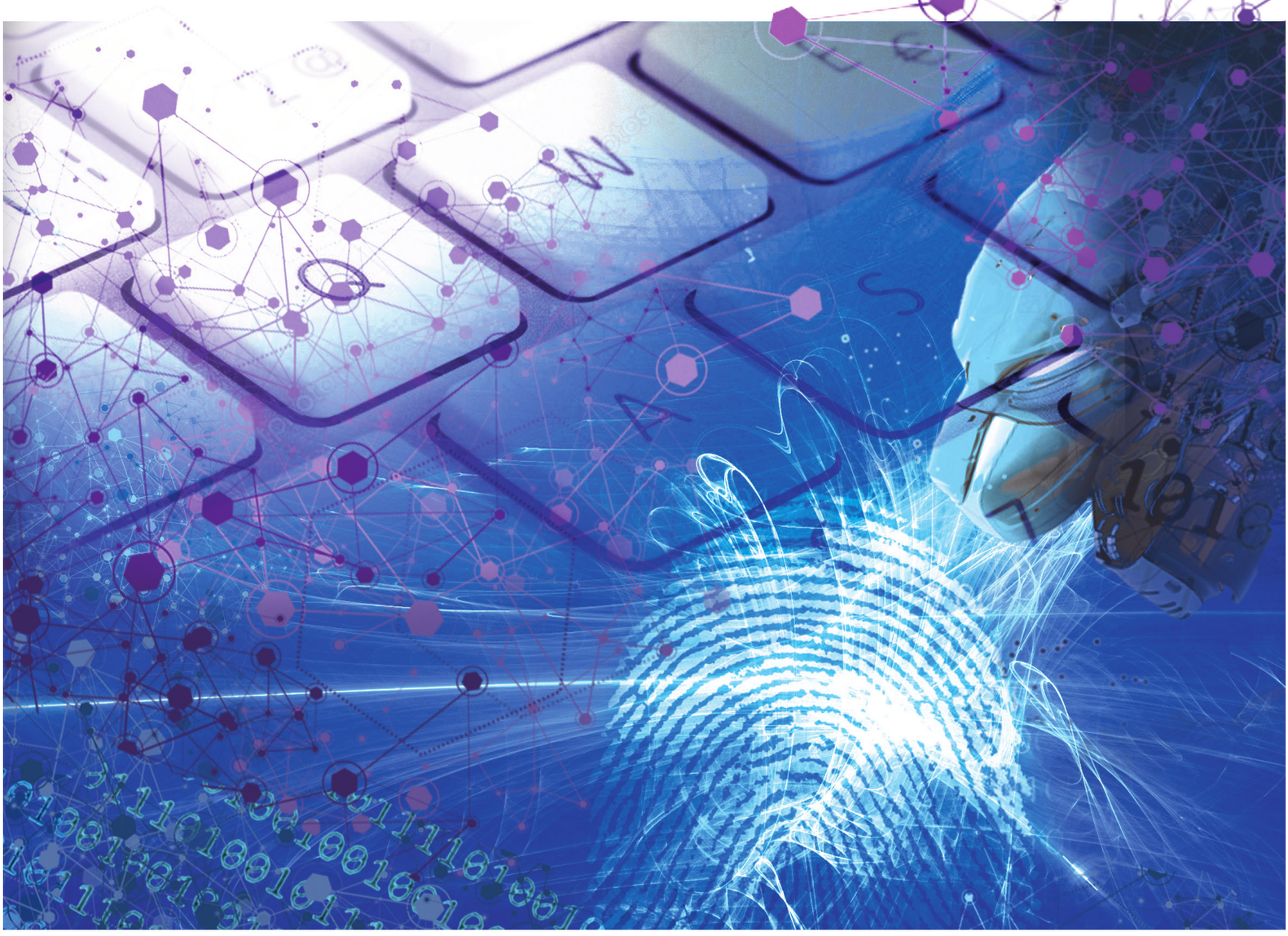
OLAF

OFFICE EUROPÉEN DE LUTTE ANTIFRAUDE

EUROPEAN COMMISSION **EUROPEAN ANTI-FRAUD OFFICE**

Live Data Forensics



This training is financed by the European Union Anti-Fraud Programme – Hercule component (UAFP)

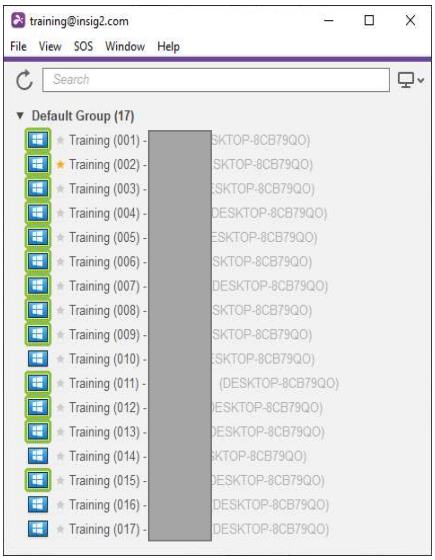
INSIG2

Advanced RAM Analysis and Forensics

Copyright 2021 – CSITech Ltd

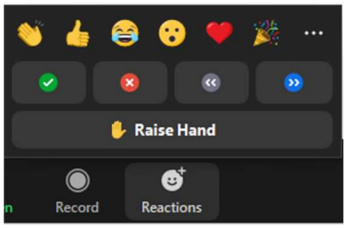
Technicalities - Splashtop

- App for remote desktop access
- **Download** the app Splashtop Business Access https://www.splashtop.com/downloads#ba
- Install/run and enter the credentials:
  - Username: training@insig2.com
  - Password: in the Moodle system
- There will be a pop-up saying your account needs to be verified and that an email has been sent to the administrator
- We will enable your connection and in 30sec just try logging in again and now you should have access
- Double click on the machine with your name



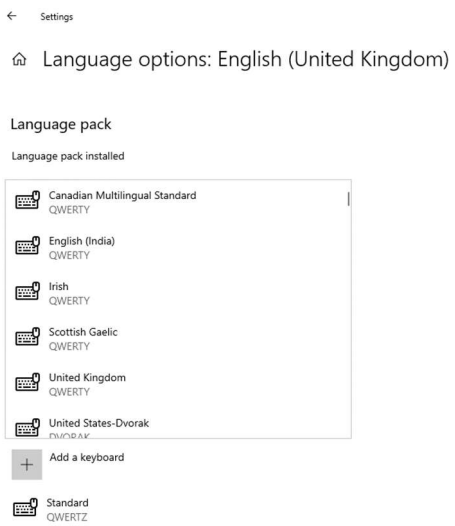
Technicalities - Zoom

- Presentation and interaction
- It is recommended to have your camera always turned on and mic on mute
- Please use the built chat for quick questions
  - Alt+Enter - go/add next row in chat
- Please use Reactions button for quick answers
  - Raise a hand for question
  - Use slow down or speed up if you have trouble flowing the class



Technicalities- Keyboard layout

- The Remote machines have Standard US keyboard layout
- You can change it to match yours as we will be doing a LOT of typing with special characters
- In Win 10:
  1. Select the **Start** button, then select **Settings** > **Time & Language** > **Language**.
  2. Under **Preferred languages**, select the language that contains the keyboard you want, and then select **Options**.
  3. Select **Add a keyboard** and choose the keyboard you want to add.



Rules

- Class runs:
  - Monday, 9:00 – 16h CEST
  - Tuesday, 9:00 – 16h CEST
  - Wednesday, 9:00 – 16h CEST
  - Thursday, 9:00 – 16h CEST
  - Friday, 9:00 – 13ish CEST
- Break every 50min + 1h lunch break at 12 (noon)
- Friday:
  - Bit of lecturing
  - Survey – you tell us how we did
  - Test
    - Open book and internet
    - Practical and theory
    - 30 multiple answer questions
    - 70% for passing

- We can see what you are doing at all times!
- We have to grade your engagement and participation at the end of class

- **NOT PERMITTED**
  - Being late
  - Recording
  - Sharing of computers or Zoom invites
  - Missing of class time
  - Cheating during exam
  - Posting on social media
  - Speak of names, places or dates

Krešimir Hausknecht

- INsig2 – private company from Croatia
  - Training, consulting and reselling DF products
- Head of Digital Forensics Department at INsig2
  - Managing, training and conducting investigations
- Zagreb University of Applied Sciences
  - Lecturer
- M.Sc. ICT, CFCE, ACE, MSAB, Cellebrite, Oxygene… certificates
- Specialist in Live Data, OSINT, Password Recovery

www.insig2.com Twitter - @sokre_haus
Kresimirh@insig2.com

You can find and follow me here:

Web site - www.insig2.com
Twitter - @sokre_haus

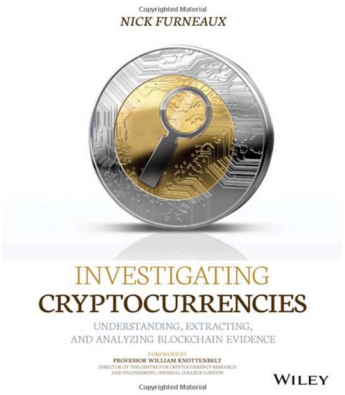
Copyright 2021 – CSITech Ltd

Petar Majić

- FPZ - mag.inf.
- Digital Forensics Consultant in INsig2 Ltd.
- Magnet Forensics, Cellebrite, FTK, Oxygen Forensics Detective…

Nick Furneaux

- MD of CSITech
- Blockchain Intelligence Expert at TRM Labs and Advisor to the Board at Asset Reality
- Specialist in Cryptocurrencies
  - Author of Investigating Cryptocurrencies
- Covert techniques and tools
- Trainer, Investigator, Author



You can find and follow me here:

Web site and occasional blog - www.csitech.co.uk
Twitter - @nickfx

Copyright 2021 – CSITech Ltd

Who are you and what do you expect from the course?

- Background
- How long have you been in DFIR
- Filed work
- Investigator, analysis, analytics
- Working form home or office?
- Tools you use
- What do you expect from the course?

- https://forms.gle/nF235F5CwQwuwKCR8

Copyright 2021 – CSITech Ltd

Why care about live data?

- Computers are regularly left on
- Corporate needs
- Encryption
- Volatile data
  - RAM
- Covert requirements
  - Less data to capture - speed
- Cause it's the right thing to do!!
  - We miss loads of data when we pull the plug
  - Any ideas?

There are many reasons why taking live data from a running machine is the right method. You would never dream of leaving a 4 gig USB key at a crime scene but we regularly pull the plug on 4 gig of RAM data. Perhaps in the past this didnt matter as 'nothing' could be got from a RAM dump. By the end of the course you will be able to carry out a significant investigation of RAM, sometimes on unique data which could never be found on the disk.

Encryption is the other major issue. However, if a computer is running and encrypted volumes are open then they should be imaged in a live state.

Covert issues are a minor consideration for most teams but when it is critical, it is much easily to grab 4 gig of RAM from an Internet Café covertly than sitting there for 3 hours trying to quietly image the hard drive over USB!

Copyright 2021 – CSITech Ltd

Why care about live data?

- Safe browsing, much still in ram
- Using a non-caching USB key running portable apps and OS'es
  - www.portableapps.com
- In one go, we can do:
  - Live system, dead box, static and dynamic malware analysis
- RAM may be all you have
- Technical limitations
- Faster to analyse

Browse to www.portableapps.com. There are applications for running purely from a USB key, they include:

Firefox
Skype
Email clients
Encryption
Eraser

These do not cache to the disk but artifacts will be available in memory.

Copyright 2021 – CSITech Ltd

What will we cover?

- Live scene
- Tool testing
- Understanding RAM
- Imaging RAM
- Quick wins
- Volatility 2 and 3
- Password cracking and decryption
- MemprocFS
- Linux
- macOS
- Practical's

Copyright 2021 – CSITech Ltd

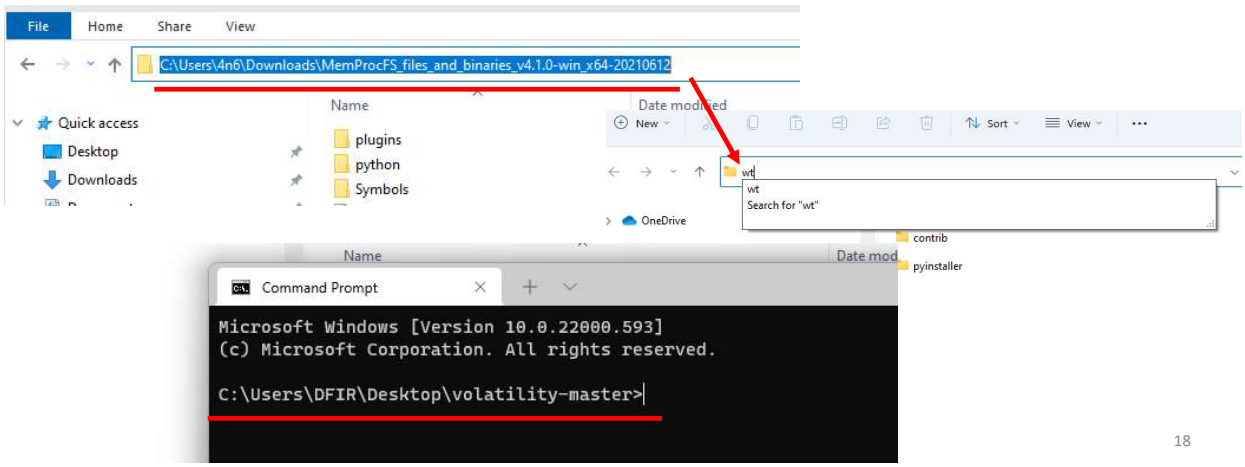
Typography

- Any **BOLD** text is purely to make a point stand out
- Any text inside <…> purely means, put something here. **This would not include the <>**
  - For example:
  - <path2yourRAMimage>
  - C:\Win10.img
- Any text in the `Courier font`, is text to be typed into a command shell
- If the command goes into next line, there is a SPACE between, not multiple commands
- **Most of the handouts HAVE NOTES beneath them, R E A D!!!! ☺**
- **TopTip:** save shell commands to a txt file so you have them for later

Copyright 2021 – CSITech Ltd

Opening command prompt in Windows Terminal

- Press or click start – type WT and press enter/return
- Left click on the address bar in explorer
  - While all is selected (in blue) type WT and press enter/return



Basic CMD commands

- CD <folder name> - go to this folder
- Dir – list continence of a folder
- Cd.. – go one folder back

- Use TAB to autocomplete the folder/file name
  - Will rotate all file/folder that start with whatever you typed
- Use CTRL + arrow keys – move form word to word, back forward

- If you want to output a command to a text file use the “>” sign at the end of command and name the output file **WITH extension**
  - Single > creates new file
  - Double >> appends to the existing file

Working on a Live System

Copyright 2021 – CSITech Ltd

Copyright 2021 – CSITech Ltd

Copyright 2021 – CSITech Ltd

Working on a Live System

- ACPO UK guideline 1 states:

> No action taken by law enforcement agencies or their agents should change data held on a computer or storage media which may subsequently be relied upon in court

With live data analysis, is it possible to work on copy?

Working on a Live System

- ACPO UK guideline 2 states:

> In circumstances where a person finds it necessary to access original data held on a computer or on storage media, that person must be competent to do so and be able to give evidence explaining the relevance and the implications of their actions

- An investigation may be done on the original data
  - Such investigations can be carried out only by a competent person, who can, at the same time, prove the need for such an investigation
- Do not let noncompetent personnel do extractions
- Possess relevant certifications, education…

24

The UK the ACPO (Association of Chief Police Officers), now NPCC, guidelines stated that if you work on a live machine that you must know the 'implications of your actions'.

Live Scene

- Before you start what do you do?
  - Always
  - Always
  - Record the date and time:
    - On your device
    - From the target machine
  - Photos tell the story of 1000 words
- An unrecorded time offset could mess up your evidence
- *Make sure the machines stays ON and unlocked!!!
  - Power settings
  - External tools

A Police Force forgot to do this and in their own evidence it looked like illegal images had been downloaded to the suspects computer AFTER they entered his home

Copyright 2021 – CSITech Ltd

Encrypted?

- Why would you want to image the drive live
- It's slow!
- Is the drive encrypted, are there encrypted files open?

- TCHunt
- Zeroview
- Elcomsoft Encrypted Disk Hunter (EEDH)
- Magnet Encryption Detection (EDD)

Tchunt is a great little tool that detects files that are actually TrueCrypt containers. It tends to fail when a container is under 5Meg

Works by checking the following file attributes:
File size modulo 512 must equal 0
At least 19KB in size (although in practice this is set to 15 MB)
File must pass a Chi Squared distribution test
File must not contain a common file header

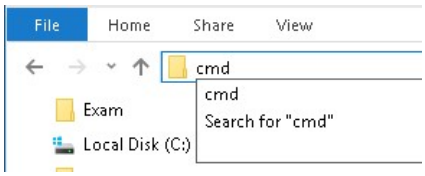
Zeroview gives you a live view of the MBR. All full disk encryption programs 'tag' the MBR. For example TrueCrypt uses the tag TrueCrypt and then the version number.

Bitlocker uses TCPA.

Copyright 2021 – CSITech Ltd

TCHunt

- Command line tool to hunt for TrueCrypt containers
- Uses a number of techniques
- Only really works on containers over 5 meg
- On key:
  - General Tools – Encryption Detection
  - Select path (address line) in explorer and type CMD
    - A CMD window will pop up in that exact folder



Copyright 2021 – CSITech Ltd

TCHunt

- Simple command

```
tchunt -d c:\
```

- Will see a lot of “Access denied”

- To get rid of errors, sent error output to nothing

```
tchunt -d c:\ 2> nul
```

- Look for ”Suspect_File”

Output from a console (Command Prompt) application or command is often sent to two separate streams. The regular output is sent to Standard Out (STDOUT) and the error messages are sent to Standard Error (STDERR). When you redirect console output using the ">" symbol, you are only redirecting STDOUT. In order to redirect STDERR you have to specify "2>" for the redirection symbol. This selects the second output stream which is STDERR.

Copyright 2021 – CSITech Ltd

Zeroview

- General Tools – Encryption Detection
  - Run Zeroview as Admin
  - Provides view of the MBR
  - Full Disk Encryption tags the MBR

Zeroview gives you a live view of the MBR. All full disk encryption programs 'tag' the MBR. For example TrueCrypt uses the tag TrueCrypt and then the version number.

UEFI machines uses GUID partition table. (GPT). Research is ongoing.

Copyright 2021 – CSITech Ltd

Zeroview

- Can look up whether Bitlocker is running on a UEFI system
  - From Admin command shell
  - Type `manage-bde –status c:`
  - Just for fun:
  - Type `manage-bde -protectors -get C:`
    - Will get you all the “ulocking mechanisms”
      - TPM, TPM+PIN, Startup key, TPM+PIN+Startup key, Password….

Zeroview gives you a live view of the MBR. All full disk encryption programs ‘tag’ the MBR. For example TrueCrypt uses the tag TrueCrypt and then the version number.

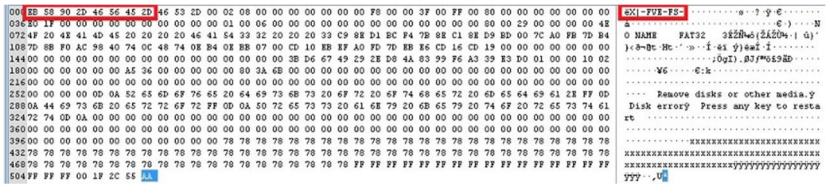
UEFI machines uses GUID partition table. (GPT). Research is ongoing.

While you have Zeroview open, you can also view each of the Partitions as they can provide useful info as well.

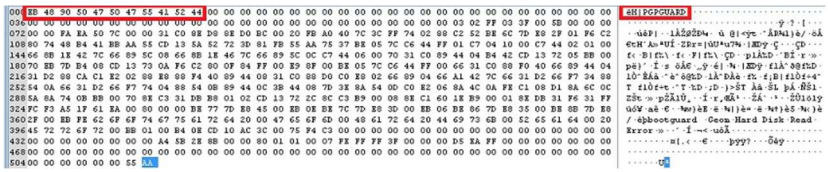
Copyright 2021 – CSITech Ltd

Zeroview

- Bitlocker (Windows 7+)



- PGP



Copyright 2021 – CSITech Ltd

Zeroview

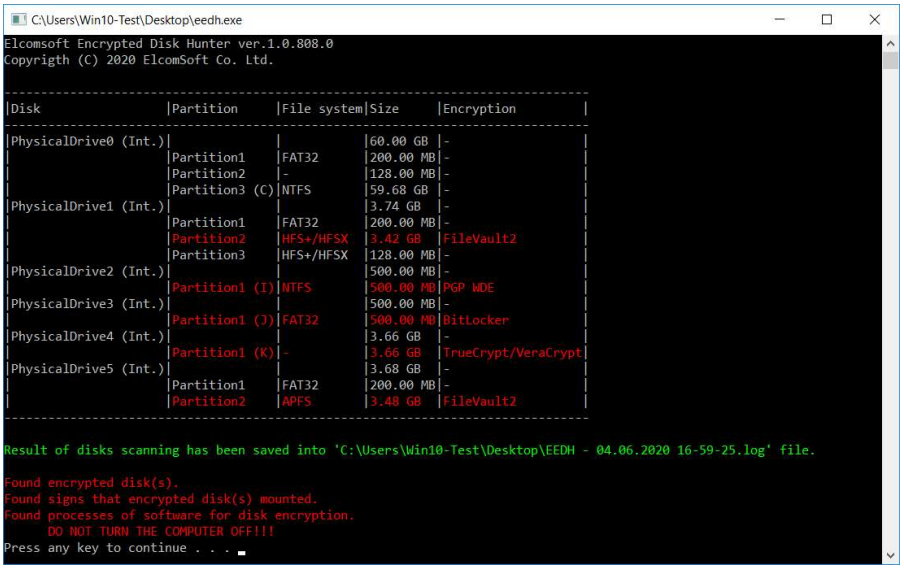
- Bitlocker – Offset 0 – ëR| FVE FS
- Check Point – Offset 90 – Protect
- Symantec Endpoint – Offset 6 – PCGM
- McAfee Safeboot – Offset 3 – SafeBoot
- Sophos – Offset 119 – SGM400
- Symantec PGP – Offset 3 - PGPGUARD
- Apple Filevault – Offset 0 - encrdsa

Copyright 2021 – CSITech Ltd

Elcomsoft Encrypted Disk Hunter

- Discover the presence of encrypted volumes when performing live system analysis
- Free, double click to run or portable command-line tool support
- Supported encryptions:
  - TrueCrypt/VeraCrypt, BitLocker, PGP WDE, FileVault2, and LUKS
- Full blog article:
  - https://blog.elcomsoft.com/2020/07/live-system-analysis-discovering-encrypted-disk-volumes/

Elcomsoft Encrypted Disk Hunter



Volatile Data

- We used to extract volatile data from the computer whilst its running
- Open command shell (Start – cmd)
  - `netstat -na` – network connections
  - `ipconfig /all` – network adapters
  - `systeminfo` – system info ☺
  - `tasklist` – list of running tasks
  - Etc…

https://technet.microsoft.com/en-gb/library/bb490890.aspx - Command-line reference A-Z

Pro-tip: ipconfig /displaydns will get you all mapped drives and network shares, very cools if doing a corporate investigation

Copyright 2021 – CSITech Ltd

Tool Testing and other things

What's going on?

Copyright 2021 – CSITech Ltd

Why test?

- What does a tool do?
- Is a tool trustworthy?
- Can you demonstrate changes made to a system when running a particular tool

As investigators we spend a lot of time using software tools but rarely think of testing what happens when they are run. Lack of time is also a major issue. Significant problems have been found in Encase and FTK in the past (in Guidance case, usually followed up by a legal letter to the researcher). In the case of a tool that is going to the used on a targets computer, it is vital to know what effect it has had on the machine.

Copyright 2021 – CSITech Ltd

Why test?

- Principle 2 or ACPO UK guidelines states:
  - ...that person must be competent to do so and be able to give evidence explaining the relevance and the implications of their actions.

The UK the ACPO (Association of Chief Police Officers) guidelines state that if you work on a live machine that you must know the 'implications of your actions'. If you run FTK lite, dd or any other tool, what changes does it make to the computer and do those changes have legal ramifications?

Copyright 2021 – CSITech Ltd

Why test?

- If we Run Netstat
  - Is it a trusted binary?
  - How can we tell?
  - What does it do to the system?

Netstat is a simple yet critical tool built into all Windows and Linux systems. It simply reports connections to the network and includes the local IP address, the remote address, the ports used and whether the connection is currently established. Remember that 127.0.0.1 is localhost – you!

A binary is a generic name for a self-contained executable file that does not need any other files to be installed for it to function.

Copyright 2021 – CSITech Ltd

Why test?

- Can also use this testing knowledge to understand dark recesses of a system
- Can carry out trojan and hacker investigations
  - Open command shell `cmd`
  - Run `netstat -na`
  - What did it just do?
  - Did it change system at all?

We will be using netstat type commands to extract network connections from RAM. This can be critical in identifying programs or processes that are communicating with the outside world that shouldn't be. This can help us find dodgy processes but also identify command and control centers.

Copyright 2021 – CSITech Ltd

Give it a try

- Open Windows Task Manager - Performance
  - Run `Netstat`
  - Processes Tab
  - Run `Netstat`
  - Applications Tab
  - Run `Netstat`

Copyright 2021 – CSITech Ltd

Process Explorer

- Sysinternals tool (now Microsoft)
- Provides detailed information about each process running on the computer
- Can see each DLL loaded by each process
- Can carry out detailed performance analysis

- General tools\Process Explorer
- Run as administrator

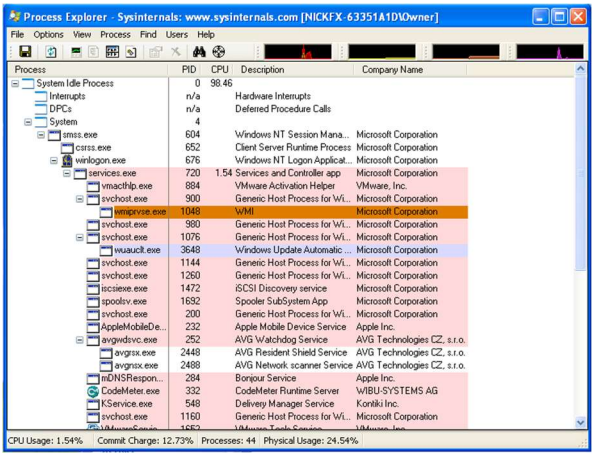
Sysinternals started as a collection of Windows analysis tools written by Mark Russinovich. He was eventually bought out by Microsoft although he seems to have free reign as to what tools he releases.

https://technet.microsoft.com/en-us/sysinternals/bb842062
Twitter - @systinternals

Copyright 2021 – CSITech Ltd

Process Explorer

- Particularly helpful in malware cases
  - Run it lets have a look



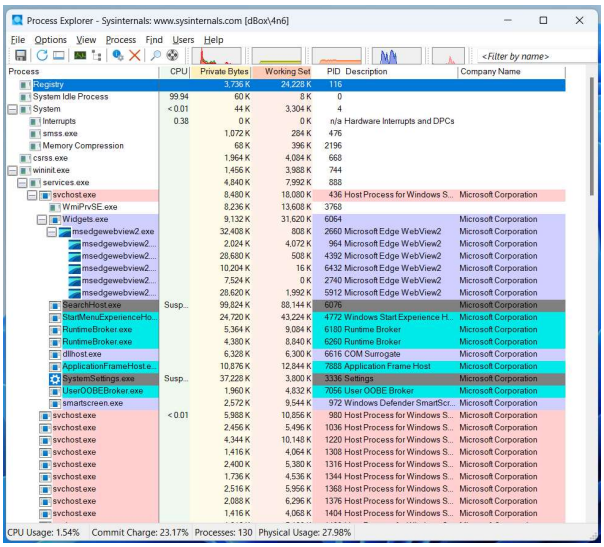
This tool can help us to see all running processes and services. Malware often runs as system name but running from the wrong location or with the wrong parent. A good example is svchost.exe. About 5 are usually running on a Windows box and so it can be easy to sneak in a 6th to look legitimate. We will learn a lot more about this when we get into the RAM later in the course.

Copyright 2021 – CSITech Ltd

Process Explorer

- Tree shows nested processes and services

Pink = Services



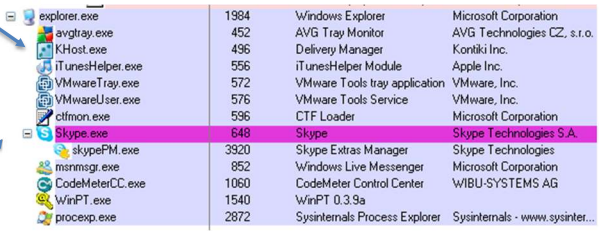
A process is a program running under the logged in user. As service is available to all users on the system.

Copyright 2021 – CSITech Ltd

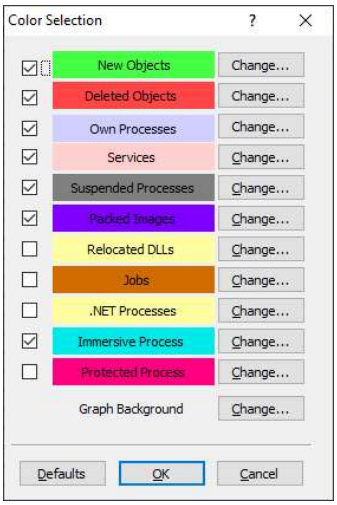
Process Explorer

- Option – configure colors

Blue = Processes running under same profile as Process Explorer



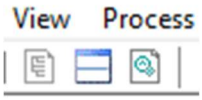
Purple = Process is packed

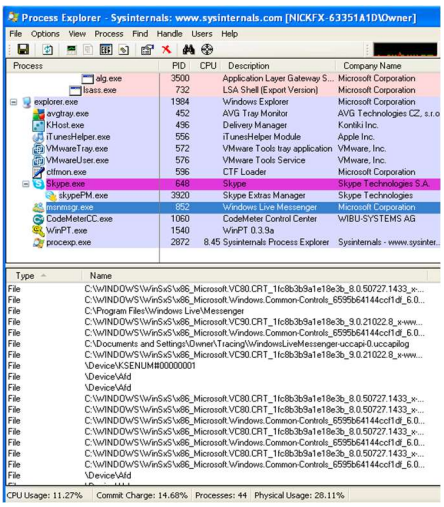


Copyright 2021 – CSITech Ltd

Process Explorer

- View lower pane to enumerate all the files and Reg keys used by the process

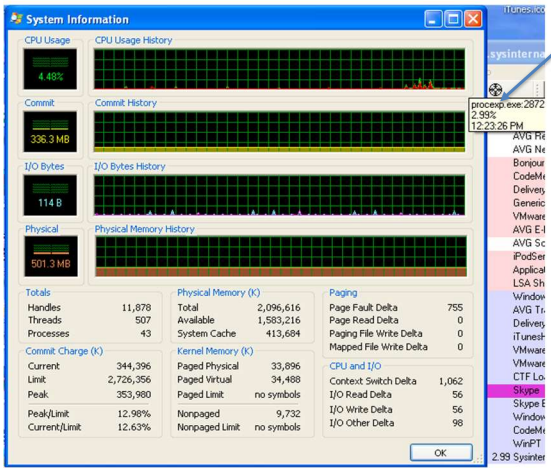




The vast majority of programs, especially on a Windows system, need DLL's, registry entries and other files in order to run correctly. This view can help us to see what files are in use by the process.

Copyright 2021 – CSITech Ltd

Process Explorer



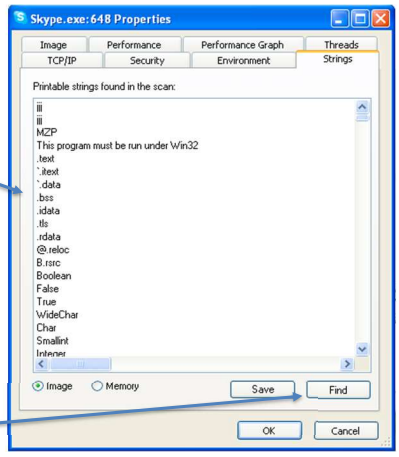
Mouse over graph to show what process is causing peaks

Ever wondered why your machine is running slow? Peaks in processor or RAM activity can be traced back to the process at fault. This can be very useful.

Copyright 2021 – CSITech Ltd

Process Explorer

- Double-click process
  - Chrome or IE
  - Strings



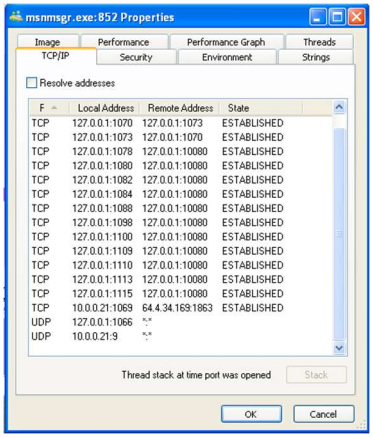
Can search for embedded http address etc

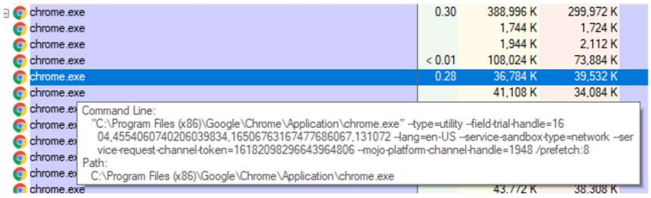
We will be using the Strings command many times during the course. It simply looks through a binary file and extracts all ACSII strings. When looking at malware it is amazing how often the programmer puts in his handle (code name), perhaps an email address or even hardcoded IP addresses. This can often be found by looking through strings.

Copyright 2021 – CSITech Ltd

Process Explorer

- Double-click process
  - TCP/IP
- Chrome
  - Chose 4th of 5th one down
  - Only one process will handle network
  - Hover over the process and search for ***type=network





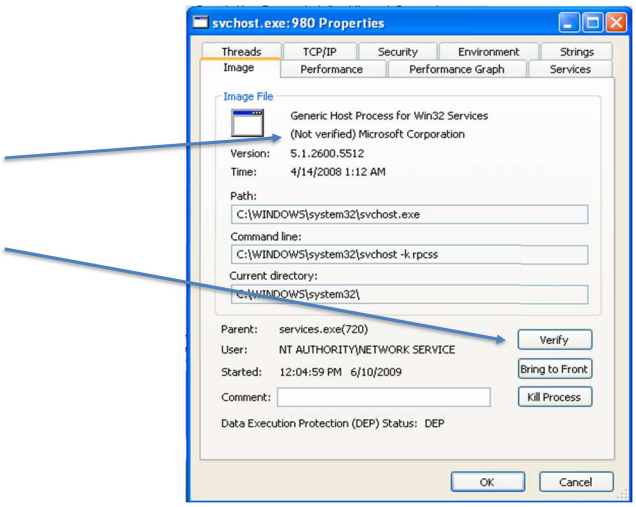
Just like Netstat this pane will tell us if the process has any network connections associated with it. Remember that a web browser with multiple tabs open will have multiple network connections referenced here.

For firefox, the main process that starts all other processes will be the one that has all the network connections. Other will have just some but probably related to the main process, like a reference.

Copyright 2021 – CSITech Ltd

Process Explorer

- Checking that MS processes are legitimate
- Image - Verify



There is also a cool program called process hacker – you can find it in your general tools. Has more or less the same options but also some additional options that might be useful. Main difference is that is open-source so no copyright issues.

Copyright 2021 – CSITech Ltd

Process Monitor

- Sysinternals tool (now Microsoft)
- Provides more detailed process activity
- File accesses
- Registry activity
- In other words - system changes

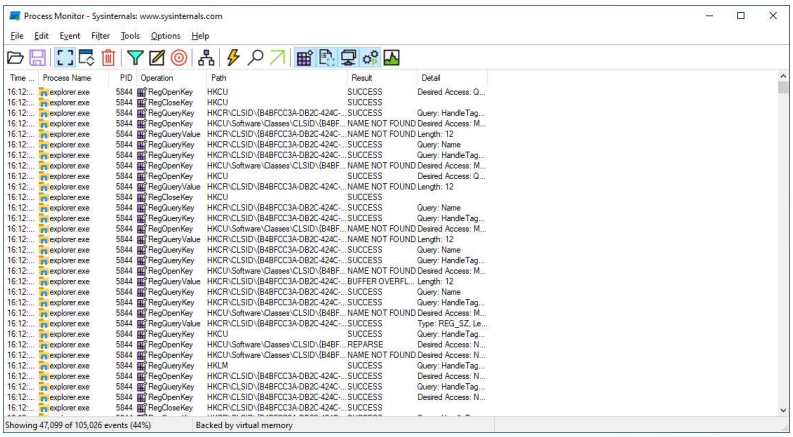
- General tools\Process Monitor
- Run as administrator

Process Monitor is in many ways a more powerful tool than Process Explorer. It can detail every system event from every process and service in real time. It can be very verbose! Using filters can be critical.

Copyright 2021 – CSITech Ltd

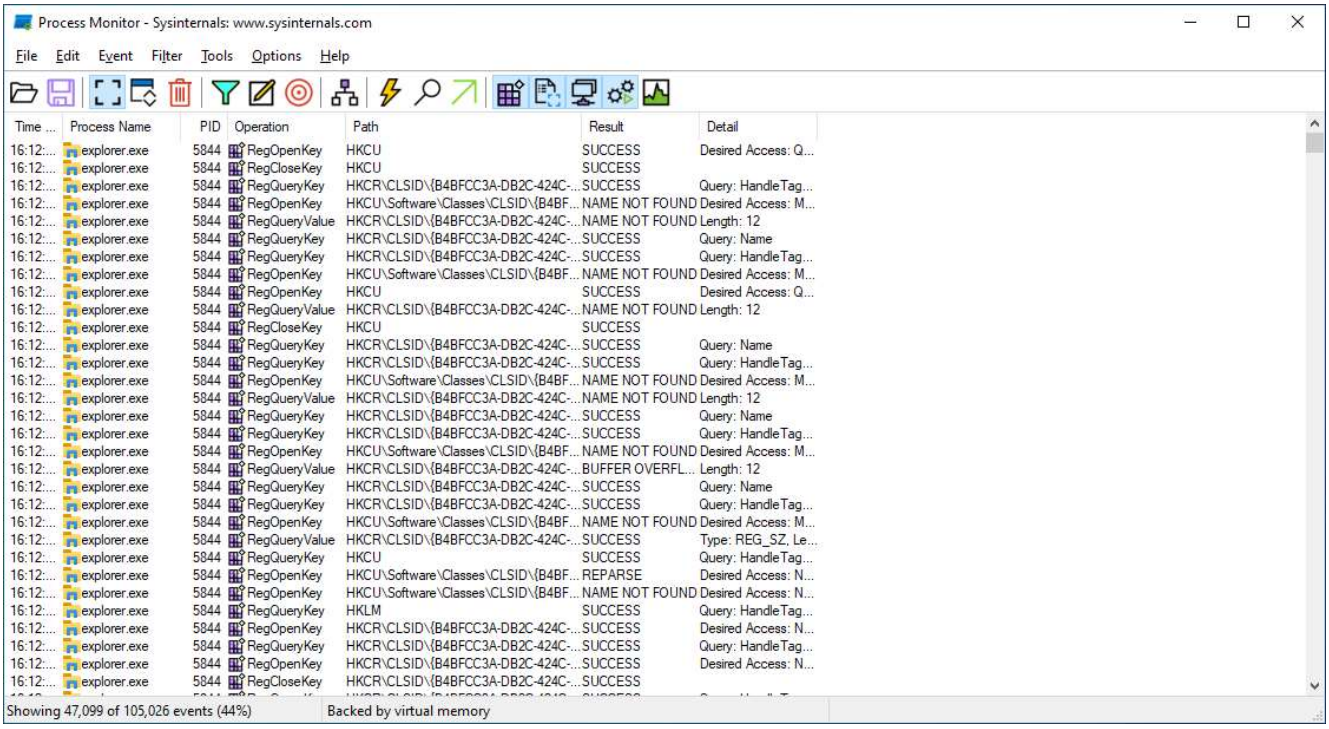
Process Monitor

- Run Process Monitor
  - Wonder at the vast array of processes running with no seeming activity!



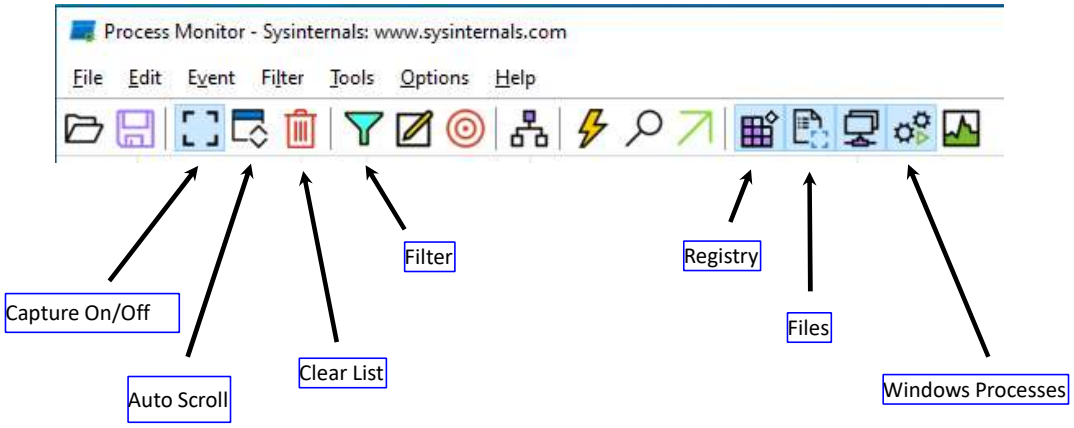
When we consider a computer doing ‘nothing’ we will see that a Windows OS is in fact very busy indeed. This helps us realise that we need to consider carefully the implications of working on a live target system. This does give us the tool we need to test what a tool does when we run it though.

Copyright 2021 – CSITech Ltd



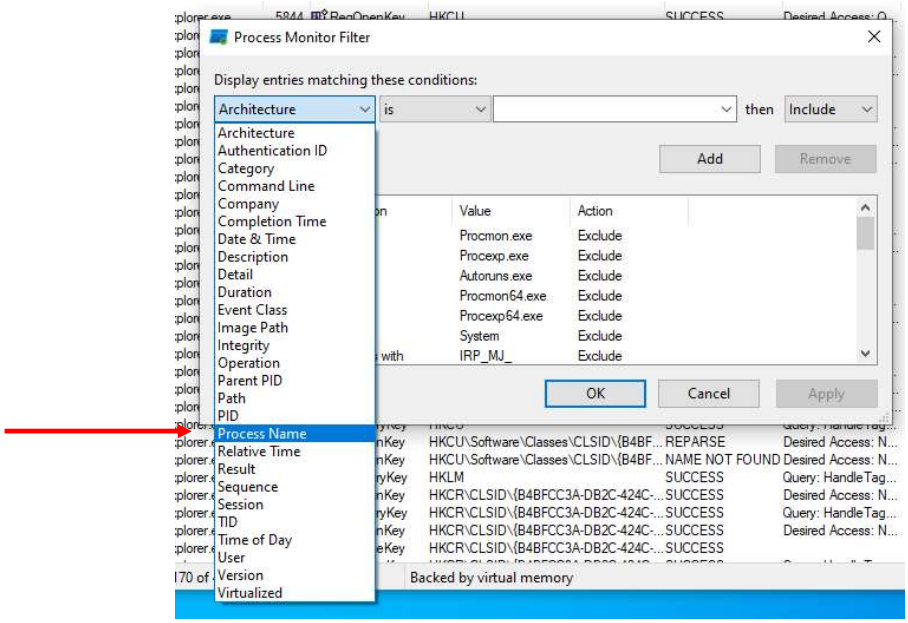
Copyright 2021 – CSITech Ltd

Process Monitor



Copyright 2021 – CSITech Ltd

Process Monitor



Creating filters are vital to be able to make sense of the huge output of data. Most of the time you filter on the name of a process. Simply select Process Name from the list and the 3rd drop down box will self-populate. Simply choose ‘is’ and then the name of the process in the list. Lastly click ‘Add’ and ‘OK’

Copyright 2021 – CSITech Ltd

Process Monitor

- Start Process Monitor
  - Open and CMD shell and run Netstat -na
  - Stop Process monitor
  - Filter by process name (cmd) – is – **cmd.exe** (Add)
  - Note all the system processes
  - Id the ‘touches’ and value changes

Copyright 2021 – CSITech Ltd

Process Monitor

- Test by starting Process Monitor and run:
  - Notepad
  - `Ipconfig /all`
  - Running the FTK Imager
    - What Registry keys are touched
      - Hint - FTK.exe
  - then....

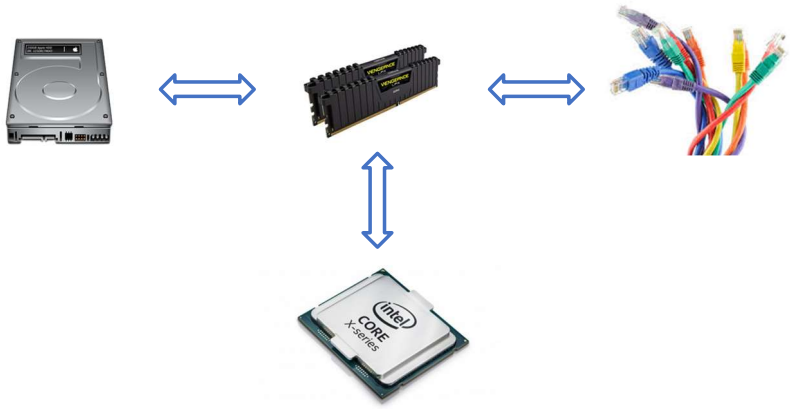
Copyright 2021 – CSITech Ltd

Advanced Memory Analysis

Get Rammed

Copyright 2021 – CSITech Ltd

RAM overview



Main memory contains evidence!!

61

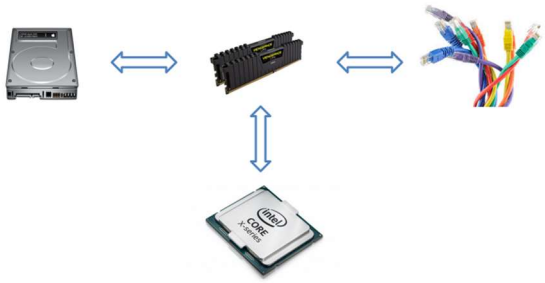
Everything that flows through a computer end up in RAM at some point. Whether that is network traffic, disk traffic, copying from a USB key or just writing pixel data to the screen, everything gets written to a RAM register. This means that evidence will be as prevalent in the RAM as it is on a disk, the difference is that RAM tells you what was happening on the computer when it was imaged, the disk just tells you what was happening.

There is also a lot of data never written to the disk. A good example is network connection data (remember Netstat) which can tell us what a person was looking at, connected to, transferring data to or talking to on Skype. This is evidence that will never be on the disk.

Copyright 2021 – CSITech Ltd

Please remember!

If it's running in the RAM



Main memory contains evidence!!

We will look later at methods used to hide running applications and processes from the OS. However, if it's running its in the RAM, it can be obfuscated but not hidden completely. You will be amazed how easily we identify the rogue processes running in Stuxnet later in the course

Copyright 2021 – CSITech Ltd

Role of Pagefile.sys

- Pagefile essentially a scratchpad
  - Used for overflow
    - Less common with larger RAM now
  - Used for paging currently unneeded or old data which may be needed again
- Sometimes referred to as SWAP file

The Pagefile.sys is a segment of HD space which is dedicated to swapping data with RAM. With larger RAM the need for constant paging to the Pagefile is reduced.

Copyright 2021 – CSITech Ltd

Role of Pagefile.sys

- Inaccurately described as ‘Extended’ RAM
  - Having a garden shed is not ‘Extended’ house

Copyright 2021 – CSITech Ltd

Role of Pagefile.sys

- Lots of types of paging
  - When code is paged from disk usually from the original exe disk location
  - *Changed* code usually goes to Pagefile as cannot go to original exe

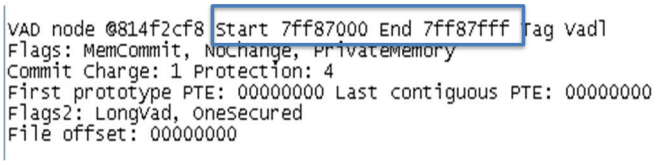
When a program looks to run a piece of code but realises its in the Pagefile, the following happens:

1. Determine the location of the data in secondary storage.
2. Obtain an empty page frame in RAM to use as a container for the data.
3. Load the requested data into the available page frame.
4. Update the page table to show the new data.
5. Return control to the program, transparently retrying the instruction that caused the page fault.

Copyright 2021 – CSITech Ltd

Role of Pagefile.sys

- Later learn about VAD (Virtual Address Descriptor)
  - Data is tagged with a virtual address rather than a physical address
  - Not uncommon to see an address in the VAD tree larger than the RAM size
  - That's a virtual address in Pagefile

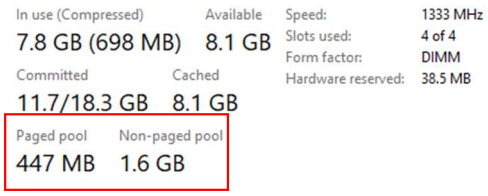


All RAM data is written to addresses. This is not recorded in an index like the MFT so each process space has a Virtual Address Descriptor which tells the OS where the process is. You may sometimes see an address in the VAD which is larger than the size of the RAM. This address is in the Pagefile which 'extends' beyond the RAM.

Copyright 2021 – CSITech Ltd

Windows 10+ Pagefile Features

- Data is being compressed prior to being written to the pagefile
- Encryption of the Pagefile
  - Cannot just string-out the file ☹
- Faster R/W from hard drive
- Pages written out to the disk only 50% as often as previous versions of the OS



Role of Pagefile.sys

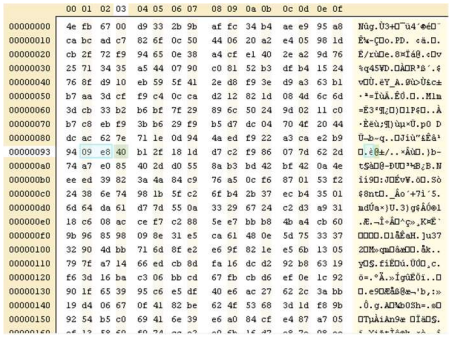
If its in Pagefile it *WAS* current

If its in RAM it is current *NOW*

Copyright 2021 – CSITech Ltd

RAM overview

- Considerable amount of intel/evidence in RAM
  - Internet History
  - Clipboard
  - OS artefacts
  - Active connections
  - Running programs and processes
  - Malware
  - Passwords/seed words etc



	00 01 02 03	04 05 06 07	08 09 0a 0b	0c 0d 0e 0f	
00000000	4e fb 67 00	d9 33 2b 9b	af fc 34 b4	ae e9 95 a8	Nûg.Ù3+□¯ü4´®é□˜
00000010	ca bc ad c7	82 6f 0c 50	44 06 20 a2	e4 05 98 1d	Ê¼-Ç□o.PD. ¢ä.□.
00000020	cb 2f 72 f9	94 65 0e 38	a4 cf e1 40	2e a2 9d 76	Ë/rù□e.8¤Ïá@.¢□v
00000030	25 71 34 35	a5 44 07 90	c0 81 52 b3	df b4 15 24	%q45¥D.□À□R³ß´.$
00000040	76 8f d9 10	eb 59 5f 41	2e d8 f9 3e	d9 a3 63 b1	v□Ù.ëY_A.Øù>Ù£c±
00000050	b7 aa 3d cf	f9 c4 0c ca	d2 12 82 1d	08 4d 6c 6d	·ª=ÏùÄ.ÊÓ.□..Mlm
00000060	3d cb 33 b2	b6 bf 7f 29	89 6c 50 24	9d 02 11 c0	=Ë3²¶¿□)□lP$□..À
00000070	b7 c8 eb f9	3b b6 29 f9	b5 d7 dc 04	70 4f 20 44	·Èëù;¶)ùµ×Ü.pO D
00000080	dc ac 62 7e	71 1e 0d 94	4a ed f9 22	a3 ca e2 b9	Ü¬b~q..□Jíù"£Êâ¹
00000093	94 09 e8 40	b1 2f 18 1d	d7 c2 f9 86	07 7d 62 2d	□.è@±/..×Âù□.}b-
000000a0	74 a7 e0 85	40 2d d0 55	8a b3 bd 42	bf 42 0a 4e	t§à□@-ÐU□³½B¿B.N
000000b0	ee ed 39 82	3a 4a 84 c9	76 a5 0c f6	87 01 53 f2	îí9□:J□Év¥.ö□.Sò
000000c0	24 38 6e 74	98 1b 5f c2	6f b4 2b 37	ec b4 35 01	$8nt□._Âo´+7ì´5.
000000d0	6d 64 da 61	d7 7d 55 0a	33 29 67 24	c2 d3 a9 31	mdÚa×}U.3)g$ÂÓ©1
000000e0	18 c6 08 ac	ce f7 c2 88	5e e7 bb b8	4b a4 cb 60	.Æ.¬Î÷Â□^ç»¸K¤Ë`
000000f0	9b 96 85 98	09 8e 31 e5	ca 61 48 0e	5d 75 33 37	□□□□.□1ÅÊaH.]u37
00000100	32 90 4d bb	71 6d 8f e2	e6 9f 82 1e	e5 6b 13 05	2□M»qm□âæ□□.åk..
00000110	79 7f a7 14	66 ed cb 8d	fa 16 dc d2	92 b8 63 19	y□§.fíË□ú.Ü□Ò□¸c.
00000120	f6 3d 16 ba	c3 06 bb cd	67 fb cb d6	ef 0e 1c 92	ö=.ºÃ.»ÍgûË□ï..□
00000130	90 1f 65 39	95 c6 e5 df	40 e6 ac 27	62 2c 3a bb	□.e9□Æå□@æ¬'b,:»
00000140	19 d4 06 67	0f 41 82 be	62 4f 53 68	3d 1d f8 9b	.□.g.A□¾bOSh=.ø□
00000150	92 54 b5 c0	69 41 6e 39	e6 a0 84 cf	e4 87 a7 05	□Tµ□iAn9æ □Ïä□§.

Because everything ends up in RAM at some point it contains significant evidence/intelligence/history.

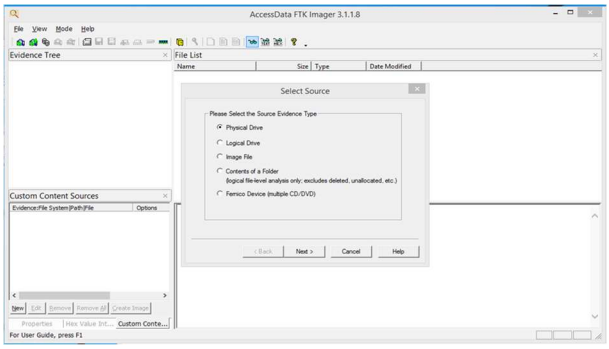
This is especially true for Malware. Malware may be able to hide from the OS but at some point it has to unpack and show itself. It can't hide from RAM!

Copyright 2021 – CSITech Ltd

HEX viewer

- FTK Imager from Accessdata
  - Version on your key
  - Can download from http://accessdata.com/product-download?/support/product-downloads

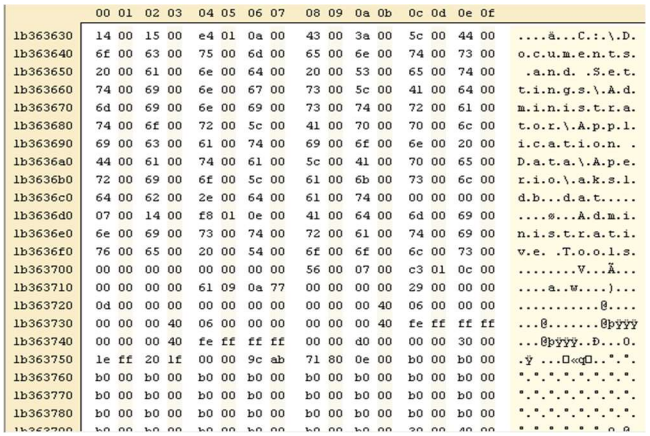
- HxD
  - Free
  - https://mh-nexus.de/en/hxd/



Copyright 2021 – CSITech Ltd

RAM overview

- Often structure amongst the chaos
- Can carve like a disk image



	00 01	02 03	04 05	06 07	08 09	0a 0b	0c 0d	0e 0f	
1b363630	14 00	15 00	e4 01	0a 00	43 00	3a 00	5c 00	44 00	....ä...C.:.\.D.
1b363640	6f 00	63 00	75 00	6d 00	65 00	6e 00	74 00	73 00	o.c.u.m.e.n.t.s.
1b363650	20 00	61 00	6e 00	64 00	20 00	53 00	65 00	74 00	.a.n.d. .S.e.t.
1b363660	74 00	69 00	6e 00	67 00	73 00	5c 00	41 00	64 00	t.i.n.g.s.\.A.d.
1b363670	6d 00	69 00	6e 00	69 00	73 00	74 00	72 00	61 00	m.i.n.i.s.t.r.a.
1b363680	74 00	6f 00	72 00	5c 00	41 00	70 00	70 00	6c 00	t.o.r.\.A.p.p.l.
1b363690	69 00	63 00	61 00	74 00	69 00	6f 00	6e 00	20 00	i.c.a.t.i.o.n. .
1b3636a0	44 00	61 00	74 00	61 00	5c 00	41 00	70 00	65 00	D.a.t.a.\.A.p.e.
1b3636b0	72 00	69 00	6f 00	5c 00	61 00	6b 00	73 00	6c 00	r.i.o.\.a.k.s.l.
1b3636c0	64 00	62 00	2e 00	64 00	61 00	74 00	00 00	00 00	d.b...d.a.t.....
1b3636d0	07 00	14 00	f8 01	0e 00	41 00	64 00	6d 00	69 00	....ø...A.d.m.i.
1b3636e0	6e 00	69 00	73 00	74 00	72 00	61 00	74 00	69 00	n.i.s.t.r.a.t.i.
1b3636f0	76 00	65 00	20 00	54 00	6f 00	6f 00	6c 00	73 00	v.e. .T.o.o.l.s.
1b363700	00 00	00 00	00 00	00 00	56 00	07 00	c3 01	0c 00	........V...Ã...
1b363710	00 00	00 00	61 09	0a 77	00 00	00 00	29 00	00 00	....a..w....)...
1b363720	0d 00	00 00	00 00	00 00	00 00	00 40	06 00	00 00	...........@....
1b363730	00 00	00 40	06 00	00 00	00 00	00 40	fe ff	ff ff	...@.......@þÿÿÿ
1b363740	00 00	00 40	fe ff	ff ff	00 00	d0 00	00 00	30 00	...@þÿÿÿ..Ð...0.
1b363750	1e ff	20 1f	00 00	9c ab	71 80	0e 00	b0 00	b0 00	.ÿ ...□«q□..°.°.
1b363760	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	°.°.°.°.°.°.°.°.
1b363770	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	°.°.°.°.°.°.°.°.
1b363780	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	b0 00	°.°.°.°.°.°.°.°.

When viewed in a hex viewer such as FTK Imager you can see that it is fundamentality chaos with little structure or recognizable elements. Also with no ‘MFT’ to guide us, it’s very hard to find what you can looking for.

By looking carefully one can see fragments of human readable words, sentences and paths etc.

Copyright 2021 – CSITech Ltd

RAM overview

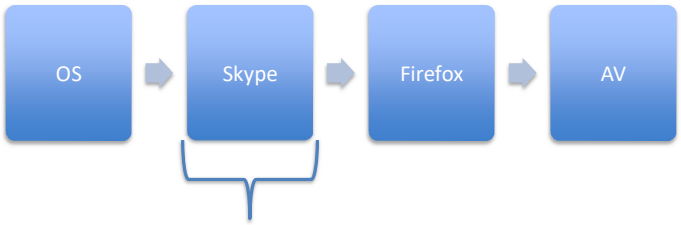
Action	% RAM unchanged	
	256 MB RAM	512 MB RAM
Start	100.0	100.0
Idle for 1 hour	90.4	96.7
Idle for 2 hours	79.7	96.1
DD (live acquisition)	76.9	89.8
Idle for 15 hours	74.8	85.6
WFT (live response)	67.2	69.4

Effects on main memory, according to Walters and Petroni (2006)

It was assumed for many years that RAM lost any useable data very quickly as it was in use and data structures were being constantly overwritten. This was until this research by Walters and Petroni which showed that even under heavy use RAM stayed pretty well untouched. RAM actually is quite static. One of the reasons is the way that processes work.

Copyright 2021 – CSITech Ltd

RAM isn't random!



Process space – everything happens here

Although it doesn't look exactly like this, essentially Processes take up a defined amount of space. So you have 2 different elements, the executable file, loaded into RAM and the Process 'space' which is the chuck of RAM space allocated to run the executable and manage files that it needs. So files needed by Skype, for example, will be loaded into the Skype Process space.

Copyright 2021 – CSITech Ltd

RAM isn't random!

When the game reports it using 100% RAM, Its only 100% of the Process space
This means that nothing else is overwritten.

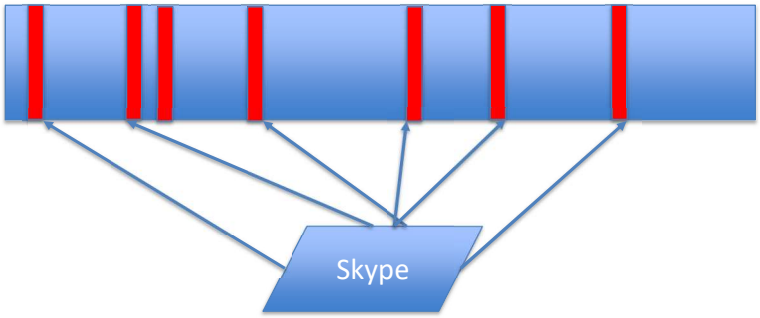
77

We had a odd situation where a game we were playing reported that 100% of RAM was being used, however, after we finished, data could be recovered from before the game was played. This was because it was only using 100% of its process space, not the whole of RAM.

Copyright 2021 – CSITech Ltd

In Reality….

- Processes in RAM really look like this



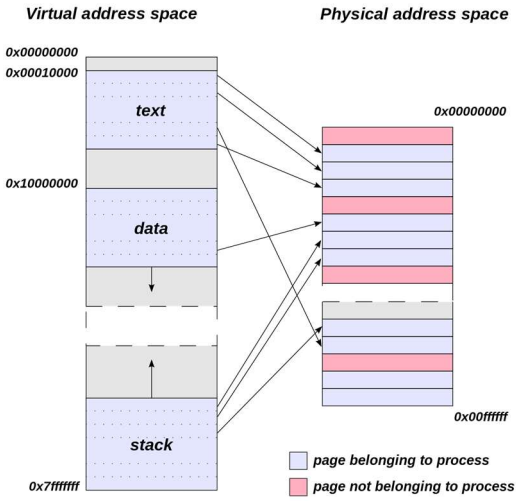
In reality each process is scattered across the RAM. A table called the VAD keeps track of it all and we will look in detail at that later.

Copyright 2021 – CSITech Ltd

Virtual vs. physical address space

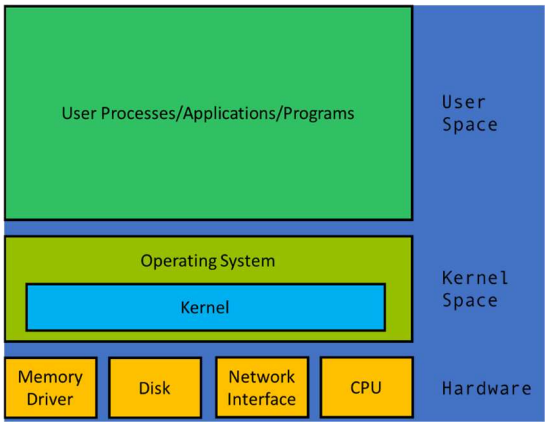
- We will be seeing this through the class…

- Briefly:
  - Virtual address space
    - Every process has their own
    - System and process address spaces reside here
  - Physical address space
    - Processes have no clue what's going on here
    - The "real" location in the memory
    - System only



RAM overview

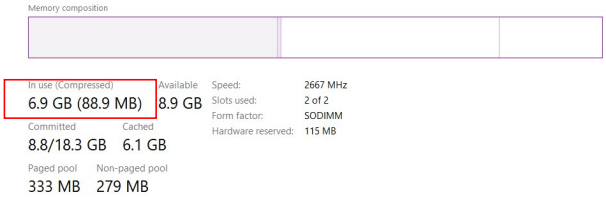
- From WinXP SP2/Vista etc RAM in the Kernel
- All modern RAM dumping tools now have this capability.
- Beware older tools such as mdd which only supported 32bit



Copyright 2021 – CSITech Ltd

RAM Compression

- Win10 now uses memory compression
- Where possible, excess data is compressed and stored in memory, rather than being “paged out” to page file



- Compression of infrequently used data stored in RAM
- Reduces the # of Reads/Writes
- Reduces Use of Memory by Apps

https://github.com/fireeye/win10_volatility - volatility support for RAM compression

https://channel9.msdn.com/Blogs/Seth-Juarez/Memory-Compression-in-Windows-10-RTM

Copyright 2021 – CSITech Ltd

RAM IMAGING

Copyright 2021 – CSITech Ltd

RAM Imaging

- There are many tools for imaging or “dumping” the RAM of a system
- They will differ for every OS – Windows, Linux and MAC
  - Each will have its own challenges
- Windows
  - There are few types of RAM dumps:
    - **RAW Format** – Extracted from a live environment, bit by bit copy
    - **Crash Dump** – Information gathered by the operating system
    - **Hibernation File** – A saved snapshot that your operating system can return to after hibernating
    - **Page File** – This is a file that stores similar information that is stored in your system RAM
    - **VMWare Snapshot** – This is a snapshot of a virtual machine, which saves its state as it was at the exact moment that the snapshot was generated

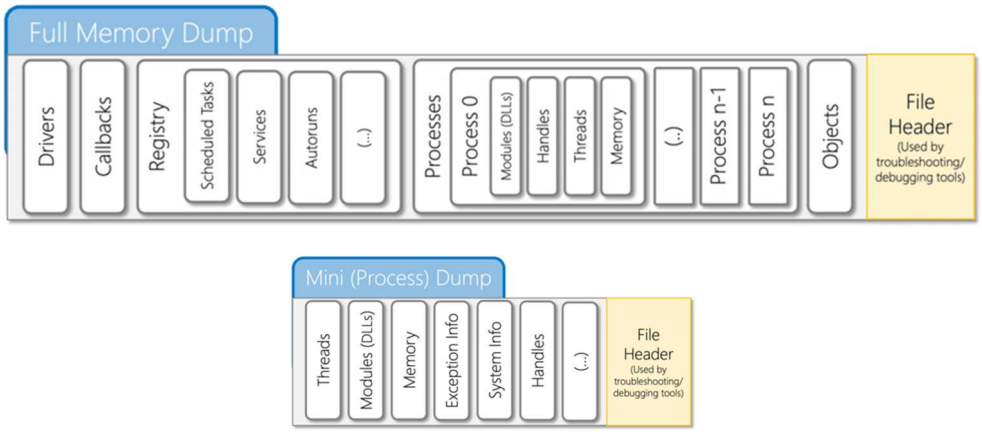
RAM Imaging - Windows

- **RAW Format**
  - Bit-by-bit copy of the data in the RAM
  - No additional data captures
  - Can be analysed in most of the tools
  - Has some limitations
  - Most common type of extraction
- **Crash Dump**
  - Contains full copy of the memory + additional information in the header
  - Interoperable with other tools like crash, drgn, or WinDbg
  - Tools can also “simulate” windows crash dump
  - Has additional data about the system – OS version, timestamp, CPU type…
  - Can be of a single application/process

https://www.magnetforensics.com/blog/full-memory-crash-dumps-vs-raw-dumps-which-is-best-for-memory-analysis-for-incident-response/

RAM Imaging - Windows

- **Crash Dump**
  - Superior to RAM memory dumps
  - Foundation of troubleshooting for all OS'es



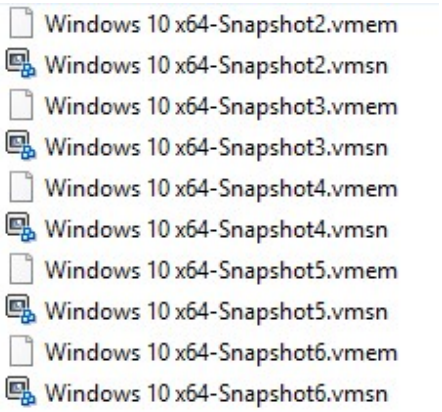
https://www.magnetforensics.com/blog/full-memory-crash-dumps-vs-raw-dumps-which-is-best-for-memory-analysis-for-incident-response/

RAM Imaging - Windows

- **Hibernation File**
  - Saved snapshot that your operating system can return to after hibernating
  - Does **NOT** contain **ALL** the memory only **USED** memory
  - No carving as no “unallocated” data
- **Page File**
  - This is a file that stores similar information that is stored in your system RAM
  - Recently used data
  - Explained earlier in the class

RAM Imaging - Windows

- Virtual machines Snapshot
  - Can treat it as a **raw** memory dump
  - Multiple .vmem files if you have multiple snapshots of the same VM
  - Create a memory dump the same way like on a regular OS (same tools)
- Extensions:
  - VMWare workstation - .vmdk, .vmem, .vmsn
  - Hyper-V - .bin, .vsv, .vhd, .vhdx, .avhd, .avhdx
  - VirtualBox - .vdi, .sav, .vbox
  - Parallels - .pvm, .hds (inside the .pvm file), .hdd



This is a really quick way of doing RAM research. Make your changes or install your malware, whatever your plans are, then suspend the VM and browse to the .vmem file which is in the VM container. You can run tools directly against this file. However, I recommend making a copy of it to be safe.

There are a number of viruses that detect the presence of the hypervisor which indicates that they are running in a virtual machine. Check out - http://goo.gl/ttyocW
.vmsn – vmware snapshot file

RAM Imaging – Linux and MAc

- Linux
  - A lot of issues as the dumping program will depend on the target OS kernel version and OS version
    - There are 10000 version of Linux in the wild
  - Requires root access
  - Some tool support – Lime, AVLM
  - Will be covered later in the class

- Mac
  - Similar issues as with Linux
  - New M- silicone – not possible
  - Will be covered later in the class

RAM Imaging - Dumpit

- Matthieu Suiche has released a tool called DumpIt
- Double click tool
- By default will create a **CRASH** dump and additional *.json file with dump metadata
- Use flag /T to specify the RAM dump type
- READ THE README.txt 😁
- In General tools\DumpIt
  - Make a folder on the root of C: called RAM, all RAM dumps should go in there
  - Give it a try!

Best up to 8 Gig.

Copyright 2021 – CSITech Ltd

RAM Imaging - Dumpit

- Has additional tools
  - **Z2Dmp** - Decompresses Microsoft crash dump files
  - **Bin2Dmp** - Converts raw memory dump images into Microsoft crash dump files
  - **Dmp2Bin** - Converts Microsoft crash dump files into raw memory dump images
  - **Dmp2Json** - Extracts metadata from Microsoft crash dump files
  - **Hibr2Dmp** - Convert Microsoft hibernation files into Microsoft crash dump files
  - **Hibr2Bin** - Enables users to uncompress Windows hibernation file
- Possibility to enable live kernel debugging session

Best up to 8 Gig.

Copyright 2021 – CSITech Ltd

RAM Imaging - Belkasoft

- Best known for Belkasoft evidence centre
- Simple tool
- GUI and Command Line
- Double click or run straight from terminal or script
- Creates only **RAW** memory dumps
- `Ramcapture <nameoffile>`

Copyright 2021 – CSITech Ltd

RAM Imaging - Magnet

- **Magnet RAM capture**
  - Can export captured memory data in **RAW**
  - Supports RAM acquisition from Windows 10 systems that have Virtual Secure Mode enabled
  - CLI support

- **Magnet Process capture**
  - Capture memory from individual running processes
  - Monitor mode
    - Continuously capture memory from selected processes, every X number of seconds
    - Analyzing malware or profiling certain processes

VSM is a Hyper-V container that isolates the lsass.exe process from the running Windows 10 machine
Monitoring mode - Useful when analyzing malware or profiling certain processes and wanting to capture the changes in memory after specific actions are taken or events occur

RAM Imaging - winpmem

- Was part of Rekall (google), Velocidex (Velociraptor) and all under Rapid7
- Simple CLI tool
- By default, outputs to AFF4, supports **RAW** dumps

- Start CMD as admin
  - CD into \General tools\Rekal folder and run:
    `winpmem_mini_x64_rc2_v4.0.exe –h` to get all the commands
    winpmem_mini_x64_rc2_v4.0.exe -o <path2mem out file>

- `winpmem_XXXX.exe -o test.raw --volume_format raw -dd –t`

- Add `-p <path2pagefile>`

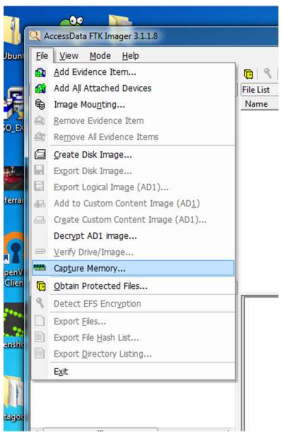
More command switches available, just run `winpmem_v3.3.rc3.exe` –h for help.
Eg.
`winpmem_v3.3.rc3.exe` -e - | nc 192.168.1.1 80
Writes an elf coredump to netcat for network transport.

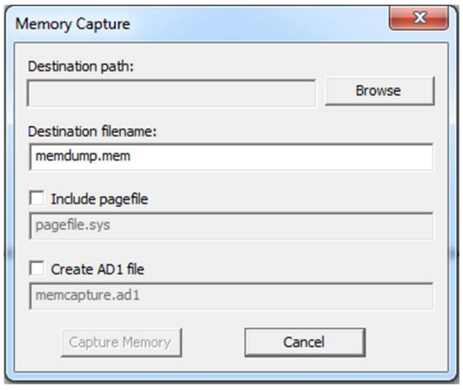
https://github.com/Velocidex/c-aff4/releases

Copyright 2021 – CSITech Ltd

RAM Imaging - FTK

- The great FTK Imager
- You can use it, but please dont!





You can also use FTK from the commandline:

https://ad-pdf.s3.amazonaws.com/Using_Command_Line_Imager.pdf

Copyright 2021 – CSITech Ltd

RAM Imaging

- RAM in a Virtual Machine
  - Browse to find the .vmem file
  - Can treat it as a raw memory dump
  - Malware can, and there are examples in the wild, detect the hypervisor
- With “Malware defense” you can virtualize the disk images and image the RAM
  - Prove that nothing nefarious is running
- Great tool for virtualization: Arsenal Recon Imager
  - https://arsenalrecon.com/

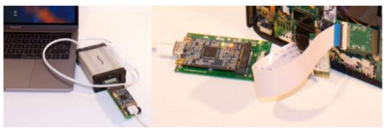
This is a really quick way of doing RAM research. Make your changes or install your malware, whatever your plans are, then suspend the VM and browse to the .vmem file which is in the VM container. You can run tools directly against this file. However, I recommend making a copy of it to be safe.

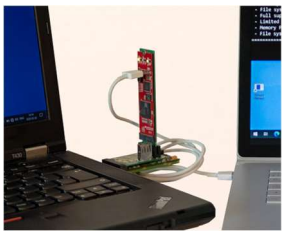
There are a number of viruses that detect the presence of the hypervisor which indicates that they are running in a virtual machine. Check out - http://goo.gl/ttyocW

Copyright 2021 – CSITech Ltd

RAM Imaging

- PCIleech - Ulf Frisk
- Hardware devices to read **and write** from the target system memory
- Using DMA over PCIe
  - Only able to read 4GB of memory natively
  - With kernel module (KMD) can read all
- Using DMA over FPGA
  - Can read ALL memory
- https://github.com/ufrisk/pcileech
- http://blog.frizk.net/

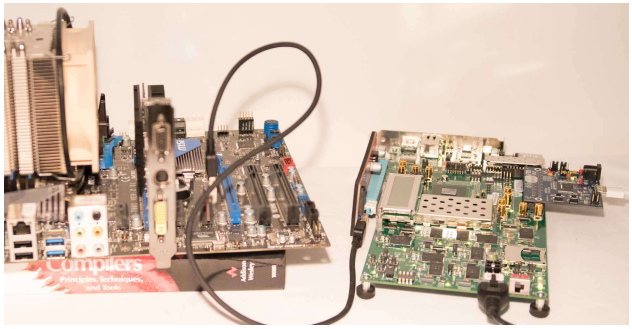


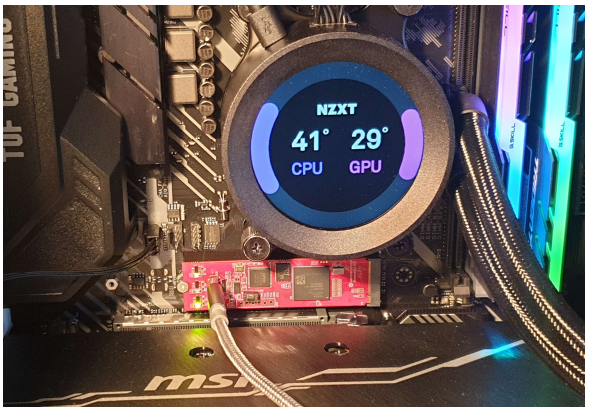


There is a commercial tool – www.windowsscope.com. Expresscard and USB for locked machines - £9k

Copyright 2021 – CSITech Ltd

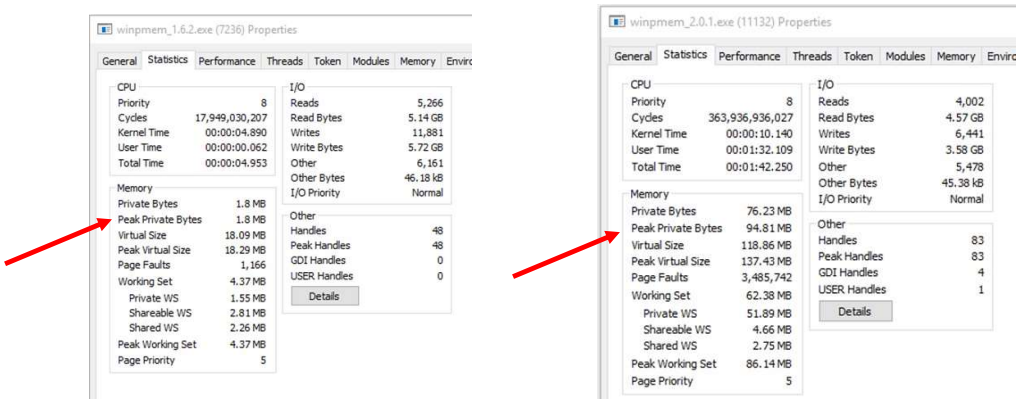
PCIleech – DMA over FPGA





RAM imaging

- **Thou shall test thy forensics tools**



- 1.8MB vs 94.81MB!!!

One good way of getting to know your tools it have a look at process properties and check the Memory - Peak Private Bytes statistics

RAM Imaging – Linux and Mac

- Lime – open source
- Magnet dumpit-linux – open source
- Cellebrite Digital Collector – commercial tool
- Sumuri Recon ITR– commercial tool
- Volexity's Surge Collect – commercial tool
- F-Response – commercial tool
  - Enables a small USB key to be inserted in target
  - Can access memory from a remote computer over the network
  - Can then use any tool to image
  - www.f-response.com
- More on some of those later….

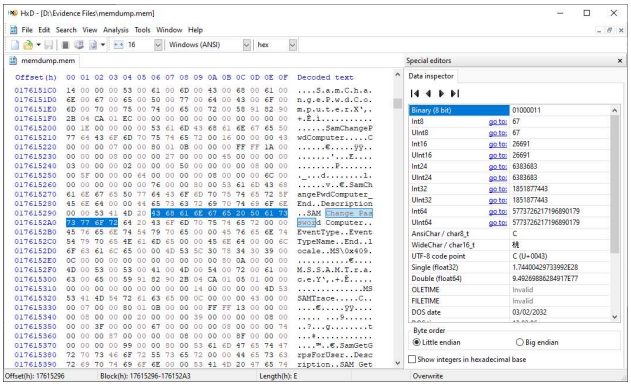
Copyright 2021 – CSITech Ltd

QUICK WINS

Copyright 2021 – CSITech Ltd

RAM Searching

- Use any HEX viewer to search for a specific string, hex, byte….
- Good tools
  - FTK imager, HxD – both on your key in *General tools*
- Takes little time, can give good pointers what can be found



In linux use single or multiple GREP’s to be more precise
Often, you can solve things just using this

Copyright 2021 – CSITech Ltd

RAM Carving

- Foremost is quick and easy
- General tools – Scripting Files
  - foremost -t all -v -i <input file> -o <outputfolder>
  - Example:

```
foremost –t all –v –i c:\Win10.img –o c:\foremost
```

*Output folder must not exist!

Files are structured in the RAM exactly as they are on the disk. Hence, the header of a JPEG file loaded in RAM is the same as the header on the disk. This means that file carvers work as well against a RAM dump as they do on the disk. Foremost by Jesse Kornblum is a quick and easy command line tool to enable carving of files.

Check out the .conf file where you can specify your own file type and headers and footers.

Copyright 2021 – CSITech Ltd

RAM Carving

- Photorec is superb, my free carving tool of choice at the moment
- http://www.cgsecurity.org/wiki/TestDisk_Download
- Carves mounted drives and images

- General tools\Photorec\testdisk-7.2-WIP

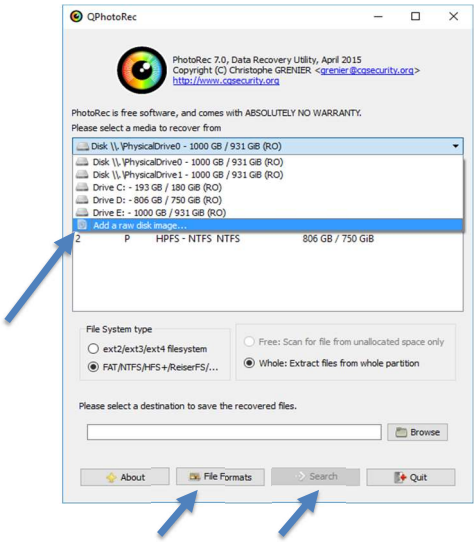
Photorec from CG Security was initially a tool for carving deleted images from drives, SD cards etc. It now carves a huge selection of file types. The only negative is that the file/drive must be mounted for it to work. The following pages show you how to achieve that with a RAM dump.

Copyright 2021 – CSITech Ltd

RAM Carving

- General tools\Photorec\testdisk-7.2-WIP

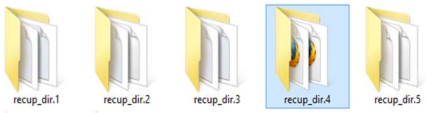
- Run **qphotorec_win.exe** to get the GUI version
- Select dropdown box and “Add raw disk image”
- Select file formats
- Press Search!

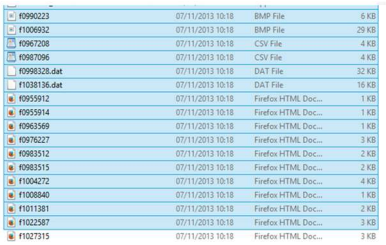


Copyright 2021 – CSITech Ltd

RAM Carving

- Copy Photorec_Sorter from key (General tools\Photorec\PhotoRec_Sorter-v1009) to folder with the recup folders in it
- Double-click Photorec_Sorter





Photorec writes all its files into folders named ‘recup_dir.x’. This can be a bit untidy to search through. Find and copy and run the PhotoRec_Sorter program in the folder that contains the recup folders to sort the file types into their own folders.

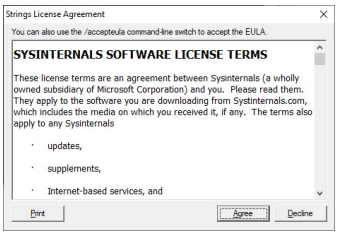
Copyright 2021 – CSITech Ltd

RAM Carving

- Also use Strings to create a password cracking dictionary
- Copy strings64 from General tools\Strings to c:\temp
- Command:
  strings <path2image> > <path2output>

  - Example:
  `strings64 c:\Win10.img > c:\temp\passwords.txt`

*on first run, make sure to accept the EULA



Strings is a create command, originally from Unix. It extracts all the 'human-readable' ASCII text from any file even a binary file such as a RAM Dump. So, if you run Strings against a RAM dump you will extract all the ASCII text which can be useful in its own right to search through. If you are looking for a Skype name, an email address or a unique word this can be helpful.

However, as the resulting file contains all the ASCII strings it also becomes a very powerful password cracking database. This is especially useful for cracking Office passwords and similar. The tools from Elcomsoft (http://www.elcomsoft.com/aopr.html) are superb at this and you can load your own dictionaries such as the one you have made with Strings.

Copyright 2021 – CSITech Ltd

RAM Carving

- Output can be MASSIVE
- Attempt to open in NP++ will result in crash
  - Open with LTFViewr – in General tools on USB key
- Split into smaller files
  - linux/mac use `split`
  - Windows use a tool like HJSplit (On USB)
- OR Reduce number of results
  - Add `-n <Length>`
  - Example: `strings64 -n 5 c:\Win10.img`
    - Only finds strings of at least 5 characters long
    - 3 is the default length

Strings is a create command, originally from Unix. It extracts all the 'human-readable' ASCII text from any file even a binary file such as a RAM Dump. So, if you run Strings against a RAM dump you will extract all the ASCII text which can be useful in its own right to search through. If you are looking for a Skype name, an email address or a unique word this can be helpful.

However, as the resulting file contains all the ASCII strings it also becomes a very powerful password cracking database. This is especially useful for cracking Office passwords and similar. The tools from Elcomsoft (http://www.elcomsoft.com/aopr.html) are superb at this and you can load your own dictionaries such as the one you have made with Strings.

Copyright 2021 – CSITech Ltd

RAM Carving

- Reducing the number of results
  - Use findstr (or multiple) parameter to the command to filter out the output file
- Filter out all string that contain http

```
strings64 c:\Win10.img | findstr /i http > 
c:\temp\http_lines.txt
```

- Find some files with locations

```
strings64 c:\Win10.img | findstr /i file:// > 
c:\temp\files.txt
```

- Find google search quarries

```
strings64 c:\Win10.img | findstr /i google.com/search?q= > 
c:\temp\google_searches.txt
```


The /i ignores the case of the characters when searching for the string.

See https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/findstr for more info

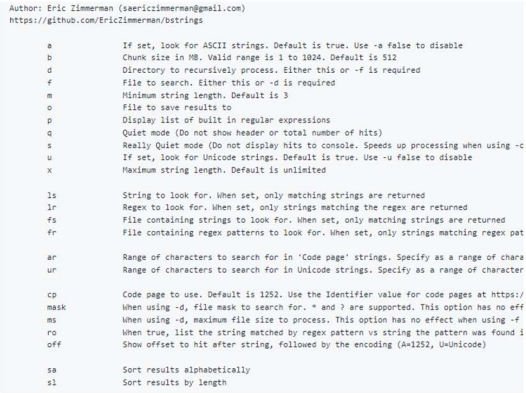
In linux, grep will be even more powerful

To search in existing file do: findstr /i <search term> <input file> > <output file>

Copyright 2021 – CSITech Ltd

RAM Carving

- Similar to strings – bstrings
- Made by Eric Zimmerman - https://ericzimmerman.github.io/
- As he calls it - A better strings utility!
- Features:
  - Pretty fast
  - Unique strings – smaller output
  - Built in regex searches
  - A lot of options – flexibility
  - Sorting
  - Works on live systems
  - Works on folders
  - Requires .NET 6+



```
Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/bstrings

        a               If set, look for ASCII strings. Default is true. Use -a false to disable
        b               Chunk size in MB. Valid range is 1 to 1024. Default is 512
        d               Directory to recursively process. Either this or -f is required
        f               File to search. Either this or -d is required
        m               Minimum string length. Default is 3
        o               File to save results to
        p               Display list of built in regular expressions
        q               Quiet mode (Do not show header or total number of hits)
        s               Really Quiet mode (Do not display hits to console. Speeds up processing when using -o
        u               If set, look for Unicode strings. Default is true. Use -u false to disable
        x               Maximum string length. Default is unlimited

        ls              String to look for. When set, only matching strings are returned
        lr              Regex to look for. When set, only strings matching the regex are returned
        fs              File containing strings to look for. When set, only matching strings are returned
        fr              File containing regex patterns to look for. When set, only strings matching regex pat

        ar              Range of characters to search for in 'Code page' strings. Specify as a range of chara
        ur              Range of characters to search for in Unicode strings. Specify as a range of character

        cp              Code page to use. Default is 1252. Use the Identifier value for code pages at https:/
        mask            When using -d, file mask to search for. * and ? are supported. This option has no eff
        ms              When using -d, maximum file size to process. This option has no effect when using -f
        ro              When true, list the string matched by regex pattern vs string the pattern was found i
        off             Show offset to hit after string, followed by the encoding (A=1252, U=Unicode)

        sa              Sort results alphabetically
        sl              Sort results by length
```


By testing with regular stings64, it comes to about 30s

RAM Carving

- In your "\General tools\Strings" folder
- Install the .NET 6 and unzip to TEMP folder
- Usage:
  - Bstrings.exe <switches>
    - -f Input file
    - -d Input directory
    - -o Output file
    - -p Display list of built in regular expressions
    - -s Really Quiet mode (Do not display hits to console. Speeds up processing when using -o)
    - -x Maximum string length, default is unlimited
    - -ls String to look for. When set, only matching strings are returned
    - -lr Regex to look for. When set, only strings matching the regex are returned
    - -fs File containing strings to look for. When set, only matching strings are returned
    - -fr File containing regex patterns to look for
    - -sa Sort results alphabetically
    - -sl Sort results by length

RAM Carving

- Usage examples:
- `bstrings.exe -f "C:\Temp\xpsp2.img" --ls URL`
- `bstrings.exe -f "C:\Temp\aBigFile.bin" --fs c:\temp\searchStrings.txt --fr c:\temp\searchRegex.txt -s`
- `bstrings.exe -d "C:\Temp" --mask "*.dll"`
- `bstrings.exe -d "C:\Temp" --ls test`
- `bstrings.exe -f "C:\Temp\someOtherFile.txt" --lr cc --sa -m 15 -x 22`

RAM Carving

- Built in regex
  - Bitlocker key
  - Crypto wallets
  - IP add
  - UNC
  - URL
  - …

```
C:\TEMP>bstrings.exe -p
Name            Description
aeon            Finds Aeon wallet addresses
b64             Finds valid formatted base 64 strings
bitcoin         Finds BitCoin wallet addresses
bitlocker       Finds Bitlocker recovery keys
bytecoin        Finds ByteCoin wallet addresses
cc              Finds credit card numbers
dashcoin        Finds DashCoin wallet addresses (D*)
dashcoin2       Finds DashCoin wallet addresses (7|X)*
email           Finds embedded email addresses
fantomcoin      Finds Fantomcoin wallet addresses
guid            Finds GUIDs
ipv4            Finds IP version 4 addresses
ipv6            Finds IP version 6 addresses
mac             Finds MAC addresses
monero          Finds Monero wallet addresses
reg_path        Finds paths related to Registry hives
sid             Finds Microsoft Security Identifiers (SID)
ssn             Finds US Social Security Numbers
sumokoin        Finds SumoKoin wallet addresses
unc             Finds UNC paths
url3986         Finds URLs according to RFC 3986
urlUser         Finds usernames in URLs
usPhone         Finds US phone numbers
var_set         Finds environment variables being set (OS=Windows_NT)
win_path        Finds Windows style paths (C:\folder1\folder2\file.txt)
xml             Finds XML/HTML tags
zip             Finds zip codes

To use a built in pattern, supply the Name to the --lr switch
```

Bulk Extractor

- Superb tool from Simson Garfinkle
  - Associate Professor at the Naval Postgraduate School in Monterey, California
  - Involved with Sleuth kit

AUTOPSY
DIGITAL FORENSICS

Copyright 2021 – CSITech Ltd

Bulk Extractor

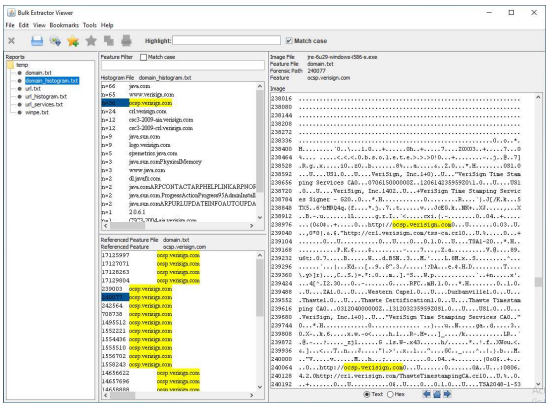
- Idea based on file carving
- To carve data from a disk, image or other device based on rules
- Can be used for fast analysis of any disk or image
- Supports multi-core machines
- Happens to work well on a RAM dump
- Based on Java
- New 2.0 version under development
  - Mac and Linux only
  - C++ based

The latest versions can be downloaded here - https://github.com/simsong/bulk_extractor/wiki/Downloads

Copyright 2021 – CSITech Ltd

Bulk Extractor

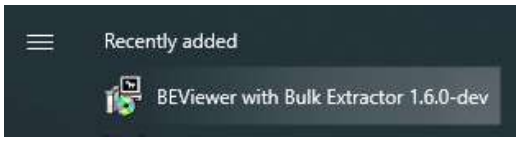
- Install ‘bulk_extractor-1.6.0-dev-windowsinstaller’ from your key
  - General tools\Bulkextractor
- Its very fast!
- Run against Win10.img
- Requires Java

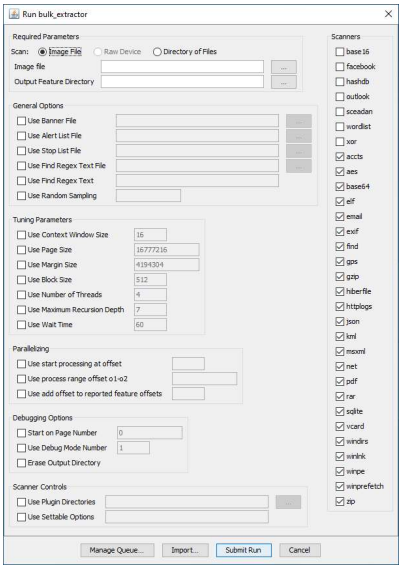


Copyright 2021 – CSITech Ltd

Bulk Extractor

- Simply open Bulk extractor
  - Select Tools – Run Bulk Extractor
  - Point to image file
  - Define output folder
  - Press “Submit Run”





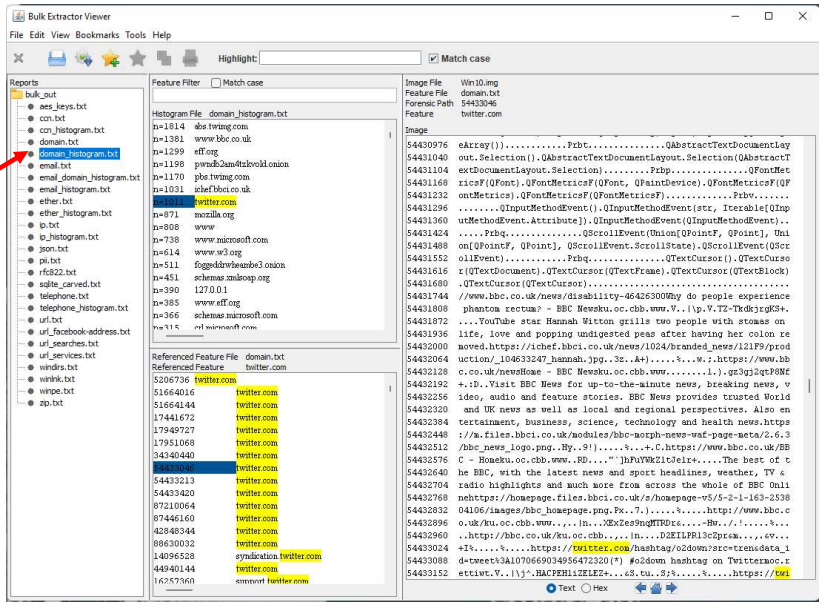
There are a lot of different options but I recommend stick with the defaults until you get time to study the manual.

Remember to define an output folder.

Copyright 2021 – CSITech Ltd

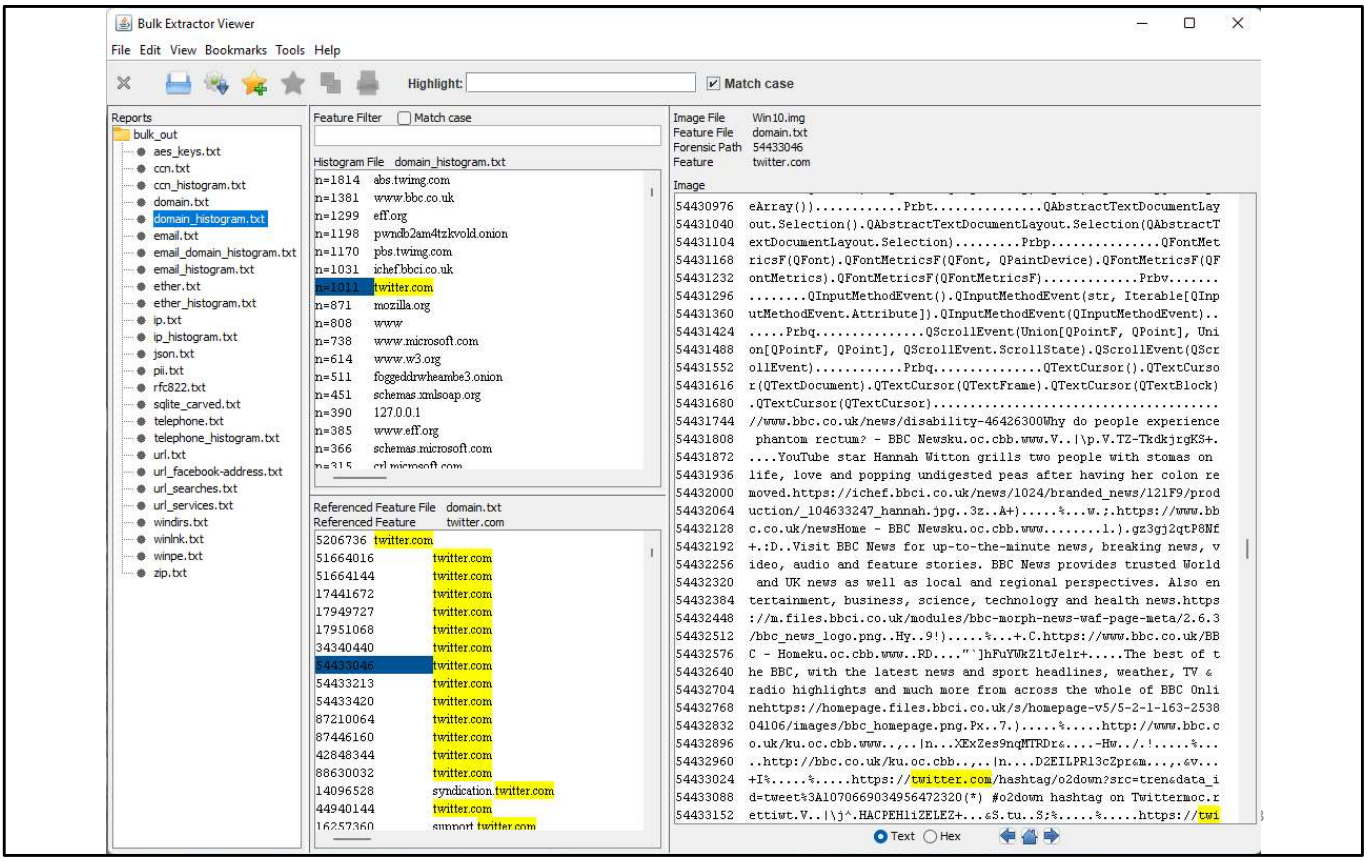
Bulk Extractor

- Wait for a while
- Open the resulting xml file



You can select the category of what you are looking for on the left. The results will appear in the middle pane. Clicking a result will open the entry on the right with the context.

Copyright 2021 – CSITech Ltd

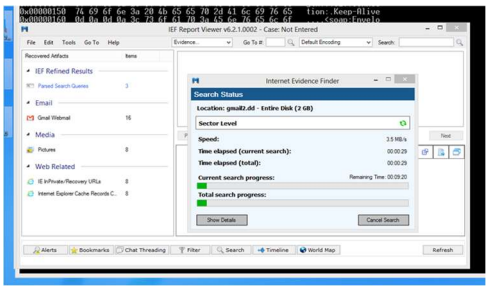


https://github.com/simsong/bulk_extractor – GitHub page
https://github.com/cc-code-public/bulk_extractor_plugins - Digital currency plugins for bulk_extractor version 1.6.0

Copyright 2021 – CSITech Ltd

Carving with Internet Evidence Finder

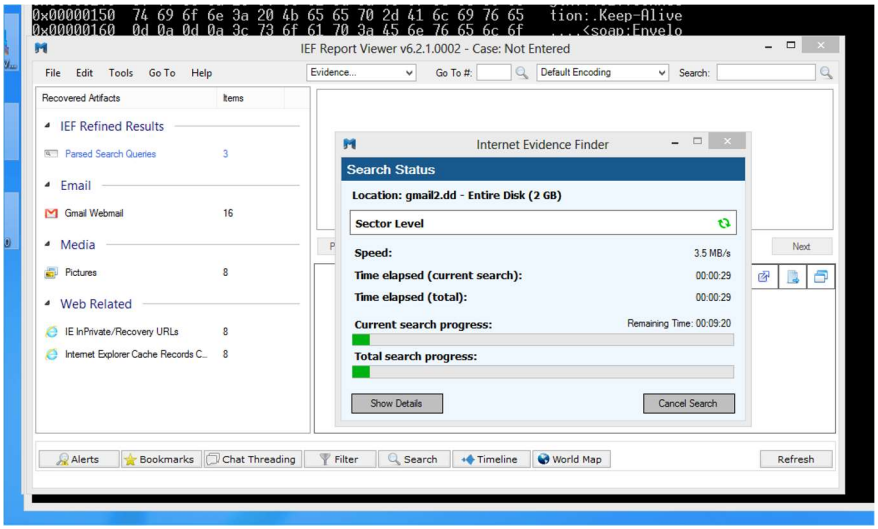
- Internet Evidence Finder been around for a while
- Company now – Magnet Forensics
- Best new forensics tool in years
- Will carve from ‘anything’ including RAM dumps



In my opinion Internet Evidence Finder (IEF) is the best new forensic tool we have had in years. It has so many carvers it is hard to keep up. Magnet Forensics are also a great company to communicate with. We have a copy on your key for you.

Copyright 2021 – CSITech Ltd

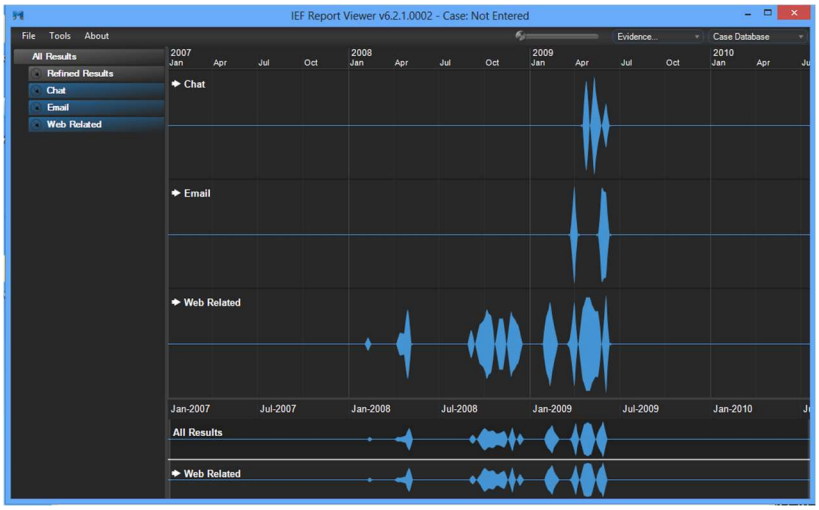
Carving with Internet Evidence Finder



Copyright 2021 – CSITech Ltd

Carving with Internet Evidence Finder

- Also a superb TimeLine viewer



The timeline viewers provides a quick visual reference as to when artifacts were created helping you see when the computer was in use and when used heavily.

Copyright 2021 – CSITech Ltd

Volatility

It’s Awesome!

Copyright 2021 – CSITech Ltd

Volatility

- Framework to allow investigation and carving of RAM elements
- First released in 2007 at Black Hat DC

- Maintained by Volatility Foundation (http://www.volatilityfoundation.org)
- **Download very latest from** https://github.com/volatilityfoundation

Volatility is a community sourced project that has taken on a life of its own in recent years. The primary contributors are Michael Hale Ligh, Andrew Case, Jamie Levy and Aaron Walters as well as a number of Alpha and Beta testers.

The tool is based on the Python programming language and hence will work on any operating system with the Python framework available, including Windows, macOS, Linux and others. Although some have started work on GUI's, they are all pretty rubbish and it remains a command line tool.

Copyright 2021 – CSITech Ltd

Volatility in Use

- Volatility 2.6.1 is the latest stable release version
  - Full support of 64bit Windows OS
  - Good Linux support
  - Good macOS support
- Latest beta version on github
  - https://github.com/volatilityfoundation/volatility

We will be using 2.6 in this class

Copyright 2021 – CSITech Ltd

Volatility in Use

- Volatility 3.0 public beta
  - Released on OSDFcon in October 2019
  - Complete rewrite of the framework
  - Based on Python 3
- For users, the highlights of the new features include:
  - Major performance boosts – its really FAST!
  - The removal of reliance on *--profile* in order for the framework to determine which symbol table (profile) is needed to match the operating system version in the memory sample
  - Proper evaluation of 32bit code on 64bit systems, such as Window's wow64
  - Automatic evaluation of in-memory code to avoid as much manual reverse engineering on part of the analyst as possible

For developers:
Much simpler integration into user 3rd-party interfaces and libraries
Extensive API documentation
The ability for plugins to directly call other plugins
Plugin versioning
Direct integration of custom symbol tables and data structures

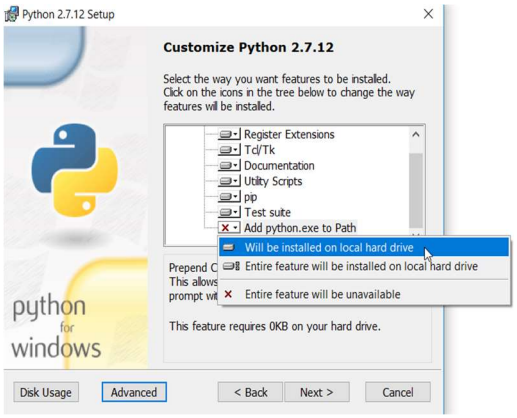
Volatility in Use

- Volatility 3.0
- For developers:
  - Much simpler integration into user 3rd-party interfaces and libraries
  - Extensive API documentation
  - The ability for plugins to directly call other plugins
  - Plugin versioning
  - Direct integration of custom symbol tables and data structures
- Roadmap
  - V2 – will be updated till August 2021
  - V3 - first full release in August 2020 - Have complete feature parity (plugins, address spaces, etc.) with Volatility 2 as well as a number of completely new analysis features.

For developers:
Much simpler integration into user 3rd-party interfaces and libraries
Extensive API documentation
The ability for plugins to directly call other plugins
Plugin versioning
Direct integration of custom symbol tables and data structures

Volatility in Use

- Install python-2.7.15.msi (**32bit**) from Volatility setup\Installers
- **IMPORTANT** – Add python 2.7 to Path!!!
- After install, **restart** the system



The following pages walk you through the installation of Volatility 2.6
Make sure you are using the 32bit version!

Copyright 2021 – CSITech Ltd

Volatility in Use

- From Installers install:
  - pycrypto-2.6.1.win32-py2.7
  - dpkt-1.7.win32
  - distorm3-3.3.1.win32
  - PIL-1.1.7.win32-py2.7
- Copy volatility-master folder from Volatility Setup\Installers to Desktop

Copyright 2021 – CSITech Ltd

Volatility - info

- This is a key command!
- Tells us:
  - What commands are available to us
  - What Profiles are available to us
  - Open CMD in volatility-master folder – click on the address bar and type CMD

```
vol.py --info
```


This is a great command and you will use it often. It lists all the profiles and all the commands that are available to you with descriptions. Give it a go.
You can run everything in PowerShell as well but typography will be a bit different

Copyright 2021 – CSITech Ltd

Volatility in Use

- PROFILES

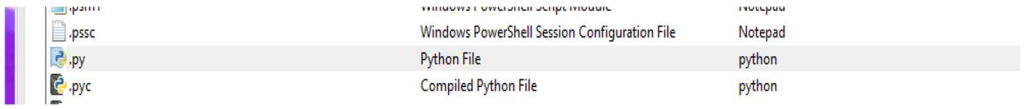
  VistaSP0x86 -- A Profile for Windows Vista SP0 x86
  VistaSP1x86 -- A Profile for Windows Vista SP1 x86
  VistaSP2x86 -- A Profile for Windows Vista SP2 x86
  Win2K8SP1x86 -- A Profile for Windows 2008 SP1 x86
  Win2K8SP2x86 -- A Profile for Windows 2008 SP2 x86
  Win7SP0x86 -- A Profile for Windows 7 SP0 x86
  WinXPSP2x86 -- A Profile for Windows XP SP2
  WinXPSP3x86 -- A Profile for windows XP SP3
  … Etc etc

Windows build versions: https://en.wikipedia.org/wiki/Windows_10_version_history or
https://www.gaijin.at/en/infos/windows-version-numbers
https://www.lifewire.com/windows-version-numbers-2625171

Copyright 2021 – CSITech Ltd

Volatility - info

- If you get a big list of data in your command shell – 👍
- If Notepad opens full of code – 👎
- Control Panel –Programs – Default Programs
- Find and change .py and .pyc to Python



This is a great command and you will use it often. It lists all the profiles and all the commands that are available to you with descriptions. Give it a go.

Copyright 2021 – CSITech Ltd

Volatility in Use

- Command line:
  - **Always from within volatility-master folder**

    vol.py <cmd> -f <path2image>

  - Example:

```
vol.py pslist -f c:\myimage.dd
```

- To view help:

```
vol.py --help
```


You ALWAYS invoke Python at the start of the command
You ALWAYS have –f and point to a RAM Dump
You ALWAYS have a command

Copyright 2021 – CSITech Ltd

Volatility in Use

- Command line:
  - To view help on a specific plugin:
  - vol.py <cmd> --help
  - `vol.py pslist --help`
  - Very useful for playing with output types and trouble shooting errors

```
C:\Users\4n6\Desktop\volatility-master>vol.py pslist --help
Volatility Foundation Volatility Framework 2.6.1
Usage: Volatility - A memory forensics analysis platform.

Options:
  -h, --help            list all available options and their default values.
                        Default values may be set in the configuration file
                        (/etc/volatilityrc)
  --conf-file=.volatilityrc
                        User based configuration file
  -d, --debug           Debug volatility
  --plugins=PLUGINS     Additional plugin directories to use (semi-colon
                        separated)
  --info                Print information about all registered objects
  --cache-directory=C:\Users\4n6/.cache\volatility
                        Directory where cache files are stored
  --cache               Use caching
  --tz=TZ               Sets the (Olson) timezone for displaying timestamps
                        using pytz (if installed) or tzset
  -f FILENAME, --filename=FILENAME
                        Filename to use when opening an image
  --profile=WinXPSP2x86
                        Name of the profile to load (use --info to see a list
                        of supported profiles)
  -l LOCATION, --location=LOCATION
                        A URN location from which to load an address space
  -w, --write           Enable write support
  --dtb=DTB             DTB Address
  --shift=SHIFT         Mac KASLR shift address
  --output=text         Output in this format (support is module specific, se
                        the Module Output Options below)
  --output-file=OUTPUT_FILE
                        Write output in this file
  -v, --verbose         Verbose information
  --physical_shift=PHYSICAL_SHIFT
                        Linux kernel physical shift address
  --virtual_shift=VIRTUAL_SHIFT
                        Linux kernel virtual shift address
  -g KDBG, --kdbg=KDBG  Specify a KDBG virtual address (Note: for 64-bit
                        Windows 8 and above this is the address of
                        KdCopyDataBlock)
  --force               Force utilization of suspect profile
  --cookie=COOKIE       Specify the address of nt!ObHeaderCookie (valid for
```


These commands provide limited help but remind you what switches (-f, -D) etc are support by each command

Copyright 2021 – CSITech Ltd

Volatility in Use

- Outputing the results to a text file using '>' or '>>' etc such as:
  `vol.py -f myram.dd pslist` **>> pslist.txt**

- 2.4+ enables data to be exported in multiple formats
- Check 'help' for what is supported
  - json, html, greptext,text,xlsx

- --output=?
- --output-file=<filename>

Some commands allow the exporting of results to other file types such as .csv, .dot files and so on. We will cover that later in the class. In the meantime use > or >> to output to a text file. Some commands are VERY verbose so it can be vital to output to a text file unless you can read the 'Matrix'!

Copyright 2021 – CSITech Ltd

Volatility in Use

- ..you have to also **add the Profile** of the RAM dump
  - XP Service Pack 1 and 2 do NOT require a Profile

  vol.py -f <path2image> **--profile=<PROFILE>** <cmd>

- Eg:

```
vol.py -f c:\myimage.dd --profile=Win7SP1x64 pslist
```


You need to tell Volatility which operating system the RAM dump is from. In a couple of pages we will see how to ascertain that. The exception is if the RAM dump is from Windows XP 32 bit. In that case the default setting will work and no Profile needs to be declared.

Copyright 2021 – CSITech Ltd

Volatility in Use

- **ALL commands we do next 4 days MUST have these 4 parts and be executed from *volatility-master* folder!!!!!!!**

1. vol.py
2. -f <path2image>
3. --profile=<PROFILE>
4. <command>

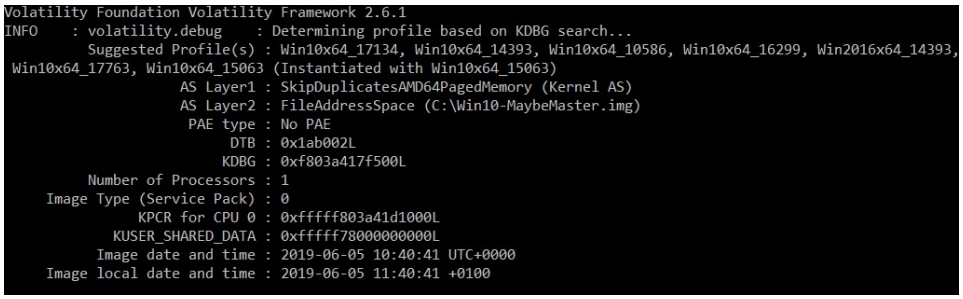
STOP AND READ THIS

Copyright 2021 – CSITech Ltd

Volatility - imageinfo

- Where do we always start?
- Imageinfo gives us data about the RAM image itself
  - Copy Win10.img to the root of c:\

```
vol.py imageinfo -f c:\Win10.img
```



Imageinfo attempts to ascertain which profile we should use. It may come back with several options.

Copyright 2021 – CSITech Ltd

Volatility - imageinfo

- Image Type
  - 2 methods to enable Windows to address RAM over 4 Gig
  - PAE - Physical Address Extension
  - PSE - Page Size Extension

```
Volatility Foundation Volatility Framework 2.6.1
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : Win10x64_17134, Win10x64_14393, Win10x64_10586, Win10x64_16299, Win2016x64_14393,
 Win10x64_17763, Win10x64_15063 (Instantiated with Win10x64_15063)
                     AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
                     AS Layer2 : FileAddressSpace (C:\Win10-MaybeMaster.img)
                      PAE type : No PAE
                           DTB : 0x1ab002L
                          KDBG : 0xf803a417f500L
          Number of Processors : 1
     Image Type (Service Pack) : 0
                KPCR for CPU 0 : 0xfffff803a41d1000L
             KUSER_SHARED_DATA : 0xfffff78000000000L
           Image date and time : 2019-06-05 10:40:41 UTC+0000
     Image local date and time : 2019-06-05 11:40:41 +0100
```

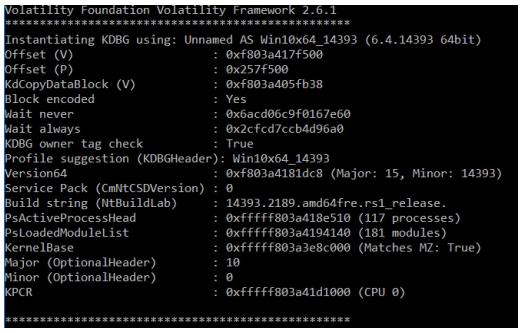

Before Windows XP SP3 the computer could no address more than 4 GIG RAM. In this case the output would say NOPAE, no physical address extension. Almost all modern Windows systems will report PAE, meaning they can address more than 4 GIG.

Copyright 2021 – CSITech Ltd

Volatility - kdbgscan

- Many suggested profiles.. Which one?
  - Trial and error.. or
  - KDBGScan

  - `vol.py kdbgscan -f c:\Win10.img`

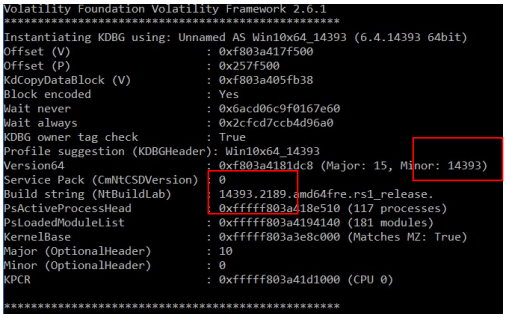


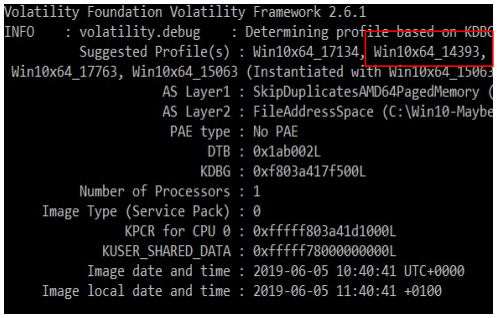
The KDBG is a structure maintained by the Windows kernel for debugging purposes. It contains a list of the running processes and loaded kernel modules. It also contains some version information that allows you to determine if a memory dump came from a Windows XP system versus Windows 7, what Service Pack was installed, and the memory model (32-bit vs 64-bit).

Copyright 2021 – CSITech Ltd

Volatility – imageinfo/kdbgscan

- Compare the results from kdbgscan and imageinfo
  - Imageinfo suggests many profiles
  - Kdbgscan suggests a minor build of 14393
- Profile for Win10.img = Win10x64_14393





The KDBG is a structure maintained by the Windows kernel for debugging purposes. It contains a list of the running processes and loaded kernel modules. It also contains some version information that allows you to determine if a memory dump came from a Windows XP system versus Windows 7, what Service Pack was installed, and the memory model (32-bit vs 64-bit).

Windows build versions: https://en.wikipedia.org/wiki/Windows_10_version_history or https://www.gaijin.at/en/infos/windows-version-numbers

Copyright 2021 – CSITech Ltd

Volatility – imageinfo/kdbgscan

- Briefly….
- Kernel Processor Control Region (KPCR) is a data structure that contains list of KDBG structures
- Kernel Debugger Block (KDBG) is a Windows debugging module
  - Contains loaded kernel modules, running processes → OS detection

Volatility - hibinfo

- Hiberfil is created when a user puts their system into Sleep/Hibernate mode
  - **Only contains Physical Memory that is in use**
  - Can enable critical evidence to be discovered even where a machine has been ‘tidied’
  - Encryption Master Keys!!

There may be times when it is impossible to get a RAM dump. However, when the disks are analysed there may be a hiberfil.sys file in the root of the C drive. This can be converted to a RAW RAM dump for analysis.

Copyright 2021 – CSITech Ltd

Volatility - hibinfo

- Convert a hiberfil.sys file into a RAW dump
  - Works only for supported profiles in Volatility

```
vol.py imagecopy -f <filename> -O <newfile>
```

- Use Hibr2Bin.exe from Matthieu Suiche
  - In General tools\Dumpit folder

Copyright 2021 – CSITech Ltd

Volatility - Where to start

- We want to know what process and applications were running
- Isolate the DLL's being used by a process
- What processes are talking to the outside world
- Recreate the exe that started a process - Virus check the exe's
- Extract the memory space of a process and analyse
- Extract open files

Copyright 2021 – CSITech Ltd

Volatility - pslist

- Very similar to the pslist from Sysinternals/Microsoft
- pslist will locate all of the running processes and their:
  - PID and PPID
  - User session number
  - Start/exit date and time

As we learned earlier programs run as processes and are specific to the logged in user. Services are used by any logged in user. The Pslist will enumerate all the running processes, we learn how this works and its limitations later.

Copyright 2021 – CSITech Ltd

Volatility - pslist

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0xffffa28760a61440	System	4	0	132	0	------	0	2019-06-05 10:21:38 UTC+0000	
0xffffa2876233c040	smss.exe	512	4	4	0	------	0	2019-06-05 10:21:38 UTC+0000	
0xffffa287627d3080	csrss.exe	612	604	11	0	0	0	2019-06-05 10:21:44 UTC+0000	
0xffffa2876292c400	smss.exe	668	512	0	--------	1	0	2019-06-05 10:21:45 UTC+0000	
0xffffa2876298e080	csrss.exe	676	668	11	0	1	0	2019-06-05 10:21:45 UTC+0000	
0xffffa287629a0080	wininit.exe	692	604	4	0	0	0	2019-06-05 10:21:45 UTC+0000	
0xffffa28762d41080	services.exe	756	692	23	0	0	0	2019-06-05 10:21:46 UTC+0000	
0xffffa28762d40080	lsass.exe	764	692	10	0	0	0	2019-06-05 10:21:46 UTC+0000	
0xffffa28762d3c080	winlogon.exe	776	668	5	0	1	0	2019-06-05 10:21:46 UTC+0000	
0xffffa28762d2a800	svchost.exe	864	756	45	0	0	0	2019-06-05 10:21:47 UTC+0000	
0xffffa28762d47800	svchost.exe	904	756	19	0	0	0	2019-06-05 10:21:47 UTC+0000	
0xffffa28762e7c800	dwm.exe	1008	776	11	0	1	0	2019-06-05 10:21:47 UTC+0000	
0xffffa28762e52800	svchost.exe	628	756	94	0	0	0	2019-06-05 10:21:48 UTC+0000	
0xffffa28762609800	svchost.exe	840	756	21	0	0	0	2019-06-05 10:21:48 UTC+0000	
0xffffa2876264e700	svchost.exe	1068	756	19	0	0	0	2019-06-05 10:21:49 UTC+0000	
0xffffa28762618800	svchost.exe	1076	756	34	0	0	0	2019-06-05 10:21:49 UTC+0000	
0xffffa28762663800	svchost.exe	1084	756	29	0	0	0	2019-06-05 10:21:49 UTC+0000	
0xffffa287629c7080	vmacthlp.exe	1256	756	2	0	0	0	2019-06-05 10:21:51 UTC+0000	
0xffffa28760acd800	svchost.exe	1384	756	23	0	0	0	2019-06-05 10:21:53 UTC+0000	
0xffffa28762c78800	svchost.exe	1428	756	9	0	0	0	2019-06-05 10:21:53 UTC+0000	
0xffffa28762ca5080	svchost.exe	1488	756	10	0	0	0	2019-06-05 10:21:53 UTC+0000	
0xffffa28762ccf800	spoolsv.exe	1616	756	15	0	0	0	2019-06-05 10:21:53 UTC+0000	
0xffffa28762a65080	svchost.exe	1812	756	13	0	0	0	2019-06-05 10:21:54 UTC+0000	
0xffffa28762ab6800	svchost.exe	1904	756	5	0	0	0	2019-06-05 10:21:54 UTC+0000	
0xffffa28760aed800	svchost.exe	1960	756	12	0	0	0	2019-06-05 10:21:55 UTC+0000	
0xffffa28762b19800	vmtoolsd.exe	1968	756	9	0	0	0	2019-06-05 10:21:55 UTC+0000	
0xffffa28762b11800	VGAuthService.	1976	756	3	0	0	0	2019-06-05 10:21:55 UTC+0000	
0xffffa28762b795c0	MsMpEng.exe	1044	756	27	0	0	0	2019-06-05 10:21:55 UTC+0000	
0xffffa28762b5b800	MemCompression	1272	4	36	0	------	0	2019-06-05 10:21:55 UTC+0000	
0xffffa28762bdd800	svchost.exe	1440	756	26	0	0	0	2019-06-05 10:21:56 UTC+0000	
0xffffa28763245540	CompatTelRunne	2220	628	3	0	0	0	2019-06-05 10:21:59 UTC+0000	
0xffffa2876324d800	sedlauncher.ex	2236	628	5	0	0	0	2019-06-05 10:21:59 UTC+0000	

The process ID is very important to us if we want to do further investigation of a running program. Volatility requires that we provide the PID, not the process name when doing analysis.

The start date can be very useful when looking for malware or other concerning programs. Many viruses and trojans attempt to hide themselves simply by calling their filename the same as a Windows system process. This includes LSASS and svchost. With 5 or 6 svchost processes running it can be hard to know if one is concerning. However, if a system file started later than the boot time, that would be worth investigation.

We also have Column:
Sess – stands for sessions – usually all will be 0 that's from the system or in WinXP, in newer version of Win system will be 0 and 1st user will be 1, 2nd 2 and onwards
Wow64 – stands for Windows 32-bit on Windows 64-bit – if the process is 64 or 32 bit essentially

Good thing to know:
Taskhost.exe - Win7
Taskhostex.exe - Win8
Taskhostw.exe - Win10

Similar to lsass - vista and up by wininit, before winlogon; cmd.exe and conhost.exe switched position in win10, cmd is parent to conhost

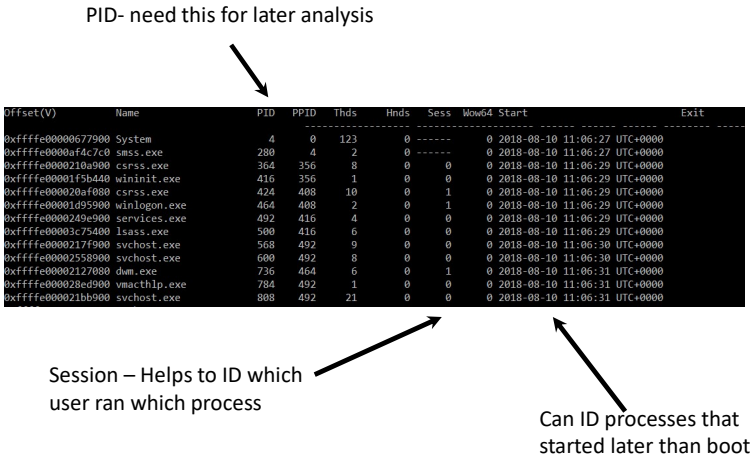
Copyright 2021 – CSITech Ltd

Volatility - pslist

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0xffffe00000677900	System	4	0	123	0	------	0	2018-08-10 11:06:27 UTC+0000	
0xffffe0000af4c7c0	smss.exe	280	4	2	0	------	0	2018-08-10 11:06:27 UTC+0000	
0xffffe0000210a900	csrss.exe	364	356	8	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00001f5b440	wininit.exe	416	356	1	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe000020af080	csrss.exe	424	408	10	0	1	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00001d95900	winlogon.exe	464	408	2	0	1	0	2018-08-10 11:06:29 UTC+0000	
0xffffe0000249e900	services.exe	492	416	4	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00003c75400	lsass.exe	500	416	6	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe0000217f900	svchost.exe	568	492	9	0	0	0	2018-08-10 11:06:30 UTC+0000	
0xffffe00002558900	svchost.exe	600	492	8	0	0	0	2018-08-10 11:06:30 UTC+0000	
0xffffe00002127080	dwm.exe	736	464	6	0	1	0	2018-08-10 11:06:31 UTC+0000	
0xffffe000028ed900	vmacthlp.exe	784	492	1	0	0	0	2018-08-10 11:06:31 UTC+0000	
0xffffe000021bb900	svchost.exe	808	492	21	0	0	0	2018-08-10 11:06:31 UTC+0000	

Copyright 2021 – CSITech Ltd

Volatility - pslist



Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exit
0xffffe00000677900	System	4	0	123	0	------	0	2018-08-10 11:06:27 UTC+0000	
0xffffe0000afdc7c0	smss.exe	280	4	2	0	------	0	2018-08-10 11:06:27 UTC+0000	
0xffffe0000210a900	csrss.exe	364	356	8	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00001f5b440	wininit.exe	416	356	1	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe000020af080	csrss.exe	424	408	10	0	1	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00001d95900	winlogon.exe	464	408	2	0	1	0	2018-08-10 11:06:29 UTC+0000	
0xffffe0000249e900	services.exe	492	416	4	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe00003c75400	lsass.exe	500	416	6	0	0	0	2018-08-10 11:06:29 UTC+0000	
0xffffe0000217f900	svchost.exe	568	492	9	0	0	0	2018-08-10 11:06:30 UTC+0000	
0xffffe00002558900	svchost.exe	600	492	8	0	0	0	2018-08-10 11:06:30 UTC+0000	
0xffffe00002127080	dwm.exe	736	464	6	0	1	0	2018-08-10 11:06:31 UTC+0000	
0xffffe000028ed900	vmacthlp.exe	784	492	1	0	0	0	2018-08-10 11:06:31 UTC+0000	
0xffffe000021bb900	svchost.exe	808	492	21	0	0	0	2018-08-10 11:06:31 UTC+0000	

The process ID is very important to us if we want to do further investigation of a running program. Volatility requires that we provide the PID, not the process name when doing analysis.

The start date can be very useful when looking for malware or other concerning programs. Many virusus and trojans attempt to hide themselves simply by calling their filename the same as a Windows system process. This includes LSASS and svchost. With 5 or 6 svchost processes running it can be hard to know if one is concerning. However, if a system file started later than the boot time, that would be worth investigation.

Copyright 2021 – CSITech Ltd

Volatility - pslist

- vol.py -f <path2yourRAMdump> pslist

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 pslist >
c:\temp\pslist.txt
```

- When did user 2 log in?
- What is SSScheduler.ex(e)?
- How many processes have session 2 explorer.exe as PPID?
- What is Soffice?

Copyright 2021 – CSITech Ltd

Volatility - pslist

- Research the following PIDS:
  - 1968, 1976, 1256
  - What does this tell you about Win10.img?
- Why are there multiple instances of Firefox?
- What process started tor.exe and why?
- What processes indicate cryptocurrency use? (2)
- What process started electrum?
- Research anything else of interest

Copyright 2021 – CSITech Ltd

Volatility – pslist

- Change your output to an XLSX file
- Remember the --help?
- We need to install additional Python dependency
- Open new CMD in following folder:

  Volatility setup\Python Plugins\openpyxl-2.1.2
- Run the command:

```
python setup.py install
```


Copyright 2021 – CSITech Ltd

Volatility – pslist

- Go to previous window, up arrow to repeat the previous command and change to:

```
vol.py -f c:\Win10.img
--profile=Win10x64_14393 pslist --output=xlsx
--output-file=c:\temp\pslist.xlsx
```


Copyright 2021 – CSITech Ltd

Volatility – pslist

- Or you can use HTML output

```
vol.py pslist -f c:\Win10.img --
profile=Win10x64_14393 --output=html
--output-file=c:\temp\pslist.html
```


Copyright 2021 – CSITech Ltd

Volatility – finding what you want

- Outputs can be HUGE
- Can find strings we are looking for in Windows using findstr

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 pslist
| findstr /i Firefox > c:\temp\out.txt
```


Findstr /? → prints info on findstr command
/I Specifies that the search is not to be case-sensitive.
/X Prints lines that match exactly.
/V Prints only lines that do not contain a match.
/R Uses search strings as regular expressions.

Use spaces to separate multiple search strings

Copyright 2021 – CSITech Ltd

Volatility - pstree

- vol.py pstree -f <path2yourRAMdump>

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 pstree >
c:\temp\pstree.txt
```

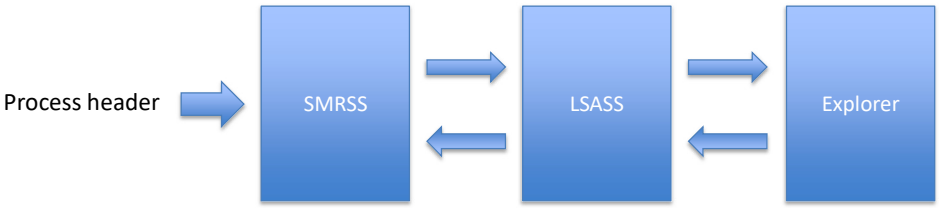
- Same as pslist but in a more friendly mode for reading – better view of what started what…

Name	Pid	PPid	Thds	Hnds	Time
0xffffa28760a61440:System	4	0	132	0	2019-06-05 10:21:38 UTC+0000
. 0xffffa2876233c040:smss.exe	512	4	4	0	2019-06-05 10:21:38 UTC+0000
.. 0xffffa28762365280:smss.exe	4664	512	0	------	2019-06-05 10:27:48 UTC+0000
... 0xffffa28763d4a080:csrss.exe	4164	4664	14	0	2019-06-05 10:27:49 UTC+0000
... 0xffffa287613a7080:winlogon.exe	5164	4664	5	0	2019-06-05 10:27:49 UTC+0000
.... 0xffffa287623aa080:userinit.exe	6180	5164	0	------	2019-06-05 10:27:57 UTC+0000
..... 0xffffa28764020080:explorer.exe	7056	6180	67	0	2019-06-05 10:27:57 UTC+0000
..... 0xffffa28763fdb080:VeraCrypt.exe	5140	7056	11	0	2019-06-05 10:29:06 UTC+0000
..... 0xffffa287648b2080:cmd.exe	9052	7056	1	0	2019-06-05 10:39:14 UTC+0000
..... 0xffffa2876481d080:electrum-3.3.3	6056	9052	17	0	2019-06-05 10:39:59 UTC+0000
..... 0xffffa28761a39800:conhost.exe	9156	9052	5	0	2019-06-05 10:39:17 UTC+0000
..... 0xffffa28761a40080:MSASCuiL.exe	5560	7056	5	0	2019-06-05 10:28:16 UTC+0000
..... 0xffffa28763dae080:SSScheduler.ex	3156	7056	2	0	2019-06-05 10:28:18 UTC+0000
..... 0xffffa28760fe5080:OneDrive.exe	6768	7056	23	0	2019-06-05 10:28:18 UTC+0000
..... 0xffffa287615c8080:notepad++.exe	6320	7056	4	0	2019-06-05 10:29:54 UTC+0000

Copyright 2021 – CSITech Ltd

Volatility – pslist/pstree

- Uses technique called ‘List-walking’
- Literally follows links from one process to the next

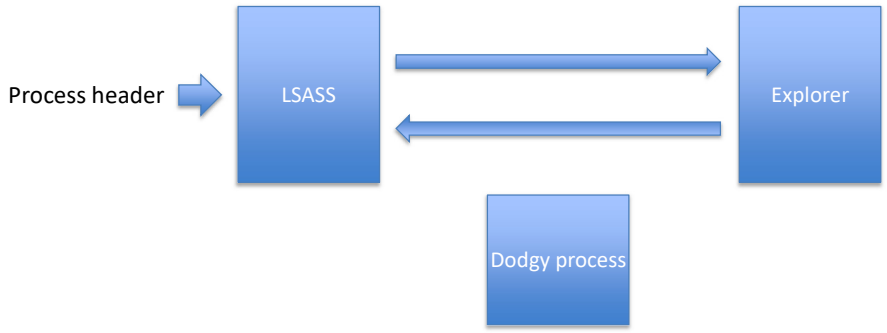


The system the OS uses, for example when you call up the Task Manager in Windows is called List-walking. It looks at the first process, reads the Flink value and jumps to the next process and so on until it hits a processes with no Flink which is the last process. It can do the same in reverse to find a process before it.

Copyright 2021 – CSITech Ltd

Volatility - psscan

- DKOM attack inserts process not referenced in linked process headers

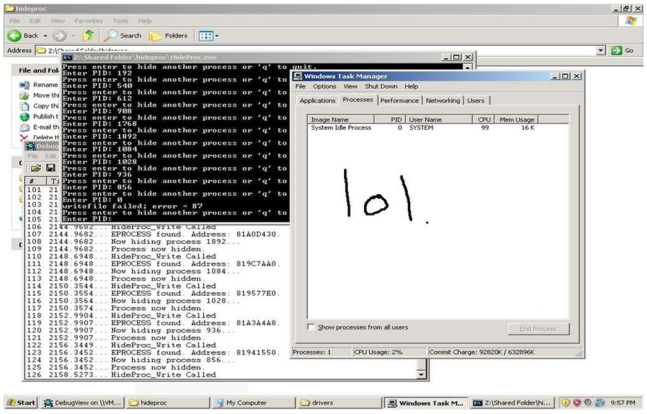


In the DKOM attack the Flink and Blink values are removed so the list-walking skips the running process leaving it invisible to the OS.

Copyright 2021 – CSITech Ltd

Volatility - psscan

- This is very effective at finding hidden processes using the DKOM (Direct Kernel Object Manipulation) method
  - http://www.rohitab.com/discuss/index.php?showtopic=23880



Psscan is a similar command to pslist except it searches for process headers, rather than following the process list (which will be discussed later). This is more effective in finding malware which is designed to hide from the OS. This is called a DKOM method.

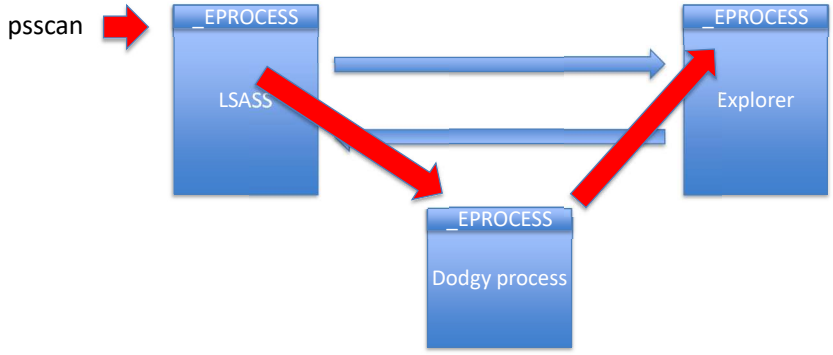
The link will make your anti-virus shout loud. It will 'remove' all the running processes from your Task List but leave the computer running fine – a DKOM attack.

Microsoft PatchGuard is a security feature that was introduced in 2005 in Windows XP. It is only available for 64-bit versions of Microsoft Windows, and its role is to prevent apps from patching the kernel.
PatchGuard didn't kill rootkits on its own, but rootkits did eventually die out, especially after the launch of Windows 10, which featured additional security features, alongside PatchGuard.

Copyright 2021 – CSITech Ltd

Volatility - psscan

- Psscan searches for the _EPROCESS structure header rather than walking the list

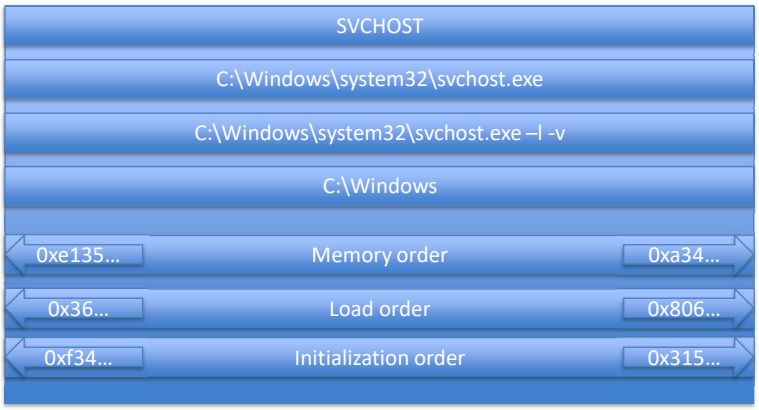


This is where Psscan is a better command to use. Because it scans for _EPROCESS headers it will find unlinked processes.

Copyright 2021 – CSITech Ltd

_EPROCESS Block

- Each process has a _EPROCESS structure or block
  - Contains and points to a number of other related data structures
  - Exist in system address space (systemland, kernel)



172

Very process running has an _EPROCESS structure which can be located and then the process carved out. The _EPROCESS contains:

The process name
The process path on the disk
The command line used with the switches
The install path of the OS
3 address blocks

Copyright 2021 – CSITech Ltd

Volatility - psscan

- Scans rather than lists the processes

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 psscan
```

Offset(P)	Name	PID	PPID	PDB	Time created	Time exited
0x0000850000061440	System	4	0	0x00000000001ab002	2019-06-05 10:21:38 UTC+0000	
0x00008500000cd800	svchost.exe	1384	756	0x000000002f840002	2019-06-05 10:21:53 UTC+0000	
0x00008500000ed800	svchost.exe	1960	756	0x000000002bb04002	2019-06-05 10:21:55 UTC+0000	
0x0000a28760a61440	System	4	0	0x00000000001ab002	2019-06-05 10:21:38 UTC+0000	
0x0000a28760acd800	svchost.exe	1384	756	0x000000002f840002	2019-06-05 10:21:53 UTC+0000	
0x0000a28760aed800	svchost.exe	1960	756	0x000000002bb04002	2019-06-05 10:21:55 UTC+0000	
0x0000a28760dda780	wuauclt.exe	6184	628	0x0000000005903002	2019-06-05 10:29:12 UTC+0000	
0x0000a28760ddf080	bitcoin-qt.exe	7548	7056	0x0000000079480002	2019-06-05 10:33:30 UTC+0000	
0x0000a28760df1080	svchost.exe	3308	756	0x000000004d69d002	2019-06-05 10:23:43 UTC+0000	
0x0000a28760e5d800	svchost.exe	3724	756	0x0000000078300002	2019-06-05 10:23:41 UTC+0000	
0x0000a28760eaf800	backgroundTask	7200	864	0x0000000077600002	2019-06-05 10:36:47 UTC+0000	
0x0000a28760edc080	OneDrive.exe	5056	4084	0x0000000063e28002	2019-06-05 10:24:02 UTC+0000	
0x0000a28760f2e800	taskhostw.exe	3820	628	0x000000004bb07002	2019-06-05 10:23:41 UTC+0000	
0x0000a28760f47800	userinit.exe	4068	776	0x000000004c924002	2019-06-05 10:23:42 UTC+0000	
0x0000a28760f79800	RuntimeBroker.	3972	864	0x000000004c348002	2019-06-05 10:23:42 UTC+0000	
0x0000a28760fb1080	ShellExperienc	6436	864	0x0000000058e00002	2019-06-05 10:27:59 UTC+0000	
0x0000a28760fe0800	explorer.exe	4084	4068	0x000000004caa0002	2019-06-05 10:23:42 UTC+0000	
0x0000a28760fe5080	OneDrive.exe	6768	7056	0x000000001f300002	2019-06-05 10:28:18 UTC+0000	
0x0000a2876102d800	SearchUI.exe	4248	864	0x0000000057272002	2019-06-05 10:23:48 UTC+0000	

Copyright 2021 – CSITech Ltd

Volatility –psscan*

- RAM Images – DKOM.vmem.zip
- Unzip and copy dkom.vmem to root
  - Run pslist and psscan
    - No profile is needed as its an Win XP image

  - `vol.py -f c:\dkom.vmem pslist > c:\temp\DKOM_pslist.txt`
  - `vol.py -f c:\dkom.vmem psscan > c:\temp\DKOM_psscan.txt`

  - Locate the DKOM'd process
  - Try using different outputs to manipulate data
  - Note the more EXIT times compared to pslist!

The DKOM RAM image has a hidden process, try and ascertain which it is and what process was responsible for starting it?

There is a cool NP++ plugin called compare. Go to Plugins – Plugin Admin, search for Compare, select and install. After the restart under the plugins menu you will se a new Compare options that you can use.

Copyright 2021 – CSITech Ltd

Volatility - Sessions

- We have already seen sessions in pslist
- The sessions command will collect all the processes in use by the system and any logged in users

Remembering the need to sometimes attribute activity to a user it can important to identify if there are multiple sessions running. This will extract both local users and those logged in over RDP (Remote Desktop Protocol)

Copyright 2021 – CSITech Ltd

Volatility - Sessions

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 sessions
```

- Session ID's start from '0'
- How many sessions are running?
  - How many processes did user 1 have running?

Copyright 2021 – CSITech Ltd

Volatility - Sessions

```
**************************************************
Session(V): ffffd8803ca40000 ID: 1 Processes: 21
PagedPoolStart: fffff70480000000 PagedPoolEnd fffff7247fffffff
 Process: 668 smss.exe 2019-06-05 10:21:45 UTC+0000
 Process: 676 csrss.exe 2019-06-05 10:21:45 UTC+0000
 Process: 776 winlogon.exe 2019-06-05 10:21:46 UTC+0000
 Process: 1008 dwm.exe 2019-06-05 10:21:47 UTC+0000
 Process: 3724 svchost.exe 2019-06-05 10:23:41 UTC+0000
 Process: 3732 sihost.exe 2019-06-05 10:23:41 UTC+0000
 Process: 3820 taskhostw.exe 2019-06-05 10:23:41 UTC+0000
 Process: 3972 RuntimeBroker. 2019-06-05 10:23:42 UTC+0000
```

```
**************************************************
Session(V): ffffd8804313c000 ID: 2 Processes: 44
PagedPoolStart: fffff70480000000 PagedPoolEnd fffff7247fffffff
 Process: 4664 smss.exe 2019-06-05 10:27:48 UTC+0000
 Process: 4164 csrss.exe 2019-06-05 10:27:49 UTC+0000
 Process: 5164 winlogon.exe 2019-06-05 10:27:49 UTC+0000
 Process: 5604 dwm.exe 2019-06-05 10:27:50 UTC+0000
 Process: 5936 sihost.exe 2019-06-05 10:27:56 UTC+0000
 Process: 6864 svchost.exe 2019-06-05 10:27:56 UTC+0000
 Process: 6776 taskhostw.exe 2019-06-05 10:27:56 UTC+0000
 Process: 6180 userinit.exe 2019-06-05 10:27:57 UTC+0000
```


Copyright 2021 – CSITech Ltd

Volatility - netscan

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 netscan
```

- Similar to running Netstat -na
- ID's processes talking to the network
- Provides the remote IP and port

- Netscan only works on vista and up
  - For xp use connscan and sockscan

These commands look for connection objects and connect them to their process. From this we can identify the IP address or addresses that are being accessed by a program. This could be Skype buddies, web pages in Firefox or malware talking to a command and control box.

Remember that 127.0.0.1 is your local machine and so in many instances is irrelevant.

The Connscan command is only for XP and 2003 systems, anything else needs to be Netscan.

Copyright 2021 – CSITech Ltd

Volatility - netscan

```
Volatility Foundation Volatility Framework 2.6.1
Offset(P)          Proto    Local Address                  Foreign Address      State          Pid      Owner          Created
0xa28760c8f9c0     UDPv4    0.0.0.0:56510                  *:*                                 1440     svchost.exe    2019-06-05 10:22:00 UTC+0000
0xa28760c8f9c0     UDPv6    :::56510                       *:*                                 1440     svchost.exe    2019-06-05 10:22:00 UTC+0000
0xa28760e34250     UDPv4    0.0.0.0:56074                  *:*                                 6056     electrum-3.3.3 2019-06-05 10:40:40 UTC+0000
0xa28760e281f0     TCPv4    192.168.211.131:50211          23.13.217.152:443    CLOSED         7612     firefox.exe    2019-06-05 10:34:09 UTC+0000
0xa28761154b30     TCPv4    127.0.0.1:9150                 0.0.0.0:0            LISTENING      7308     tor.exe        2019-06-05 10:32:13 UTC+0000
0xa28760ef1230     TCPv4    127.0.0.1:50113                127.0.0.1:50114      ESTABLISHED    4396     firefox.exe    2019-06-05 10:31:59 UTC+0000
0xa28760ef53e0     TCPv4    192.168.211.131:50289          104.244.42.65:443    CLOSED         7612     firefox.exe    2019-06-05 10:36:40 UTC+0000
0xa28761154820     TCPv4    192.168.211.131:50169          54.213.41.50:443     CLOSED         7612     firefox.exe    2019-06-05 10:33:48 UTC+0000
0xa287611b1540     TCPv4    127.0.0.1:50137                127.0.0.1:50138      ESTABLISHED    8068     firefox.exe    2019-06-05 10:32:42 UTC+0000
0xa287614b5800     TCPv4    192.168.211.131:50405          216.58.206.141:443   CLOSED         6864     svchost.exe    2019-06-05 10:39:53 UTC+0000
0xa287614f6950     TCPv4    192.168.211.131:50381          143.204.192.51:443   CLOSED         7612     firefox.exe    2019-06-05 10:38:33 UTC+0000
0xa287615708a0     TCPv4    127.0.0.1:50118                127.0.0.1:50117      ESTABLISHED    7308     tor.exe        2019-06-05 10:32:13 UTC+0000
0xa2876203bd30     UDPv4    0.0.0.0:56077                  *:*                                 6056     electrum-3.3.3 2019-06-05 10:40:40 UTC+0000
0xa2876051270      UDPv4    0.0.0.0:5355                   *:*                                 1384     svchost.exe    2019-06-05 10:40:40 UTC+0000
0xa287620a92e0     UDPv4    0.0.0.0:5353                   *:*                                 1384     svchost.exe    2019-06-05 10:40:40 UTC+0000
0xa287620c17c0     UDPv4    0.0.0.0:5353                   *:*                                 1384     svchost.exe    2019-06-05 10:40:40 UTC+0000
0xa287620c17c0     UDPv6    :::5353                        *:*                                 1384     svchost.exe    2019-06-05 10:40:40 UTC+0000
0xa287620fd940     UDPv4    0.0.0.0:56076                  *:*                                 6056     electrum-3.3.3 2019-06-05 10:40:40 UTC+0000
0xa28761a6a1e0     TCPv4    127.0.0.1:50131                127.0.0.1:50132      ESTABLISHED    7772     firefox.exe    2019-06-05 10:32:24 UTC+0000
0xa28761a6c480     TCPv4    127.0.0.1:50422                127.0.0.1:9150       CLOSED         4396     firefox.exe    2019-06-05 10:40:26 UTC+0000
0xa28761ec9d00     TCPv4    127.0.0.1:50165                127.0.0.1:50166      ESTABLISHED    5824     firefox.exe    2019-06-05 10:33:46 UTC+0000
0xa28762020310     TCPv4    127.0.0.1:50164                127.0.0.1:50163      ESTABLISHED    7612     firefox.exe    2019-06-05 10:33:43 UTC+0000
0xa287620ae010     TCPv4    127.0.0.1:9151                 127.0.0.1:50127      ESTABLISHED    7308     tor.exe        2019-06-05 10:32:22 UTC+0000
0xa28762206770     TCPv4    192.168.211.131:50376          212.58.249.199:443   CLOSED         7612     firefox.exe    2019-06-05 10:38:19 UTC+0000
0xa28762210ae0     TCPv4    192.168.211.131:50345          208.100.17.173:443   CLOSED         7612     firefox.exe    2019-06-05 10:37:53 UTC+0000
0xa28762c02a70     UDPv4    192.168.211.131:137            *:*                                 4        System         2019-06-05 10:21:53 UTC+0000
0xa28762c72860     UDPv4    192.168.211.131:138            *:*                                 4        System         2019-06-05 10:21:53 UTC+0000
0xa287629baec0     TCPv4    0.0.0.0:49664                  0.0.0.0:0            LISTENING      692      wininit.exe    2019-06-05 10:21:47 UTC+0000
0xa287629baec0     TCPv6    :::49664                       :::0                 LISTENING      692      wininit.exe    2019-06-05 10:21:47 UTC+0000
0xa28762a21180     TCPv4    0.0.0.0:49667                  0.0.0.0:0            LISTENING      1616     spoolsv.exe    2019-06-05 10:21:54 UTC+0000
0xa28762a21180     TCPv6    :::49667                       :::0                 LISTENING      1616     spoolsv.exe    2019-06-05 10:21:54 UTC+0000
0xa28762a21e40     TCPv4    0.0.0.0:49667                  0.0.0.0:0            LISTENING      1616     spoolsv.exe    2019-06-05 10:21:54 UTC+0000
0xa28762be0010     TCPv4    0.0.0.0:49668                  0.0.0.0:0            LISTENING      764      lsass.exe      2019-06-05 10:21:57 UTC+0000
0xa28762be26b0     TCPv4    0.0.0.0:49668                  0.0.0.0:0            LISTENING      764      lsass.exe      2019-06-05 10:21:57 UTC+0000
0xa28762be26b0     TCPv6    :::49668                       :::0                 LISTENING      764      lsass.exe      2019-06-05 10:21:57 UTC+0000
0xa28762c188a0     TCPv4    0.0.0.0:49665                  0.0.0.0:0            LISTENING      1068     svchost.exe    2019-06-05 10:21:51 UTC+0000
```


It is worth keeping a copy of the pslist or psscan that you run at the start as you regularly get PID's and it provides a quick reference to remind yourself that 3276 is Firefox for example. As in this example you can get false positives which can be ignored.

Copyright 2021 – CSITech Ltd

Volatility - netscan

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 netscan
```

- Output is vast and messy
  - Use findstr to tidy the output
- Try removing the lines with 127.0.0.1
  - `| findstr /v 127.0.0.1`
  - Remove other 'junk' lines by adding another `| findstr /v <junk>` to the end. And repeat

These commands look for connection objects and connect them to their process. From this we can identify the IP address or addresses that are being accessed by a program. This could be Skype buddies, web pages in Firefox or malware talking to a command and control box.

Remember that 127.0.0.1 is your local machine and so in many instances is irrelevant.

The Connscan command is only for XP and 2003 systems, anything else needs to be Netscan.

Copyright 2021 – CSITech Ltd

Volatility - netscan

```
| findstr /i /v 127.0.0.1 | findstr /v *:* | findstr
/v 0.0.0.0:0 | findstr /v :::0
```

- Adding this to the end of your netscan command will leave you with a list of ip addresses
- Or you can filter on a process
  - `| findstr /i <process name>`

These commands look for connection objects and connect them to their process. From this we can identify the IP address or addresses that are being accessed by a program. This could be Skype buddies, web pages in Firefox or malware talking to a command and control box.

Remember that 127.0.0.1 is your local machine and so in many instances is irrelevant.

The Connscan command is only for XP and 2003 systems, anything else needs to be Netscan.

Copyright 2021 – CSITech Ltd

Volatility - netscan

- Use these sites to find information on ip addresses
  - www.viewdns.info
  - https://www.whois.com/whois
  - https://www.infobyip.com/ipbulklookup.php
  - https://mxtoolbox.com/NetworkTools.aspx
  - https://www.ipqualityscore.com/free-ip-lookup-proxy-vpn-test

To answer the second question you will need to head out on the internet and look up the IP address.

My favourite site is viewdns.info

Copyright 2021 – CSITech Ltd

Volatility - netscan

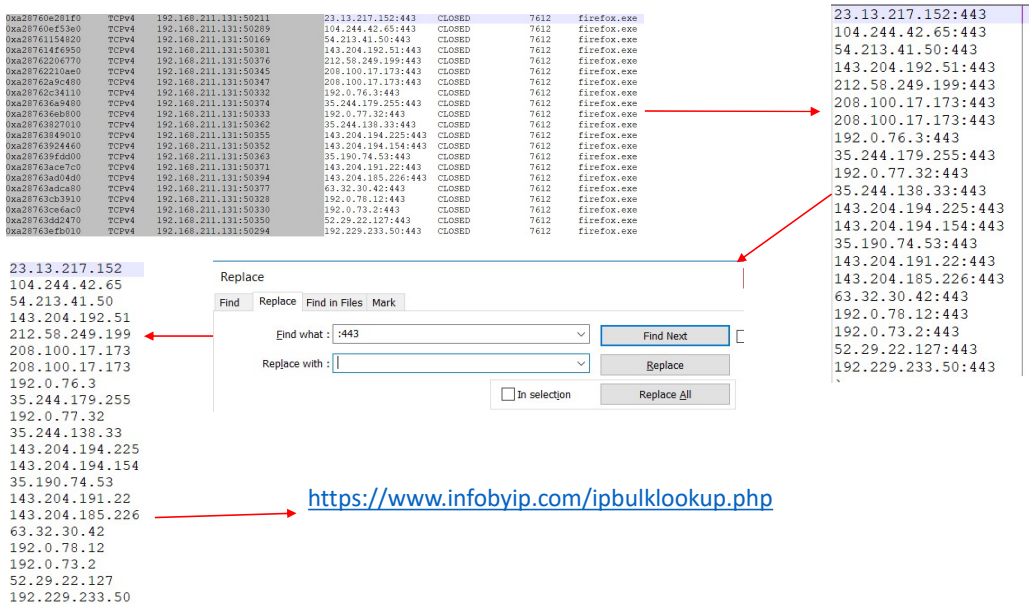
```
vol.py -f c:\Win10.img --profile=Win10x64_14393 netscan
| findstr /i /v 127.0.0.1 | findstr /v *:* | findstr /v
0.0.0.0:0 | findstr /v :::0 | findstr /i Firefox >
c:\temp\firefox_IP.txt
```

- Command is in the General tools - Long commands.txt
- The above command will isolate the list of Firefox IP addresses
- Open in NP++ -> 'Shift + alt' to highlight vertically -> Remove everything before/after external IP address
- Find and replace :443 with nothing, also see the note hint!
- https://www.infobyip.com/ipbulklookup.php

You can use NP++ and use RegEx search \:.* that will select everything after : including it, makes the fixing the output quite easy!

Copyright 2021 – CSITech Ltd

Volatility -netscan



You can use NP++ and use RegEx search \:.* that will select everything after : including it, makes the fixing the output quite easy!

Copyright 2021 – CSITech Ltd

Volatility - netscan

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 netscan
```

- What processes were talking to the internet?
- How many external IP's was Firefox talking to?
- What social network was being accessed by Firefox?
- What news outlet was being browsed?
- What port does bitcoin-qt use?

Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- Possible errors
  - Volatility Command issues
  - No memory file specified
  - No profile specified
  - Output to forbidden things
    - Folder
    - C:\
  - No output specified
  - Wrong running folder
  - …

Usually as in 99.99999% it will be a simple mistype, extra space, missing dash, copy paste error….

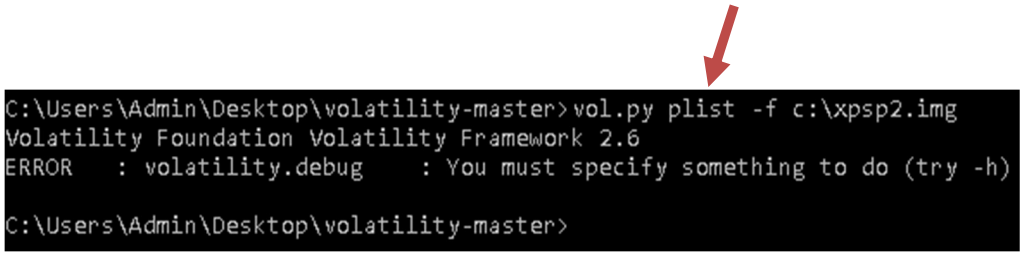
Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- Volatility Command issues

  `You must specify something to do (try -h)`

  - Check your spelling!!!



Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- No memory file specified

  `The requested file doesn't exist`

  - Check filename, path, extension!!

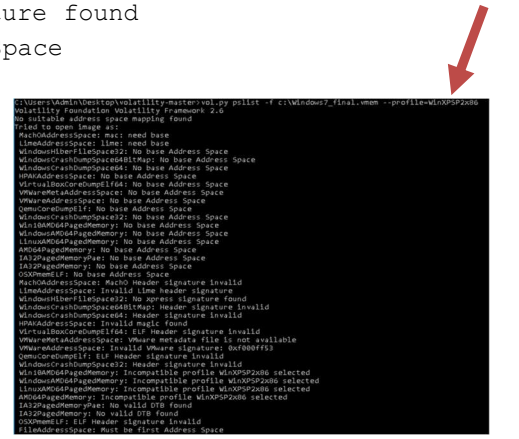
Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- No profile specified

```
No suitable address space mapping found
Win10AMD64PagedMemory: Incompatible profile WinXPSP2x86 selected
WindowsHiberFileSpace32: No base Address Space
WindowsHiberFileSpace32: No xpress signature found
FileAddressSpace: Must be first Address Space
HPAKAddressSpace: Invalid magic found
```

  - Run kdbgscan, check spelling
  - Profile names are case sensitive!



Additional issue with this command is that even simple commands as pslist will run very very very long and then throw and error as in the screenshot. If you notice that something is running longer than usual, do a break (Ctr+C) and check if all is in order with the command and profile.

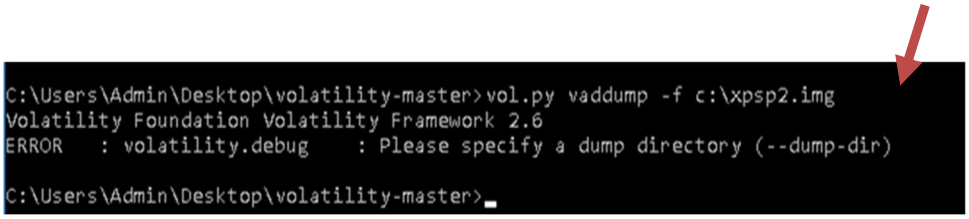
Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- No output specified

  `Please specify a dump directory (--dump-dir).`

  - Check command, run --help, spelling…



Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- Output to forbidden things

  `The system cannot find the path specified.`
  `Access is denied.`
  - Check filename, path, extension, spaces…

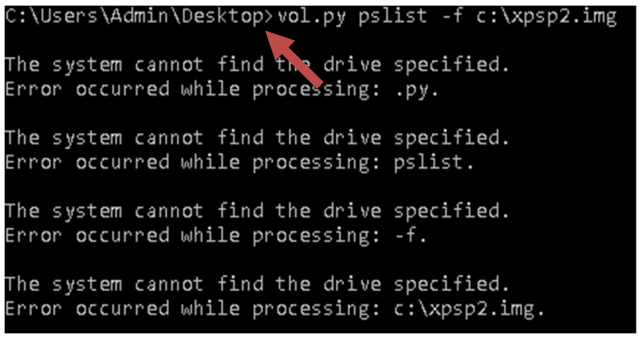
Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- Wrong running folder

  `The system cannot find the drive specified.`
  `Error occurred while processing: .py.`

  - Check path!



Copyright 2021 – CSITech Ltd

Volatility – what can go wrong?

- There are more possible errors but should be self explanatory…
- Eg. non exisiting PID or registry Key

```
C:\Users\Admin\Desktop\volatility-master>vol.py vaddump -f c:\xpsp2.img -p 12412
Volatility Foundation Volatility Framework 2.6
ERROR   : volatility.debug    : Cannot find PID 12412. If its terminated or unlinked, use psscan and then
 supply --offset=OFFSET

C:\Users\Admin\Desktop\volatility-master>
```

```
C:\Users\Admin\Desktop\volatility-master>vol.py printkey -f c:\gmail.dd -K Banana
Volatility Foundation Volatility Framework 2.6
Legend: (S) = Stable   (V) = Volatile

The requested key could not be found in the hive(s) searched

C:\Users\Admin\Desktop\volatility-master>
```


Copyright 2021 – CSITech Ltd

Volatility - dlllist

- Very similar to the listdlls command
- dlllist will provide all the files being accessed by a process
- Using just -f will provide a MASSIVE output
- Add -p <PID> will provide just the process identified in pslist
  - Can select multiple processes with comma eg:
    - -p 3276,800,4

This is a hugely important command when investigating a process. DLL's are dynamic link libraries which are chunks of code often shared with other programs. Most Windows programs use ntdll.dll for example. You can tell a lot about what a program does by looking at the dll's that it calls. We will look more closely at this during the Stuxnet portion of the class.

The output from this command can be huge so it's best to specify the process we want to examine. This is simply done by adding a '-p' and then the process ID into the command line. So it might look like this:

Python vol.py dlllist –f c:\xpsp2.img –p 3276

Copyright 2021 – CSITech Ltd

Volatility - dlllist

```
firefox.exe pid: 3276
Command line : "C:\Program Files\Mozilla Firefox\firefox.exe"
Service Pack 2

Base            Size            Path
0x400000        0x669000        C:\Program Files\Mozilla Firefox\firefox.exe
0x7c900000      0xb0000         UNKNOWN
0x7c800000      0xf4000         C:\WINDOWS\system32\kernel32.dll
0x60070000      0x54000         C:\Program Files\Mozilla Firefox\js3250.dll
0x60130000      0x26000         C:\Program Files\Mozilla Firefox\nspr4.dll
0x77dd0000      0x9b000         C:\WINDOWS\system32\ADVAPI32.dll
0x77e70000      0x91000         C:\WINDOWS\system32\RPCRT4.dll
0x71ad0000      0x9000          C:\WINDOWS\system32\WSOCK32.dll
0x71ab0000      0x17000         C:\WINDOWS\system32\WS2_32.dll
0x77c10000      0x58000         C:\WINDOWS\system32\msvcrt.dll
0x71aa0000      0x8000          C:\WINDOWS\system32\WS2HELP.dll
0x602d0000      0x60000         C:\Program Files\Mozilla Firefox\xpcom.dll
0x60200000      0x7000          C:\Program Files\Mozilla Firefox\plc4.dll
0x60210000      0x6000          C:\Program Files\Mozilla Firefox\plds4.dll
0x7c9c0000      0x814000        C:\WINDOWS\system32\SHELL32.dll
0x77f10000      0x46000         C:\WINDOWS\system32\GDI32.dll
0x77d40000      0x90000         C:\WINDOWS\system32\USER32.dll
0x77f60000      0x76000         C:\WINDOWS\system32\SHLWAPI.dll
0x774e0000      0x13d000        C:\WINDOWS\system32\ole32.dll
0x60230000      0x1a000         C:\Program Files\Mozilla Firefox\smime3.dll
0x60160000      0x55000         C:\Program Files\Mozilla Firefox\nss3.dll
0x60250000      0x5a000         C:\Program Files\Mozilla Firefox\softokn3.dll
0x602b0000      0x1b000         C:\Program Files\Mozilla Firefox\ssl3.dll
0x60330000      0x14000         C:\Program Files\Mozilla Firefox\xpcom_compat.dll
0x763b0000      0x49000         C:\WINDOWS\system32\comdlg32.dll
0x773d0000      0x102000        C:\WINDOWS\WinSxS\X86_Microsoft.Windows.Common-Control
```


The output can give us clues as to the validity of the process or dll's. The executable name and command path is displayed for example. Also the complete paths of the dll's. Should one be referenced to the wrong place on the disk then that would be suspicious. For example if svchost.exe is not running from Windows\system32, that would be bad!

We also see the base value where the process or DLL us loaded into RAM. Microsoft specify exactly where a file should be in RAM for its primary system files. If this is wrong compared to a clean test system then again that would be a sign that something was wrong.

Lastly, the file size in Hex is provided. If this is different to the file size on a clean system, again, that would be a problem. If you do the output in html you will get the size in bytes.

Copyright 2021 – CSITech Ltd

Volatility - dlllist

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 -p <PID> dlllist
```

- Where was Firefox (Pid 4396) started from?
- Compare the file sizes for pid 4396 and 7612
  - Which Firefox.exe is bigger?
    - Utility on your key - "General tools\NumberConvertor.exe"
- What is noticeable about cmd path for 6056?
  - Why might this be the case? (Consider PPID)

Copyright 2021 – CSITech Ltd

Volatility - dlllist

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 -p <PID> dlllist
```

- What are the names of the two logged in users?
  - Search or findstr for \users\

Copyright 2021 – CSITech Ltd

Volatility - handles

- Very useful command
- Shows all the files being used by a process
- Shows files, registry keys, mutexes, named pipes, events, window stations, desktops, threads, and all other types of securable executive objects
- Can focus on specific PID by adding `-p` or `--pid=PID`
- Can also filter by object type using `-t` or `--object-type=OBJECTTYPE`

```
vol.py handles -f c:\Win10.img
--profile=Win10x64_14393 -p 7612 -t file
```


Although the output from the Handles command can be confusing and rather verbose there are very important clues in here. Remeber, you can use –t for filtering on object types! For example it will provide the paths to files that are opened by the program. For a browser that could be caches or history files. This is the first time we will also see full user file paths which can identify the logged in user.

Copyright 2021 – CSITech Ltd

Volatility - handles

- Shows the handles for Firefox 7612 (Non tor)

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 –p 7612 handles
```

- In what folder are the cookies stored?
  - What database format?
- What fonts are associated with this browser?

Copyright 2021 – CSITech Ltd

Volatility - filescan

- Can show the paths to ALL the files that were open on the computer

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 filescan
```

  - Output to txt file

- Will also get offset to where each file resides in the memory
  - Can use this information later in dumpfiles to try and carve that specific file from the memory!

Again a very useful command, “what was open and begin worked on at the time the system was imaged?”, is a good question to ask.

Copyright 2021 – CSITech Ltd

Volatility - filescan

- Could just focus on the files open in a particular profile

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 filescan
| findstr /i \Beth\ > c:\temp\filebeth.txt

vol.py –f c:\Win10.img --profile=Win10x64_14393 filescan
| findstr /i \harrison\ > c:\temp\fileharr.txt
```


Using the Findstr command we can isolate what we are looking for. In this example we are just looking for \beth\ which should find file paths just in Sarah's user space.

Note if we are using volatility along with findstr, it has to be the last command before >!

Copyright 2021 – CSITech Ltd

Volatility - filescan

```
vol.py –f c:\Win10.img --profile=Win10x64_14393
filescan
```

- Focus on Harrison's profile:
  - We saw evidence of cryptocurrency usage
    - What wallet files can you find?
      - Search for wallet
  - How many txt files were open?
    - Which is most likely to be user created?

Using the Findstr command we can isolate what we are looking for. In this example we are just looking for \beth\ which should find file paths just in Sarah's user space.

Note if we are using volatility along with findstr, it has to be the last command before >!

Copyright 2021 – CSITech Ltd

Volatility - yarascan

- YARA is a tool designed to help malware researchers identify and classify malware samples
- It's been called the pattern-matching Swiss Army knife for security researchers
- Yarascan - volatility plugin that scan a memory image for yara signature
  - Can be uses with rule file (use –y flag) or you can define what are you looking for on the fly – e.g. specific string (use –Y flag)
- Will take some time to run depending on the RAM size

Be careful, the commands might be different in linux
The –y and –Y are case sensitive so be careful

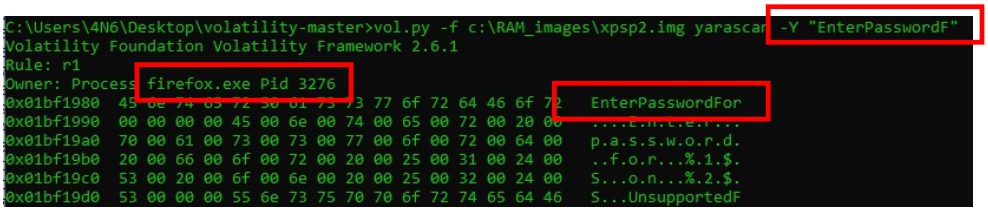
Volatility - yarascan

- YARA installation
- Relatively easy process
- Lets make sure that our Python 2.7 is as up to date as possible
  - `python –m pip install --upgrade pip setuptools wheel`
- From *"Volatility setup\Installers\Python 2.7 installers" CMD, run:*
  - `python –m pip install yara_python-3.11.0-cp27-cp27m-win32.whl`

Volatility - yarascan

- Scenario
  - You have a string output of your RAM image
  - You open RAM image in an HEX editor
  - There is an interesting line, code snip, IP address, word… but don't know where is it coming from – which process does it belong to?
- Yarascan

```
vol.py –f <path2image> --profile=<setProfile>
yarascan -Y "<search string>"
```



Volatility - yarascan

- In our RAM image there is a mention of cryptocurrency usage and there are traces of an IP address:

  “136.34.203.180”
- Which app is referencing that value?
  - Firefox, Electrum, Tor
  - Our investigation can go in different directions based on the answer
- Let’s take that address and search for it:

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 yarascan
-Y "136.34.203.180" > c:\temp\yara.txt
```


We are using a snip of the address as there is very little chance that we have those exact characters in the same order just randomly. In addition, we can find more fragments!
We can first run the address through Strings output or do a quick search in HEX viewer – you can use HxD from your thumb drive in General Tools folder

Volatility - yarascan

- It will take a bit to run, but will get the results eventually:

```
C:\Users\DFIR\Desktop\volatility-master\volatility-master>vol.py -f c:\Win10.img --profile=Win10x64_14393 yarascan -Y "136.34.203.180"
Volatility Foundation Volatility Framework 2.6.1
Rule: r1
Owner: Process bitcoin-qt.exe Pid 7548
0x24a687e80990  31 33 36 2e 33 34 2e 32 30 33 2e 31 38 30 3a 38   136.34.203.180:8
0x24a687e80a0  33 33 33 00 00 00 00 00 88 ab 5f 2c 00 7a 05 88   333......._,.z..
0x24a687e80b0  d0 a2 44 68 4a 02 00 00 3d 10 00 00 00 00 00 00   ..DhJ...=.......
0x24a687e80c0  29 10 00 00 01 02 00 00 8e ab 5d 2c 00 7b 05 80   ).........],.{..
0x24a687e80d0  36 34 2e 31 33 37 2e 31 33 32 2e 32 3a 38 33 33   64.137.132.2:833
0x24a687e80e0  33 00 04 00 12 00 00 00 8c ab 5b 2c 4a 7c 05 88   3.........[,J|..
```

- The IP address has much greater meaning than before…

Volatility - procdump

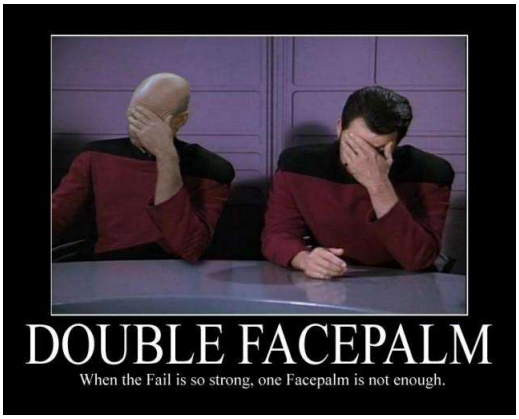
- Attempts to carve the process out of RAM into an exe
- This means we can virus check the process!!
- Bye Bye trojan defense

Procexedump will attempt to file carve the original executable from the RAM dump. This can be useful in seeing information about the executable, analyzing for malware or simply to see the associated icon when one was embedded.

Copyright 2021 – CSITech Ltd

Volatility - procdump

- DO THIS IN A VIRTUAL MACHINE!
- Carving out malware onto your primary PC is a bad thing!



NOTE – you could well be carving viruses out of the RAM dump onto your hard drive. In 'real world' work we would recommend a virtual machine. You really need to turn off AV as it will try to clean and delete your new executable which isn't what you want.

Copyright 2021 – CSITech Ltd

Volatility - procdump

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 procdump
-D c:\temp
```

- Requires output directory and it has to exist
  - -D for folder to dump exe's in
  - Add -p for PID specification
- Creates files in your output folder with file names as:
  - executable.<PID>.exe
- Is normal to get errors related to paging and PEB – will explain later on in the class

Note the use of the –D switch here to tell Volatility where to put the exe's. Still use >> for the report.

Copyright 2021 – CSITech Ltd

Volatility - procdump

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 procdump
-D c:\temp
```

- What exe is responsible for Cortana?
  - Hint: add “File description” column to your explorer
- Upload Electrum to Virustotal
  - What ‘Malware’ is tagged?

If the process is hidden it will be skipped! You have to specify PID of hidden process
Europol virus scanner - https://emas.europol.europa.eu/authentication/login

Copyright 2021 – CSITech Ltd

MFTparser

- Extract and parse the Master File Table (MFT)!
- Includes the Created, Accessed, Modified and MFT modified dates for every file
- If a file is less that 1024b, its data is in the MFT
- An extraordinary resource
- Can be used to confirm what was found with filescan
  - Find additional files and info

All information about a file, including its size, time and date stamps, permissions, and data content, is stored either in MFT entries, or in space outside the MFT that is described by MFT entries.

Copyright 2021 – CSITech Ltd

MFTparser

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 mftparser >
c:\temp\mft.txt
```

- Open with Notepad ++
- What is the modified date for cookies.sqlite-shm? (One of Harrison's cookies we saw in Handles earlier)

If you are bored scrolling down the HUGE output, simply use Notepad++ search feature!

Copyright 2021 – CSITech Ltd

Dumpfiles

- Dumpfiles reaches into the MFT and attempts to find and carve the actual files that were open
- Creates virtual files that sometimes need interpreting a little
- Can get us right to the data we need
- Optional switches are available
  - -n tries to preserve the original file name
  - -p will focus on the files used by a specific process PID

Copyright 2021 – CSITech Ltd

Dumpfiles

- Create a folder called dump1 in temp
- Let's try and see the contents of the cookie file that popped up in the Handles command

```
vol.py –n –p 7612 –f c:\Win10.img --profile=Win10x64_14393 dumpfiles
–D c:\temp\dump1
```


The –n switch inserts the recovered file name into the new file name making it easier for us to find what we are looking for. We specify the PID as we are looking for browser cookies, hence the file will be recoverable from the process space of Firefox.

Copyright 2021 – CSITech Ltd

Dumpfiles

```
vol.py -n -p 7612 -f c:\Win10.img --profile=Win10x64_14393 dumpfiles
-D c:\temp\dump1
```

- In handles we saw cookie files called cookies.sqlite – use NP++ to open the 'vacb' file
- Back when we ran the netscan command, we saw Firefox was talking to Wordpress (192.0.77.32)
  - What Wordpress site can be found in the dumped cookie files?

Copyright 2021 – CSITech Ltd

Dumpfiles

- Similar to the MFT case, using the knowledge from filescan output, we can try and dump/carve listed files out of the memory
- Filescan gave us the output with noted **PHYSICAL** address of where files reside in memory:

```
Offset(P)            #Ptr   #Hnd Access Name
------------------ ------ ------ ------ ----
0x0000850000040930     24      0 RW-rwd \Device\HarddiskVolume3\$Mft
0x000085000004e080     16      0 RW-rwd \Device\HarddiskVolume3\$Directory
0x0000850000052960  32737      1 ------ \Device\DeviceApi\CMNotify
0x0000850000054a10  32763      1 R--r-- \Device\HarddiskVolume3\Windows\Registration\R000000000006.clb
```


Copyright 2021 – CSITech Ltd

Dumpfiles

- For all the files in the filescan list, there is a good chance they were loaded into the memory at the time RAM was dumped and therefore we can try and get them
- Similar command as before with noted **PHYSICAL** address of the file with switch –Q
- To dump the MFT we would use

```
vol.py dumpfiles –n –Q 0x000085000040930
–f c:\Win10.img --profile=Win10x64_14393
–D c:\temp\dump1
```

Symlinkscan

- Scans for symbolic link objects and outputs their information
- Can be used to link drive letters (i.e. D:, E:, F:, etc) to external or encrypted volumes (i.e. \Device\TrueCryptVolume)

- `vol.py -f c:\Win10.img --profile=Win10x64_14393 symlinkscan > c:\temp\symlink.txt`

- Any hits or clues on our Veracrypt process we found earlier?
- Any other external devices referenced?

Search for all drive letters [A-Z]:

#PTR – page table registry
#Hnd - handles

Volatility - userassist

- Pslist and psscan will show what is running on the system
- With userassist we can know when and how many times something was run on the system
- Displays programs executed on a Windows machine, complete with running count and last execution date and time
- Can prove a lot about user activity

UserAssist allows investigators to see what programs were recently executed on a system. This can be extremely valuable in an investigation where an examiner wishes to see if a particular application was run, such as an encryption or wiping tool. UserAssist data will include information on whether an application was run from a shortcut (LNK file) or directly from the executable. This provides examiners with additional context around the execution of a program

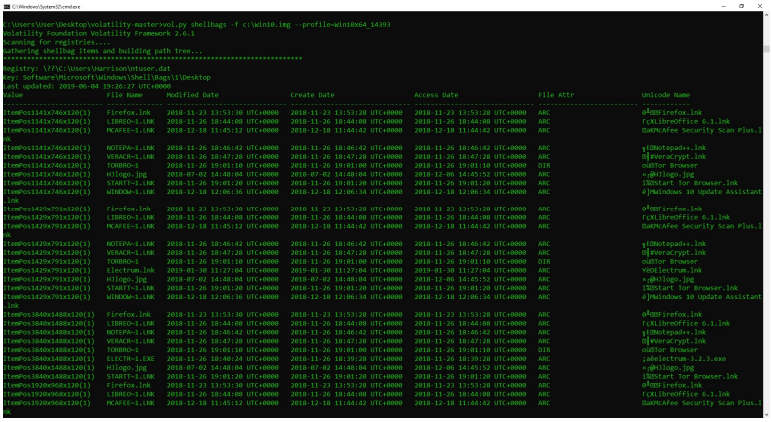
Application prefetch files are not specific to a user.
UserAssist consists of a series of Registry subkeys within the user's NTUSER.DAT hive, that have the effect of recording information about the user's interaction with the system via the Windows Explorer shell.

Will not work on later versions of Windows unfortunately as the REG key changed

Copyright 2021 – CSITech Ltd

Volatility - userassist

- `vol.py -f c:\Win10.img --profile=Win10x64_14393 userassist > c:\temp\userassist.txt`



Copyright 2021 – CSITech Ltd

Volatility - userassist

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 userassist >
c:\temp\userassist.txt
```

- How many times was cmd.exe run?
  - How long was it in focus for?
- We saw VeraCrypt in use in pslist.
  - How many times has it been run?
- Any other interesting programs ran?

No run counts? https://medium.com/@forensic_matt/no-run-counts-in-userassist-4f2a78cb9e2c
IDRIX – VeraCrypt core dev

Copyright 2021 – CSITech Ltd

Volatility - Shellbags

- Parses registry information for Shellbag data
  - Track user window viewing preferences specific to Windows Explorer
    - Window size
    - View type
    - Pane sizes
    - Sorting etc
  - In essence - What folders a user account has navigated to
- Can help determine file system traversal activity
- Prove that certain interactions between the user and system must have, or must not have happened during a certain time period

http://www.dfrws.org/sites/default/files/session-files/paper-using_shellbag_information_to_reconstruct_user_activities.pdf

Copyright 2021 – CSITech Ltd

Volatility - Shellbags

- `vol.py -f` **`c:\xpsp2.img`** `shellbags`
- Run the command

- What folders have been browsed to?
- Did the user open the folders Downloads or Program files?
- Try the Win10.img
  - What was the most used folder?

ARC – Archive
DIR - direcory

Copyright 2021 – CSITech Ltd

Volatility – Win 10 toast notifications

- In Windows 10, whenever a notification message is required to show to the user, its shown in the form of a toast notification
- Toast notifications are a part of the new modern UI of Windows 10 which are shown in rectangular shape using slide-in and slide-out animation
- Notification types:
  - Messaging, emails, USB related events, VPN, Windows applications, brower notification, network changes, calendar….
- Rolf Govers and Max de Bruijn created a parser for the Volatilty 2019 annual contest

Volatility – Win 10 toast notifications

- Plugin: https://github.com/volatilityfoundation/community/tree/master/WindowsToastNotifications
- Copy the plugin from the Volatility setup\Python Plugins\toastplugin.py to volatility-master\volatility\plugins
- *requires Yara to be installed

- `vol.py toastplugin -f c:\Win10.img --profile=Win10x64_14393`

- What notifications can you find?

Volatility - screenshot

- Below is the screenshot from xpsp2.img on your key

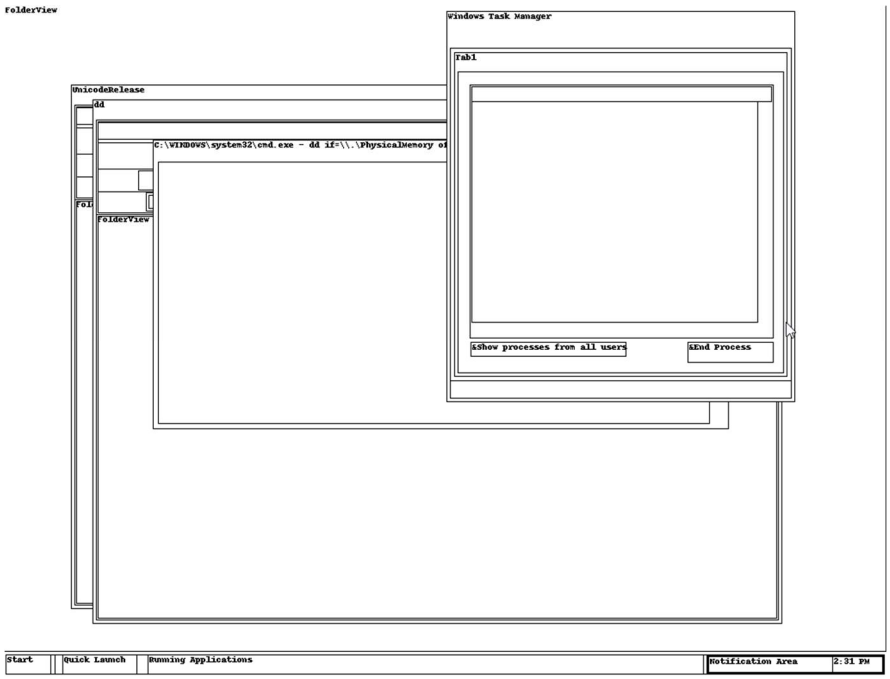
```
vol.py screenshot -f c:\xpsp2.img –D c:\temp
```

I would always recommend taking photographs at the scene of a live acquisition and this would include the screen. However, in times where this was missed, where RAM was taken covertly and other possibilities, one can recreate the screen as it was when it was imaged.

The excellent thing about this is that if there other logged in users you also get a wireframe of their screen which would not have been possible with a photograph.

Copyright 2021 – CSITech Ltd

Volatility - screenshot



Copyright 2021 – CSITech Ltd

Creating a custom Windows Volatility 2 profile

- Process….Not great not terrible 😑
- Things needed:
  - Taking the ntoskrnl.exe form the target system
    - Located in C:\Windows\System32
    - Will need this to generate vtypes
  - Pdbparse python 2 script - https://github.com/moyix/pdbparse
    - Used for generating the pdb files – symbols tables for specific windows distribution
    - Used for parsing the pdb files
    - Will have some dependencies for it to work properly
    - Sometimes can be a pain to make it work
  - Microsoft Visual C++ Compiler for Python 2.7
    - Needed for proper installation od compilation of pdbparse

PDB files are arranged into streams, each of which contains a specific bit of debug information; for example, stream 1 contains general information on the PDB file, and stream 2 contains information on data structures.

Creating a custom Windows Volatility 2 profile

- Copy out ntoskrnl.exe form the target system to C:\TEMP
  - Located in C:\Windows\System32
- Install VCForPython27.msi from *"Volatility setup\Installers\Python 2.7 installers"*
- Installation of Pdbparse python 2 script
  - Download or just copy form *"Volatility setup\Python Plugins" to TEMP*
  - Will need to "fix" our python 2 installation first
  - In CMD run the following commands

```
Python –m pip install --upgrade setuptools pip wheel
```

Creating a custom Windows Volatility 2 profile

- Installation of Pdbparse python 2 script
  - In "TEMP/pdbparse-master" folder run the command to install pdbparser

```
python –m pip install pdbparse
```

  - *There will be an error related to the version of construct, disregard
- Copy the ntoskrnl.exe to the *"…\pdbparse-master\examples\"*
  - Open CMD and run:

```
symchk.py -e ntoskrnl.exe
pdb_tpi_vtypes.py ntkrnlmp.pdb > dump.py
```


`symchk.py will analyse the` ntoskrnl.exe and than connect to the MS servers to download the symbols tables that correspond to that kernel for that specific version of windows
The second command will convert the symbols tables into vtypes that are needed for the volatility profile – that will actually be our volatility profile.

Creating a custom Windows Volatility 2 profile

- The resulting dump.py file is actually our volatility 2 profile
- There are few steps before we can use it
  - **YOU NEED TO** rename the dump.txt to the corresponding Windows version with suggested format:

    win<XX>_x64_<XXXXXX>_vtypes.py

  - win<XX> - 10, 11, 12? ☺
  - <XXXXX> - specific minor windows version e.g. 19044

Windows 11	Windows 11 (21H2)	10.0.22000
Windows 10	Windows 10 (21H2)	10.0.19044
	Windows 10 (21H1)	10.0.19043
	Windows 10 (20H2)	10.0.19042
	Windows 10 (2004)	10.0.19041

To find the windows version on the target machine, simply look at System information application and check the Version info, or open CMD and type systeminfo and output to .txt file

Creating a custom Windows Volatility 2 profile

- The win<XX>_x64_<XXXXXX>_vtypes.py goes with its friends in *"...volatility-master\volatility\plugins\overlays\windows"*
  - You can now see why the name…
- Last thing we need is to modify the win10.py script
  - That's our list of win10 profiles that we see in Volatility info
  - There is no list for 11, 12+ version of Windows so we will just use the 10

  "…\volatility-master\volatility\plugins\overlays\windows\win10.py"
  - Open the file with np++
  - Scroll all the way to the end
  - We will copy and paste the last class and use it as our template for the new profile

Creating a custom Windows Volatility 2 profile

- Last thing we need is to modify the win10.py
  - We will modify the last class to **match the profile and filename we just created**

```
1139 class Win10x64_19041(obj.Profile):
1140     """ A Profile for Windows 10 x64 (10.0.19041.0 / 2020-04-17) """
1141     _md_memory_model = '64bit'
1142     _md_os = 'windows'
1143     _md_major = 6
1144     _md_minor = 4
1145     _md_build = 19041
1146     _md_vtype_module = 'volatility.plugins.overlays.windows.win10_x64_19041_vtypes'
1147     _md_product = ["NtProductWinNt"]
```

  - Eg – set all to Win10 x64 19044
  - Save and verify by running in CMD
  - **BE CAREFUL** not to include the *.py extension in _md_vtype_module
  - `Vol.py --info` and see if your new profile is there under Windows profiles

We will probably get …. Cannot find a 32-bit equivalent profile as… exactly what it says…we don't care :D
MAKE SURE the file in overlays/widows matches the name in the _md_vtype_module line EXACTLY!!!!!!!!!

Tested one some versions up to Win11 22H2. Sometimes it works sometimes not, or just few commands work…try and see

Volatility 3

The same – but different

Volatility 3

- Launched October 2019 at OSDF Conference
- New Custom Licence (VSL)
- Significantly faster (much of the time) – Python 3
- Easier to run – no profiles

- Limited plugins available at the time of writing
- Still buggy
- V2.x will stop being developed in 2021

Volatility 3

- Some significant differences to Volatility 2.x

- Although you can check that your RAM dump is support, you do not need to run imageinfo or kdbginfo.
- No profile needs to be included in the command path
- Format of the commands is different
- In some cases will need Internet connection for proper profile interpretation

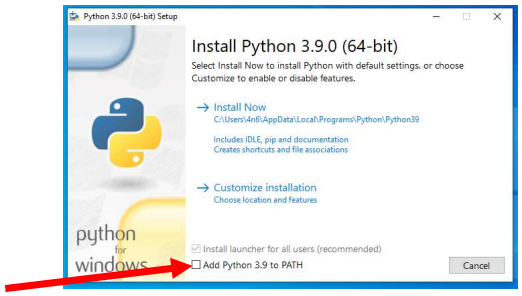
Volatility 3

- Latest version 2.4.0 - Dec 14, 2022
  - New plugins
    - linux.mountinfo
    - linux.psaux
    - windows.devicetree
    - windows.joblinks
    - windows.ldrmodules
    - windows.mbrscan
    - windows.mftscan
    - windows.sessions
  - Introduced the concept of modules and module requirements
  - Unified symbol handling and ISF file caching between OS versions
  - Better QEVM support (fixed the QEMU PCI hole)
  - Exposed an API for automatic PDB symbol table use
- Previous versions - https://github.com/volatilityfoundation/volatility3/releases

Volatility 3 setup

- Download the latest release from - https://github.com/volatilityfoundation/volatility3

- Install Python **3.6+ 64bit** - https://www.python.org/downloads/
- Make sure you **select** – **add to PATH** on the first screen when installing



Documentation: https://readthedocs.org/projects/volatility3/downloads/pdf/latest/

Volatility 3 setup

- Install Python **3.6+ 64bit** - https://www.python.org/downloads/
- Make sure you select – **add to PATH** on the first screen when installing

- **If you have python 2 already installed, you will need to specify the Python version at the start of each command**

  - `Py -2`
  - `Py -3`

Documentation: https://readthedocs.org/projects/volatility3/downloads/pdf/latest/

Volatility 3 setup

- Unzip the volatility3-2.4.0.zip form Volatility Setup folder
- Copy volatility3-2.4.0 folder from to Desktop
- This will be our master folder from where all Volatility 3 commands will run

- **Please make sure you are in the right folder and don't mix Vol2 with Vol3!**

- **Close all running CDM's or WT's**

Volatility 3 setup

- Installing dependencies to enable the full range of Volatility 3 functionality – from Volatility 3 Master folder on desktop
  - `pip3 install --upgrade pip setuptools wheel`
  - `pip3 install -r requirements.txt`
- Run to check if any plugins error out
  - `py -3 vol.py -vv`
- *If you followed along the steps, we can omit the `py -3`, and all should work properly

How to fix the whole yara-python installation - https://github.com/VirusTotal/yara-python/issues/150
Download the wheel file from https://pypi.org/project/yara-python/#files
Rename it from yara_python-4.0.2-cp38-cp38m-win_amd64.whl to yara_python-4.0.2-cp38-cp38-win_amd64.whl ("cp38m" -> "cp38")

py -3 -m pip install yara_python-4.0.2-cp38-cp38-win_amd64.whl

For Microsoft Visual C++ run vs_buildtools__511488784.1606247133.exe, in Workloads select only C++ Buildtools

After install and reboot, set the environmental variable for MS VS, in CMD rum:
set CL=/FI "C:\Program Files (x86)\Microsoft Visual Studio\2019\BuildTools\VC\Tools\MSVC\14.28.29333\include\stdint.h" %CL%

Run:

```
py -3 -m pip install wheel
py -3 -m pip install pycrypto
```

Pycrypto, offline install - unzip the pycrypto-2.6.1.tar.gz and run *py -3 setup.py install*

https://stackoverflow.com/questions/41843266/microsoft-windows-python-3-6-pycrypto-installation-error

Volatility 3 setup

- You can download the Symbol (Profile) packs
  - Already in *"Volatility setup\Installers\Python 3 installers"*

- https://downloads.volatilityfoundation.org/volatility3/symbols/windows.zip
  https://downloads.volatilityfoundation.org/volatility3/symbols/mac.zip
  https://downloads.volatilityfoundation.org/volatility3/symbols/linux.zip

- After download, you copy the zip files to ~/ volatility3-1.0.1/volatility3/symbols

Most compiled programs know of their own templates, and define the structure (and location within the program) of these templates as a Symbol. A Symbol is often an address and a template and can be used to refer to either independently. Lookup tables of these symbols are often produced as debugging information alongside the compilation of the program. Volatility 3 provides access to these through a SymbolTable

Volatility 3 setup

- You can list available plugins:
  - `py -3 vol.py --help`
- Commands look different, prefixed with OS type
- **Order of commands is important!**
- List seems to show command twice – windows.pslist.PsList
- It is to allow nested commands such as:
  - windows.vadinfo.**VadInfo**
  - windows.verinfo.**VerInfo**
- **When running, just type the <os>.<last_Command>**
  - E.g. `windows.vadinfo or windows.registry.printkey`

Volatility 3 setup

```
                    image.
mac.pstree.PsTree   Plugin for listing processes in a tree based on their
                    parent process ID.
mac.tasks.Tasks     Lists the processes present in a particular mac memory
                    image.
mac.trustedbsd.trustedbsd
                    Checks for malicious trustedbsd modules
timeliner.Timeliner
                    Runs all relevant plugins that provide time related
                    information and orders the results by time.
windows.cmdline.CmdLine
                    Lists process command line arguments.
windows.dlldump.DllDump
                    Dumps process memory ranges as DLLs.
windows.dlllist.DllList
                    Lists the loaded modules in a particular windows
                    memory image.
```

Volatility 3 setup

- You can check that your RAM dump is supported:
  *Note the command must be last or it will error out

  - `py -3 vol.py -f c:\win10.img windows.info`

```
Variable        Value

Kernel Base     0xf803a3e8c000
DTB     0x1ab000
Symbols file:///C:/Users/MPCCU%20VM%201/Desktop/vo
primary 0 WindowsIntel32e
memory_layer    1 FileLayer
KdVersionBlock  0xf803a4181dc8
Major/Minor     15.14393
MachineType     34404
KeNumberProcessors      1
SystemTime      2019-06-05 10:40:41
NtSystemRoot    C:\Windows
NtProductType   NtProductWinNt
NtMajorVersion  10
NtMinorVersion  0
PE MajorOperatingSystemVersion  10
PE MinorOperatingSystemVersion  0
PE Machine      34404
PE TimeDateStamp        Fri Mar 30 03:25:04 2018
```

Volatility 3 setup

- Commands are simple with no profile
- Best to run complete command by if only one command in the nested structure than can shorten the command
- Case non-sensitve
  - `py -3 vol.py –f c:\win10.img windows.pslist.PsList`
- Can just run
  - `py -3 vol.py –f c:\win10.img windows.pslist`

Volatility 3 setup

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wow64	CreateTime	ExitTime
4	0	System	0xa28760a61440	132	-	N/A	False	2019-06-05 10:21:38.000000	N/A
512	4	smss.exe	0xa2876233c040	4	-	N/A	False	2019-06-05 10:21:38.000000	
612	604	csrss.exe	0xa287627d3080	11	-	0	False	2019-06-05 10:21:44.000000	
668	512	smss.exe	0xa2876292c400	0	-	1	False	2019-06-05 10:21:45.000000	
676	668	csrss.exe	0xa2876298e080	11	-	1	False	2019-06-05 10:21:45.000000	
692	604	wininit.exe	0xa287629a0080	4	-	0	False	2019-06-05 10:21:45.000000	
756	692	services.exe	0xa28762d41080	23	-	0	False	2019-06-05 10:21:46.000000	
764	692	lsass.exe	0xa28762d40080	10	-	0	False	2019-06-05 10:21:46.000000	
776	668	winlogon.exe	0xa28762d3c080	5	-	1	False	2019-06-05 10:21:46.000000	
864	756	svchost.exe	0xa28762d2a800	45	-	0	False	2019-06-05 10:21:47.000000	
904	756	svchost.exe	0xa28762d47800	19	-	0	False	2019-06-05 10:21:47.000000	
1008	776	dwm.exe	0xa28762e7c800	11	-	1	False	2019-06-05 10:21:47.000000	N/A
628	756	svchost.exe	0xa28762e52800	94	-	0	False	2019-06-05 10:21:48.000000	
840	756	svchost.exe	0xa28762609800	21	-	0	False	2019-06-05 10:21:48.000000	
1068	756	svchost.exe	0xa2876264e700	19	-	0	False	2019-06-05 10:21:49.000000	
1076	756	svchost.exe	0xa28762618800	34	-	0	False	2019-06-05 10:21:49.000000	
1084	756	svchost.exe	0xa28762663800	29	-	0	False	2019-06-05 10:21:49.000000	
1256	756	vmacthlp.exe	0xa287629c7080	2	-	0	False	2019-06-05 10:21:51.000000	
1384	756	svchost.exe	0xa28760acd800	23	-	0	False	2019-06-05 10:21:53.000000	
1428	756	svchost.exe	0xa28762c78800	9	-	0	False	2019-06-05 10:21:53.000000	
1488	756	svchost.exe	0xa28762ca5080	10	-	0	False	2019-06-05 10:21:53.000000	
1616	756	spoolsv.exe	0xa28762ccf800	15	-	0	False	2019-06-05 10:21:53.000000	
1812	756	svchost.exe	0xa28762a65080	13	-	0	False	2019-06-05 10:21:54.000000	
1904	756	svchost.exe	0xa28762ab6800	5	-	0	False	2019-06-05 10:21:54.000000	
1960	756	svchost.exe	0xa28760aed800	12	-	0	False	2019-06-05 10:21:55.000000	
1968	756	vmtoolsd.exe	0xa28762b19800	9	-	0	False	2019-06-05 10:21:55.000000	

Volatility 3 setup

- Currently output to >> as normal
- Output is a bit messy
- Add `–r pretty` to the command line
- **Needs to be after vol.py!!!**

- `py -3 vol.py –r pretty –f c:\win10.img windows.pslist`

Volatility 3 setup

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wow64
4	0	System	0xa28760a61440	132	-	N/A	False
512	4	smss.exe	0xa2876233c040	4	-	N/A	False
612	604	csrss.exe	0xa287627d3080	11	-	0	False
668	512	smss.exe	0xa2876292c400	0	-	1	False
676	668	csrss.exe	0xa2876298e080	11	-	1	False
692	604	wininit.exe	0xa287629a0080	4	-	0	False
756	692	services.exe	0xa28762d41080	23	-	0	False
764	692	lsass.exe	0xa28762d40080	10	-	0	False
776	668	winlogon.exe	0xa28762d3c080	5	-	1	False
864	756	svchost.exe	0xa28762d2a800	45	-	0	False
904	756	svchost.exe	0xa28762d47800	19	-	0	False
1008	776	dwm.exe	0xa28762e7c800	11	-	1	False
628	756	svchost.exe	0xa28762e52800	94	-	0	False
840	756	svchost.exe	0xa28762609800	21	-	0	False
1068	756	svchost.exe	0xa2876264e700	19	-	0	False
1076	756	svchost.exe	0xa28762618800	34	-	0	False
1084	756	svchost.exe	0xa28762663800	29	-	0	False
1256	756	vmacthlp.exe	0xa287629c7080	2	-	0	False
1384	756	svchost.exe	0xa28760acd800	23	-	0	False
1428	756	svchost.exe	0xa28762c78800	9	-	0	False
1488	756	svchost.exe	0xa28762ca5080	10	-	0	False
1616	756	spoolsv.exe	0xa28762ccf800	15	-	0	False
1812	756	svchost.exe	0xa28762a65080	13	-	0	False
1904	756	svchost.exe	0xa28762ab6800	5	-	0	False
1960	756	svchost.exe	0xa28760aed800	12	-	0	False
1968	756	vmtoolsd.exe	0xa28762b19800	9	-	0	False
1976	756	VGAuthService.	0xa28762b11800	3	-	0	False

Volatility 3 setup

- Many commands are missing
- Currently output to >> as normal
- Recommend sticking with 2.x for the time being
- Can be helpful on large memory dumps as faster (mostly)
- Need to know how to use is as V3 will be the primary development platform

Volatility 3 setup

- Figure out the command and run:
  - Info
  - Psscan
  - Pstree
  - Netscan
  - Cmdline
  - Svcscan
  - Procdump
  - Handles

Handles will fail for this image
https://blog.onfvp.com/post/volatility-cheatsheet/?utm_source=pocket_reader

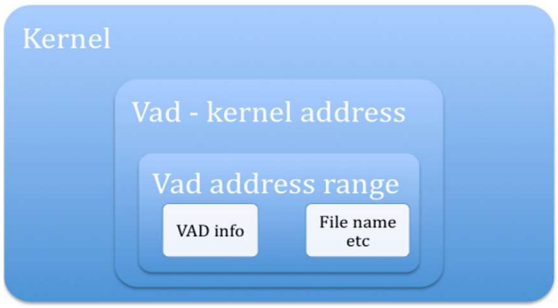
More about how memory works

Hardish bit

Copyright 2021 – CSITech Ltd

Volatility - svchost

- VAD – Virtual Address Descriptors
  - Contains info about each process and the process itself
  - Sets virtual addresses to a block within Kernel space
  - Contains linking information, can't be unlinked

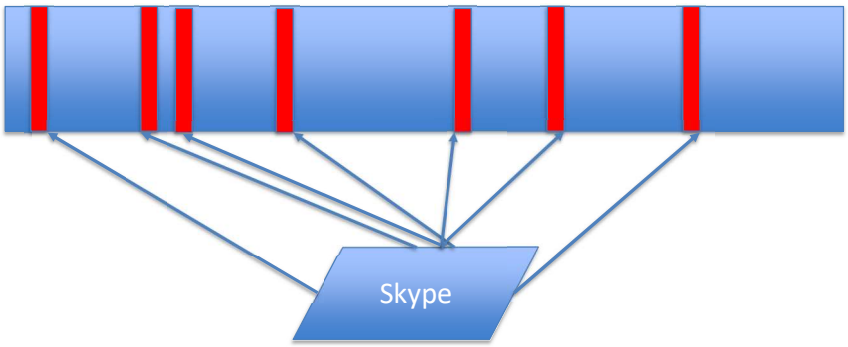


Whereas the PEB contains information ABOUT the process, the VAD contains a list of addresses which make up the entire process and process space in the RAM. Also, it can't be unlinked.

Copyright 2021 – CSITech Ltd

Volatility - svchost

- Processes in RAM really look like this



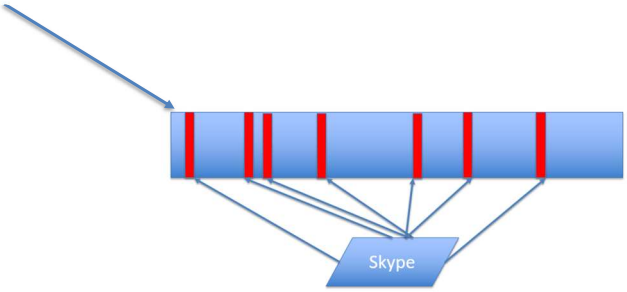
We talked earlier in the class about processes being in blocks. In reality they are not contiguous but are spread throughout the RAM space. It is the VAD that 'describes' each address space being used by the process. A Virtual Address Descriptor.

Copyright 2021 – CSITech Ltd

Volatility - svchost

- Each block of text in the VAD corresponds to a block of RAM

```
VAD node @820da750 Start 7ffdf000 End 7ffdffff Tag Vadl
Flags: MemCommit, NoChange, PrivateMemory
Commit Charge: 1 Protection: 4
ControlArea @823dbe30 Segment 823dbdec
Dereference list: Flink 823dbd0c, Blink 823dbcc8
NumberOfSectionReferences: 2185084532 NumberOfPfnReferences:  1598310742
NumberOfMappedViews:       2185048880 NumberOfUserReferences:     393216
WaitingForDeletion Event:  00000000
Flags:
FileObject: none
First prototype PTE: 00000000 Last contiguous PTE: 00000001
Flags2: LongVad, OneSecured
File offset: 00000000
```



Copyright 2021 – CSITech Ltd

Volatility - Vadinfo

- Run
  - Pslist and find the PID for a Svchost

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 vadinfo
-p <PID> > c:\temp\vadinfo.txt
```

  - In notepad++, search for svchost
    - This is hardcoded address
    - Cannot be faked – Win could not run it

If you search the output for the string ‘svchost’ you will find a node where the path on the disk is referenced. This cannot be faked and so definitively tells us that this is likely a legitimate svchost.

Copyright 2021 – CSITech Ltd

Volatility - vaddump

- As the VAD for a process contains the running process itself, we can use it to dump the memory space being specifically used by the process.
  - Includes the code
  - Files in use by the process
  - etc

You will remember that earlier in the class we looked at the difference between the space allocated for the executable and the space dedicated to the entire process with all its related files and work space. Dumping the entire space enables us to look for data attributable to the process. As the Vad file contains all the address blocks for each process we can use it to carve the whole space.

Copyright 2021 – CSITech Ltd

Volatility - vaddump

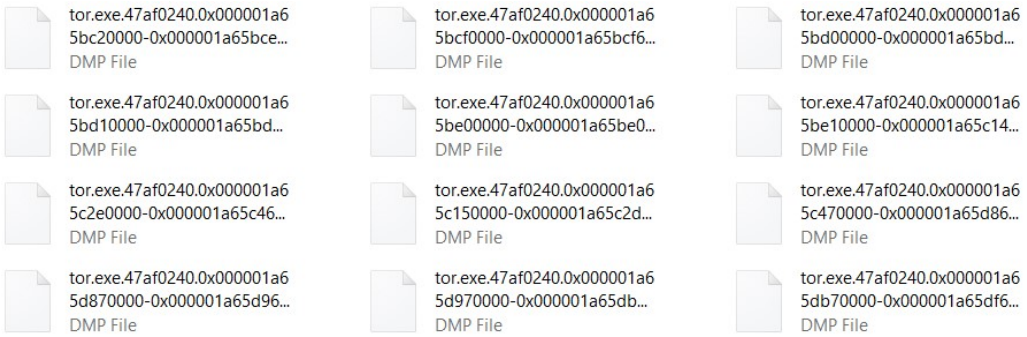
- Run vaddump to extract the **Tor** process space from Win10.img
- Have to make and specify the folder to write the files to

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 vaddump –p <pid>
–D c:\temp\vaddump
```


Copyright 2021 – CSITech Ltd

Volatility - vaddump

- Results are 1 file per VAD block

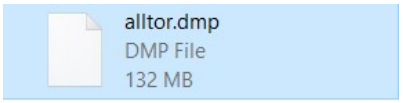


These files are all the blocks of the process from the RAM that were reference by the Vad. They need to be combined.

Copyright 2021 – CSITech Ltd

Volatility - vaddump

- CD into c:\temp\vaddump
- Combine them all
  - `Copy /b *.dmp alltor.dmp`



You can delete all the little files once you have a combined one.
All the files will be put together in right order based on their file names witch are numbered by how they reside in memory.

Copyright 2021 – CSITech Ltd

Volatility - vaddump

- Now use strings to find what we want
  - Strings finds all ACSII text for us
  - Strings is an exe on your key
- As we are looking at tor.exe we are interested in .onion strings
  - Use findstr to filter

```
strings64 c:\temp\vaddump\alltor.dmp | findstr /i .onion: >
c:\temp\onion.txt
```


Copyright 2021 – CSITech Ltd

Volatility - vaddump

```
strings64 c:\temp\vaddump\alltor.dmp | findstr /i .onion: >
c:\temp\onion.txt
```

- What onion sites were browsed to?
- Google is your friend!

Copyright 2021 – CSITech Ltd

Volatility - vaddump

```
strings64 c:\temp\vaddump\alltor.dmp > c:\temp\all_onion.txt
```

- Now run strings on the entire alltor.dmp file without findstr
- In Notepad++ Search for the onion sites to see what else can be found

Copyright 2021 – CSITech Ltd

Volatility - vaddump

```
strings64 c:\temp\vaddump\alltor.dmp > c:\temp\all_onion.txt
```

- What username did Harrison search for on pwndb?
- Can you find any ip addresses? Try searching for the tor IP address found in netscan

You could use regex searching for IP addresses
\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b

HINT: searching for requests form website can help

Copyright 2021 – CSITech Ltd

Volatility - vaddump

- Repeat the steps with the notepad++ process
  - Vaddump the process, combine the .dmp files then strings the combined dmp file
- What .txt files have been accessed in notepad++?
  - What drive letter is associated with the .txt files?

Copyright 2021 – CSITech Ltd

Volatility - vaddump

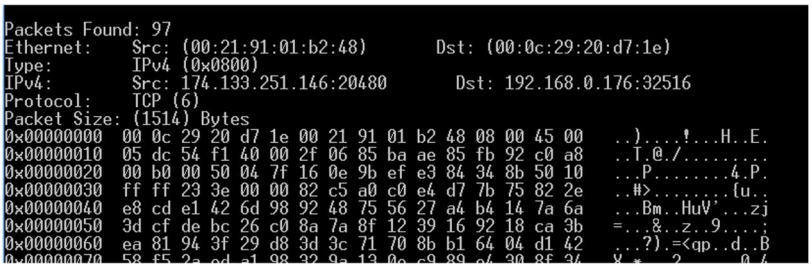
- Repeat the process with Firefox.exe (**PID=7612**)
  - Vaddump the process, copy the .dmp files then strings the combined dmp file
  - Use findstr to filter on the lines with https://
- What news articles were browsed to?
- What twitter post was looked at?
- Can you find a url of a picture of Trump?
- Can you find a connection between github and the earlier onion site?

Copyright 2021 – CSITech Ltd

Networking

Volatility – ethscan

- Ethscan carves the RAM dump for network data packets
- Can save as a standard PCAP file for analysis
- Very, VERY slow, but superb tool



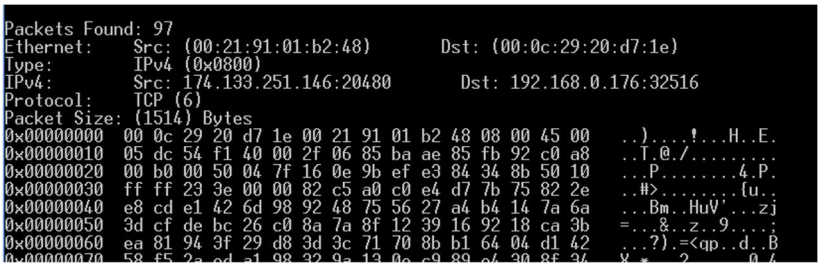
This is a recent (late 2013) and extraordinary capability, to be able to carve network packets out of a RAM dump. We will just look at HTTP traffic but this could be used to investigate illegal image distribution, Botnets, recreate chat sessions, almost anything.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- Copy ethscan.py from “Volatility setup\Python Plugins” to …“volatility-master\volatility\plugins”

```
vol.py ethscan –f c:\xpsp2.img
–D c:\temp –C xpsp2.pcap
```



The –D is the output directory and the –C the name of the .pcap file you want.

Copyright 2021 – CSITech Ltd

Bulk Extractor - pcap carving

- New capability from Bulk Extractor 1.4
- Scriptable ☺
- Go to:
  - C:\Program Files (x86)\Bulk Extractor 1.6.0-dev
  - Copy out bulk_extractor.exe to your temp directory

Copyright 2021 – CSITech Ltd

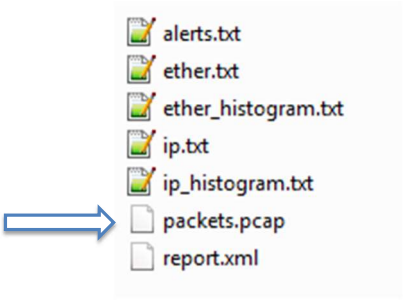
Bulk Extractor - pcap carving

- Usage:
  - bulk_extractor.exe –x all –e net –o <out dir> <path2image>
  - NOTE: output directory must not already exist
- Try in cmd:
  - `bulk_extractor.exe -x all -e net -o c:\temp1 c:\xpsp2.img`

Copyright 2021 – CSITech Ltd

Bulk Extractor - pcap carving

- Output should be similar to:



Since we ran network scanner plugin we additionally received alerts, ether and ip outputs.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- Ethscan can take hours/days to run, depends on speed of machine and size of RAM dump
- Bulkextractor is good free and fast alternative
- There are paid tools as well
  - CapLoader from Netresec – trail in your files

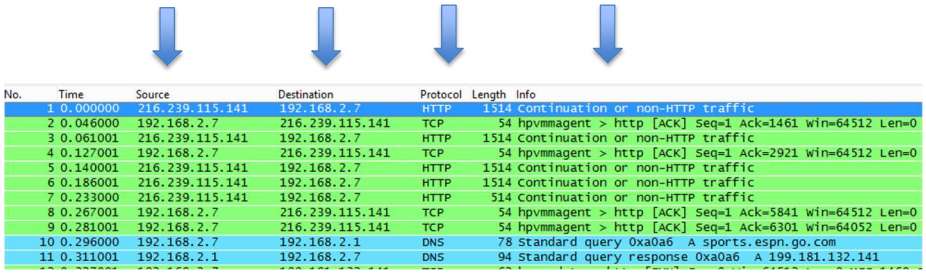
- For the next part we will examine a pcap dump for WinXP image
- Xpsp2.pcap is on your key in PCAP folder

Almost any packet capture analysis tool could be used to do the analysis but Wireshark is free and awesome. If you don't want to spend megabucks but want a more user friendly tool to recreate packet data back into what the user saw check out NetResident. - http://www.tamos.com/products/netresident/

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- Install Wireshark from general tools
- Open xpsp2.pcap



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	216.239.115.141	192.168.2.7	HTTP	1514	Continuation or non-HTTP traffic
2	0.046000	192.168.2.7	216.239.115.141	TCP	54	hpvmmagent > http [ACK] Seq=1 Ack=1461 Win=64512 Len=0
3	0.061001	216.239.115.141	192.168.2.7	HTTP	1514	Continuation or non-HTTP traffic
4	0.127001	192.168.2.7	216.239.115.141	TCP	54	hpvmmagent > http [ACK] Seq=1 Ack=2921 Win=64512 Len=0
5	0.140001	216.239.115.141	192.168.2.7	HTTP	1514	Continuation or non-HTTP traffic
6	0.186001	216.239.115.141	192.168.2.7	HTTP	1514	Continuation or non-HTTP traffic
7	0.233000	216.239.115.141	192.168.2.7	HTTP	514	Continuation or non-HTTP traffic
8	0.267001	192.168.2.7	216.239.115.141	TCP	54	hpvmmagent > http [ACK] Seq=1 Ack=5841 Win=64512 Len=0
9	0.281001	192.168.2.7	216.239.115.141	TCP	54	hpvmmagent > http [ACK] Seq=1 Ack=6301 Win=64052 Len=0
10	0.296000	192.168.2.7	192.168.2.1	DNS	78	Standard query 0xa0a6 A sports.espn.go.com
11	0.311001	192.168.2.1	192.168.2.7	DNS	94	Standard query response 0xa0a6 A 199.181.132.141

These are the core columns to look at. Where the packet came from, was sent to, what protocol it used and some information about it.

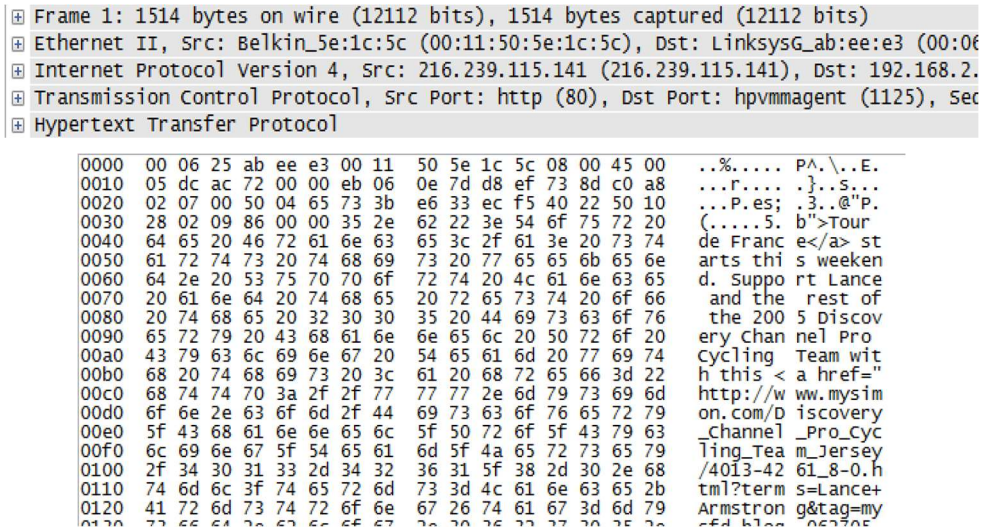
If you haven't used Wireshark before then this is a baptism of fire. If you want to learn more then this tutorial isn't bad - http://goo.gl/JhYhPo

I also recommend the book Practical Packet Analysis - http://goo.gl/nlLJVA

Colour coding – View - clulor

Copyright 2021 – CSITech Ltd

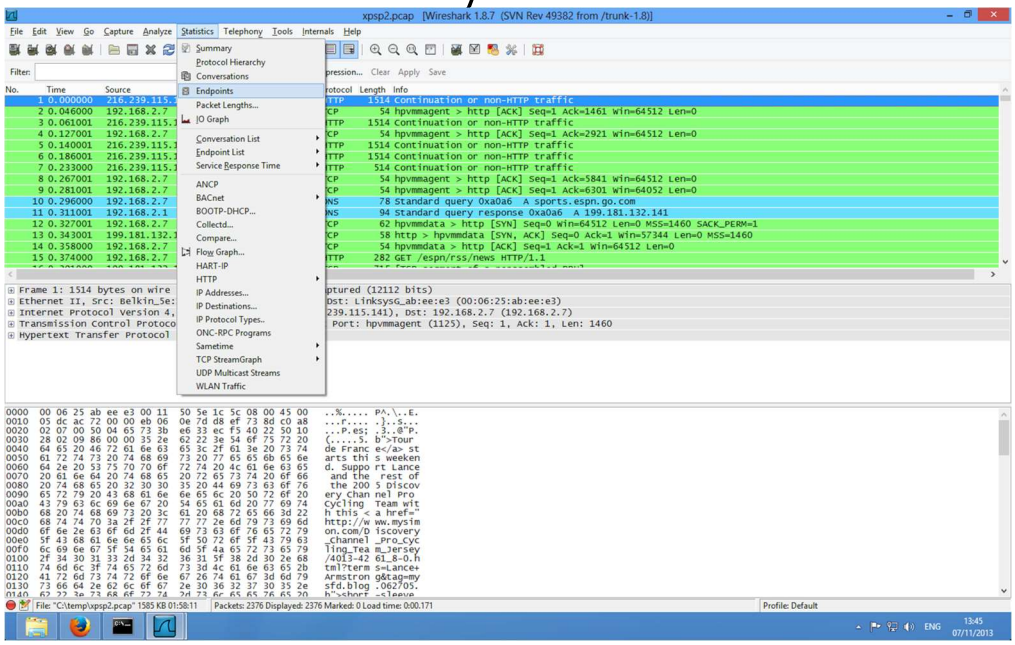
Volatility – ethscan



If you click on a packet the bottom pane is the raw data in the packet. The grey area, with the plus signs that can be expanded, are an interpretation of the raw data.

Copyright 2021 – CSITech Ltd

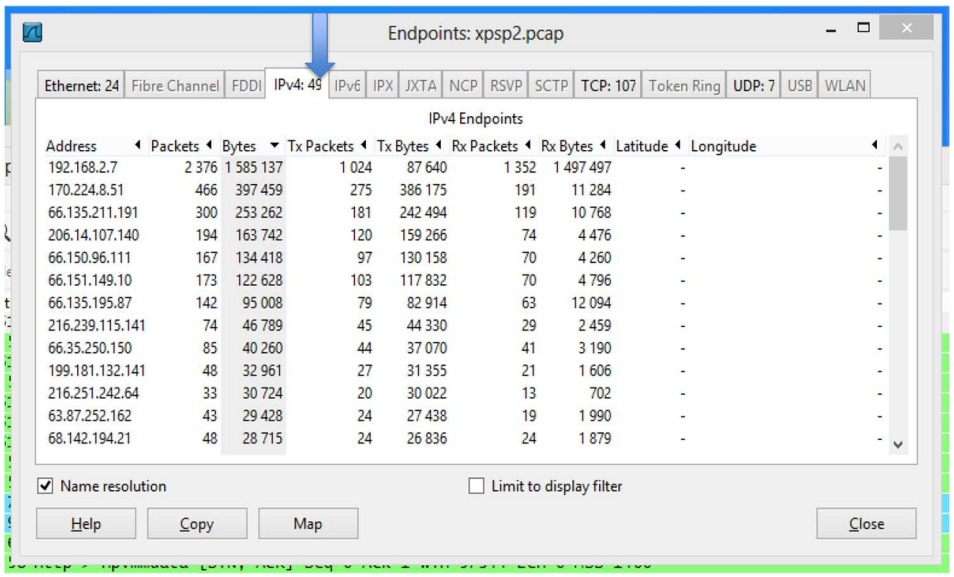
Volatility – ethscan



A good way to get an idea of the data 'conversations' that are happening is to browse to Statistics – Endpoints.

Copyright 2021 – CSITech Ltd

Volatility – ethscan



Next, do to the IPv4 tab. It is good to sort on the Packets column and see the IP addresses that had the most data sent and received by them. This helps us to see the primary sites, servers etc that were communicated with.

To apply a filter – right click on an item, apply as a filter and then selected.

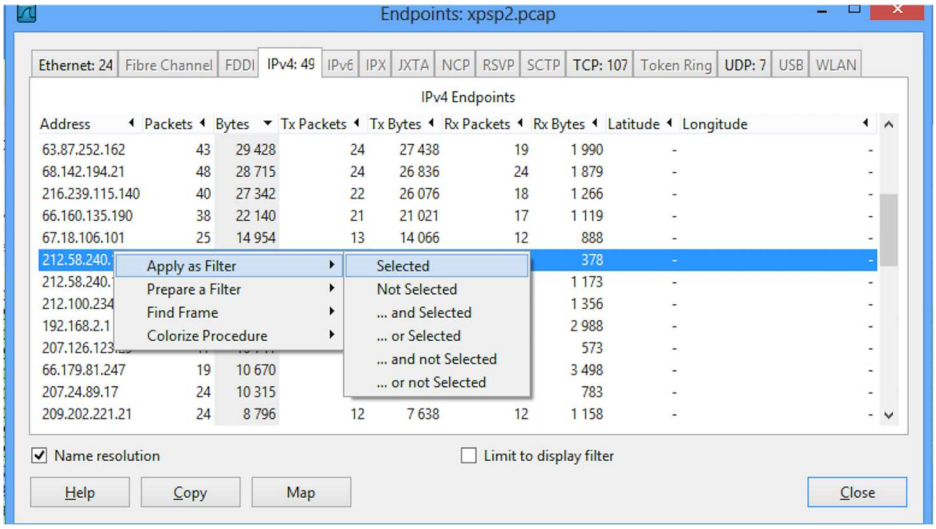
Copyright 2021 – CSITech Ltd

Volatility – ethscan

- Run pslist and connscan against xpsp2.img
  - `vol.py -f C:\xpsp2.img pslist`
  - `vol.py -f C:\xpsp2.img connscan`
- Note down the IP addresses and PID's
  - Cross reference PID's with pslist to find out what they were
- Find IP address Firefox was using
  - Look it up on https://viewdns.info/ or bulk on https://www.infobyip.com/ipbulklookup.php

Copyright 2021 – CSITech Ltd

Volatility – ethscan



In the endpoints dialogue, find the IP address Firefox was talking to, right click and Apply as Filter – Selected.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

```
32 2f 68 69 2f 61 73 69   c.co.uk/ 2/hi/asi
63 2f 34 36 34 38 31 32   a-pacifi c/464812
75 69 64 3e 0d 0a 09 09   1.stm</g uid>....
3e 4d 6f 6e 2c 20 30 34   <pubDate >Mon, 04
31 30 3a 33 30 3a 34 36    Jul 05  10:30:46
62 44 61 74 65 3e 0d 0a    GMT</pu bDate>..
69 74 65 6d 3e 0d 0a 20   ......</ item>..
0d 0a 09 09 09 3c 69 74         .. .....<it
20 20 20 20 20 3c 74 69   em>..         <ti
75 6d 6d 69 74 20 62 61   tle>G8 s ummit ba
65 73 20 61 72 65 20 64   ttle lin es are d
74 6c 65 3e 0d 0a 20 20   rawn</ti tle>..
65 73 63 72 69 70 74 69         <d escripti
72 73 20 61 6e 64 20 70   on>Leade rs and p
73 20 66 72 6f 6d 20 61   rotestor s from a
65 20 77 6f 72 6c 64 20   round th e world
```

338

By looking through the packets we can see ASCII text that would seem to have come from a web page or similar.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- Scroll down through the packets to see the raw data from the BBC site
- Select the first packet, right click and select – Follow TCP Stream

'Follow TCP Stream' attempts to collate all the packets in a conversation and make them as human readable as possible.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

This seems to be a page with Headlines of news stories

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- In Pslist we see that process 368 is PluckUpdater which Is a RSS feed reader
- Identify the IP addresses used
  - What adverts were being displayed
  - What RSS feeds had the user selected

Copyright 2021 – CSITech Ltd

Volatility – ethscan

```
        <item>
            <title>Lovely by Sarah Jessica Parker</title>
            <link>http://www.mysimon.com/
Lovely_by_Sarah_Jessica_Parker/4050-4144_8-6253322.html?
part=rss&amp;subj=mysblog&amp;tag=Lovely_by_Sarah_Jessica_Parker</link>
            <category>Fragrance</category>
            <description>&lt;img alt="Lovely by Sarah Jessica Parker" src="http://
www.mysimon.com/i/ms/fd/featprod/lovelysjp.jpg" border="0" height="120" width="120"
hspace="5" vspace="5" align="left" /&gt;<![CDATA[Nordstrom is now taking preorders for
<a href="http://www.mysimon.com/4013-4144_8-0.html?terms=Lovely+Sarah+Jessica
+Parker&tag=mysfd.blog.062705.a">Lovely, Sarah Jessica Parker's new fragrance</a> for
Coty. With lavender, orchid, amber, apple martini, and mandarin, Lovely is supposed to
capture SJP's casual glamour. Wear it with (casual) <a href="http://
www.mysimon.com/4013-4105_8-0.html?companyId=301759&tag=mysfd.blog.062705.a">Gap
clothes</a> and (glamorous) <a href="http://www.mysimon.com/4013-4105_8-0.html?
terms=manolo&tag=mysfd.blog.062705.a">Manolo Blahniks</a> as you watch rer
5a8
uns of <a href="http://www.mysimon.com/4016-4183_8-0.html?terms=Sex+and+the
+City&tag=mysfd.blog.062705.a">Sex and the City</a>. ]]></description>
            <pubDate>June 27, 2005, 11:32AM </pubDate>
        </item>

        <item>
            <title>Epson P-2000 Photo Viewer</title>
            <link>http://www.mysimon.com/
Epson_P_2000_Photo_Viewer/4050-6447_8-6253278.html?
part=rss&amp;subj=mysblog&amp;tag=Epson_P_2000_Photo_Viewer</link>
            <category>Digital Camera Accessories</category>
            <description>&lt;img alt="Epson P-2000 Photo Viewer" src="http://
www.mysimon.com/i/ms/fd/featprod/epsonp2000.jpg" border="0" height="120" width="120"
```


Here we can extract chucks of code from adverts that were loaded into the RSS reader

Copyright 2021 – CSITech Ltd

Volatility – ethscan

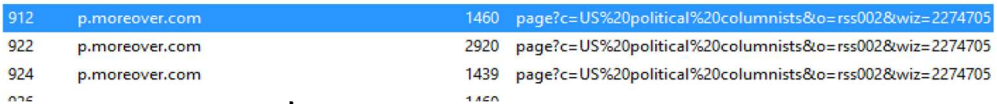
```
 encoding="iso-8859-1"?>
>

r Technologies - Sports: American football news</ti
ww.moreover.com/rss</link>
oreover Technologies - Sports: American football ne
real-time RSS news feeds.</description>
s</language>

ver Technologies</title>
i.moreover.com/pics/rss.gif</url>
/www.moreover.com/rss</link>
width>
height>
>Moreover Technologies - Real-time RSS news feeds f
ources...</description>
```


Do a little research on Moreover Technologies. Here we see links to sports stories.

Copyright 2021 – CSITech Ltd

Volatility – ethscan

- You can also reconstruct a file to some degree
- File – Export Objects – HTTP
- Look for this:



912	p.moreover.com	1460	page?c=US%20political%20columnists&o=rss002&wiz=2274705
922	p.moreover.com	2920	page?c=US%20political%20columnists&o=rss002&wiz=2274705
924	p.moreover.com	1439	page?c=US%20political%20columnists&o=rss002&wiz=2274705

- Save as – moreover.htm
- Open in Notepad or Firefox

Copyright 2021 – CSITech Ltd

Volatility – ethscan

Sat, 2 Jul 2005 18:24:00 GMT Arianna Online http://c.moreover.com/click/here.pl?r345457609&w=2274705 Combine that with rank incompetence and you've got quite a potent - and deadly - combo. It was in full display last night during the president's speech on Iraq and last week during Donald Rumsfeld's multiple public appearances. http://c.moreover.com/click /here.pl?r345457609&w=2274705 Sat, 2 Jul 2005 18:24:00 GMT Arianna Online http://c.moreover.com/click /here.pl?r345438899&w=2274705 Iraq: Not Your Father's Anti-War Movement June 15, 2005 [Printer-friendly version] "What Korea was to Truman, and Vietnam was to LBJ, Iraq will be to George W. Bush," Arthur Schlesinger told me last week. http://c.moreover.com/click/here.pl?r345438899&w=2274705 Sat, 2 Jul 2005 17:27:00 GMT

Copyright 2021 – CSITech Ltd

Goto the Malware Practical

- Lots of fun with a Banking Trojan!

Copyright 2021 – CSITech Ltd

Passwords and Decryption

Copyright 2021 – CSITech Ltd

Volatility – True/VeraCrypt

- New capabilities from Volatility 2.4
- If TrueCrypt was in use can provide significant information
- Perhaps an encrypted USB was in the system when the RAM imaged
- …or just RAM imaged, and power pulled
- …or covert image taken

TrueCrypt is probably the best and most widely used encryption software available today. You can create simple encrypted containers or use full disk encryption. You can even hide an invisible encrypted container inside another one. And I mean invisible, its simply impossible to detect it.

Copyright 2021 – CSITech Ltd

Volatility – True/VeraCrypt

- Works best if TrueCrypt was running at the time
- This is where Hiberfil files can be awesome!
- If user closed lid of laptop with TrueCrypt mounted, master keys will be in Hiberfil

TrueCrypt is probably the best and most widely used encryption software available today. You can create simple encrypted containers or use full disk encryption. You can even hide an invisible encrypted container inside another one. And I mean invisible, its simply impossible to detect it.

Copyright 2021 – CSITech Ltd

Volatility – True/VeraCrypt

- Seen evidence of VeraCrypt usage
  - Primarily from pslist
    - Also seen in userassist, devicetree and throughout the analysis
- VeraCrypt is a fork of the TrueCrypt software
- Analysis can be done using volatility
  - Requires external plugin from Kresimir (INsig2)
- Copy vcaudit.py from *Volatility setup\Python Plugins\veracrypt* into *volatility-master\volatility\plugins*

Copyright 2021 – CSITech Ltd

Volatility - VeraCrypt

- VeraCryptsummary
  - This is an outstanding command which provides incredible information
    - (Takes a minute or so)

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 veracryptsummary >
c:\temp\VCsummary.txt
```


After work done by a number of researchers has been pieced together we have 2 tools that can really help us when TrueCrypt is in use. The first command TrueCryptsummary attempts to give you information about the session of TrueCrypt that was running.

Copyright 2021 – CSITech Ltd

Volatility - VeraCrypt

- **Password** **Jaffa123 at offset 0xfffff8068d3f8b44**
- Process VeraCrypt.exe at 0xffffa28763fdb080 pid 5140
- Service VeraCrypt state SERVICE_RUNNING
- Kernel Module VeraCrypt.sys at 0xfffff8068d330000 - 0xfffff8068d3fe000
- Symbolic Link **A: -> \Device\VeraCryptVolumeA** mounted 2019-06-05 10:29:31 UTC+0000
- Symbolic Link Volume{2334d70b-f882-11e8-8be3-b0359fd847ca} -> \Device\VeraCryptVolumeA mounted 2019-06-05 10:29:31 UTC+0000
- Symbolic Link **A: -> \Device\VeraCryptVolumeA** mounted 2019-06-05 10:29:31 UTC+0000
- File Object \Device\VeraCryptVolumeA\ at 0xa28763afd550
- File Object \Device\VeraCryptVolumeA\ at 0xa28763ff48d0
- Driver \Driver\VeraCrypt at 0xa287622fd060 range 0xfffff8068d330000 - 0xfffff8068d3fe000
- Device **VeraCryptVolumeA** at 0xffffa28764152080 type FILE_DEVICE_DISK
- **Container** **Path: \??\C:\Users\Harrison\Documents\SecretStuff.txt**
- Device VeraCrypt at 0xffffa287622fde40 type FILE_DEVICE_UNKNOWN

- Now know that the container called secretstuff.txt was mounted at drive a:
- Think back to vaddump of notepad++
  - Two txt files sat at the root of A:\ (vaddump of notepad++)
  - Although not in this list, these are the files inside of the container.
- Note: Password is only found when the user turns on “remember password” in VC. Usually this is off.

They key things to note are:

The version of VeraCrypt being used
The drive letter it was given when mounted
The file used that is the VeraCrypt container
The password used to encrypt the container

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- What else can be done with True/VeraCrypt
- TrueCryptsummary
  - Unzip TrueCryptdecrypttest.zip image
  - Run the command:

```
vol.py -f c:\Win8.1x64.vmem --profile=Win8SP1x64 truecryptsummary >
c:\temp\tcsummary.txt
```


After work done by a number of researchers has been pieced together we have 2 tools that can really help us when TrueCrypt is in use. The first command TrueCryptsummary attempts to give you information about the session of TrueCrypt that was running.

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- Process TrueCrypt.exe at 0xffffe000432a65c0 pid 3604
- Service TrueCrypt state SERVICE_RUNNING
- Kernel Module TrueCrypt.sys at 0xfffff800d7800000 - 0xfffff800d7841000
- Symbolic Link Volume{539b6e41-48ec-11e6-825f-28f07632de91} -> \Device**TrueCryptVolumeE mounted 2016-07-13 11:26:47 UTC+0000**
- Symbolic Link E: -> \Device\TrueCryptVolumeE mounted 2016-07-13 11:26:47 UTC+0000
- Symbolic Link E: -> \Device\TrueCryptVolumeE mounted 2016-07-13 11:26:47 UTC+0000
- File Object \Device\TrueCryptVolumeE\ at 0x3fc1a20
- File Object \Device\TrueCryptVolumeE\ at 0x3fc1bf0
- File Object \Device\TrueCryptVolumeE\$RECYCLE.BIN\$IWM2SVZ.bmp at 0x20f856c0
- File Object \Device\TrueCryptVolumeE\$RECYCLE.BIN\desktop.ini at 0x54119420
- **File Object** **\Device\TrueCryptVolumeE\passwords.rtf** at 0x6b40f6f0
- Driver \Driver\TrueCrypt at 0x1a317f0 range 0xfffff800d7800000 - 0xfffff800d7841000
- Device TrueCryptVolumeE at 0xffffe00042dea080 type FILE_DEVICE_DISK
- **Container** **Path: \??\C:\Users\MPCCU VM 1\Desktop\tctest.tc**
- Device TrueCrypt at 0xffffe00041c44e40 type FILE_DEVICE_UNKNOWN

They key things to note are:

The version of TrueCrypt being used
The drive letter it was given when mounted
File or files accessed INSIDE the container. This could be invaluable in figuring out what could be inside a container and what the user was accessing.
The file used that is the TrueCrypt container

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- It is also possible to extract the Master key used to decrypt the data on the fly

```
*vol.py -f c:\Win8.1x64.vmem --profile=Win8SP1x64 truecryptmaster
-D c:\temp
```

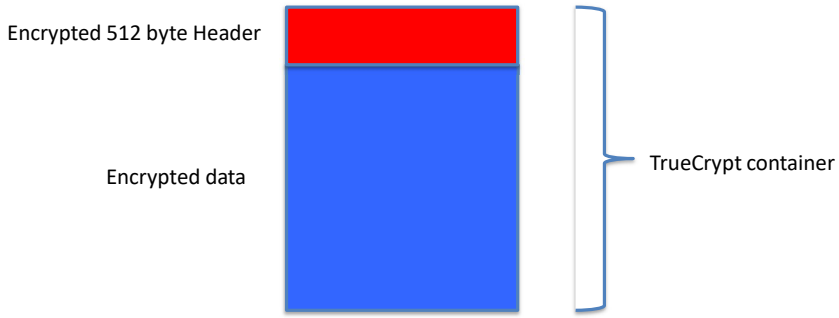
- Using the Master key requires patching the TrueCrypt Source Code which is beyond the scope of this course
- http://mweissbacher.com/blog/2011/05/17/plaidctf-writeup-fun-with-firewire

It is now possible to extract the master key and decrypt the container but this is far from straight forward. The master key is a hash of the passphrase and any key files used. To effect this TrueCrypt needs patching specifically to make use of the master key you recover. The URL provided is an overview of the that process. Its slightly beyond the scope of this course.

This could, in some circumstances be a game changer.

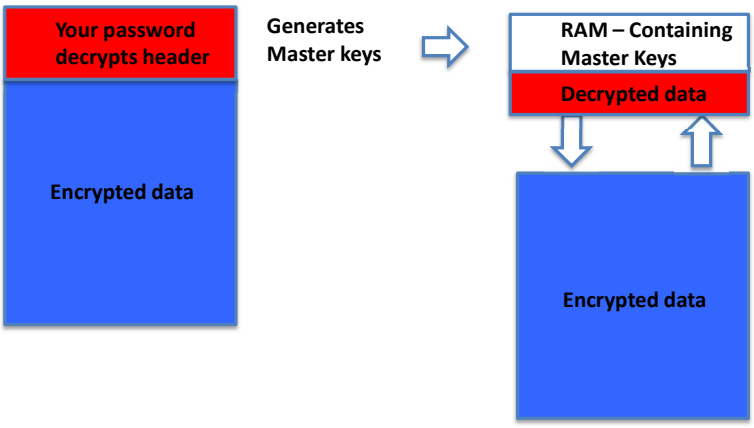
Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt



Copyright 2021 – CSITech Ltd

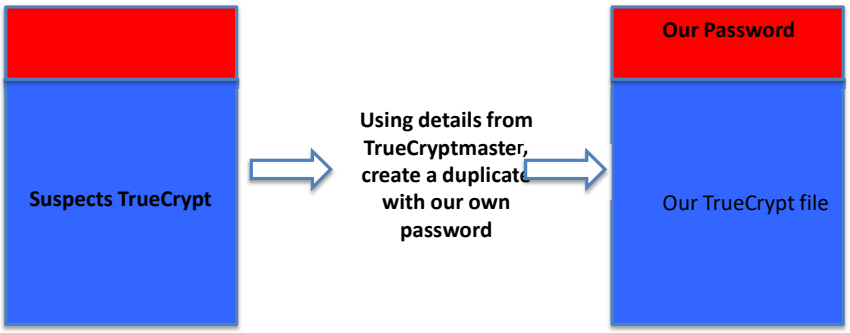
Volatility - TrueCrypt



It is very important that you image a logical drive if you find it on scene because if you image the physical drive it will still be encrypted! It is only decrypted in memory as you request specific files from the container.

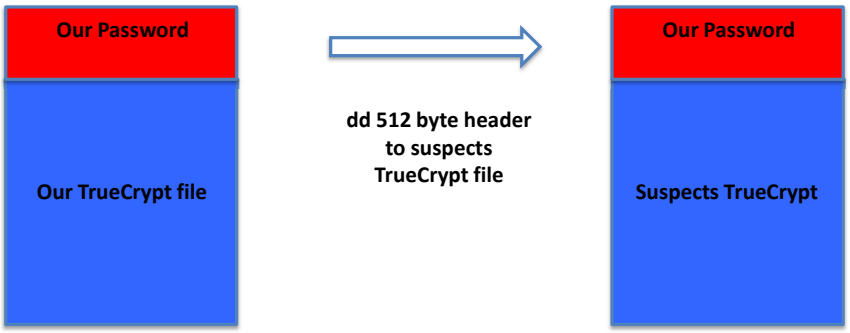
Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt



Copyright 2021 – CSITech Ltd

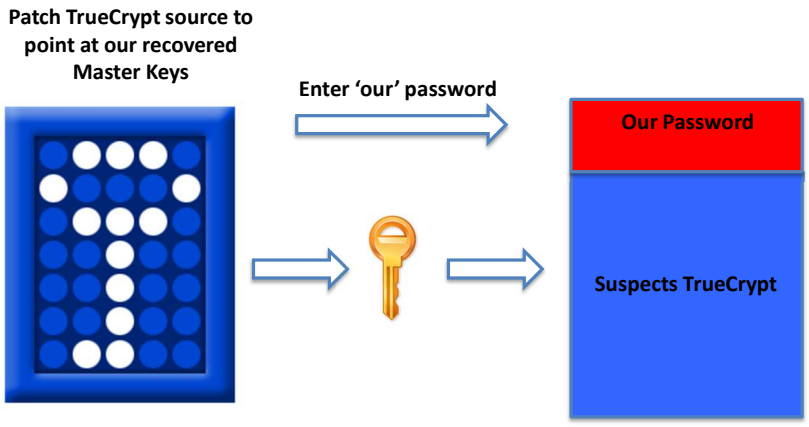
Volatility - TrueCrypt



Dd if=ourfile of=suspectfile bs=512b count=1 conv=notrunc

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt



Patch - http://downloads.volatilityfoundation.org/omfw/2013/OMFW2013_Ligh.pdf

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- This is a very complex process
- Teaching a class in early 2016, discussed issue with Gov researcher called Matt Smith
- Wrote a proof of concept to simplify the process
- It works!!

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- MKDecrypt
- https://github.com/AmNe5iA/MKDecrypt
- Linux based only as exploits the way that Linux kernel decrypts TrueCrypt containers
- Will decrypt any AES TrueCrypt container from any OS
- We just need the Container and Master Key

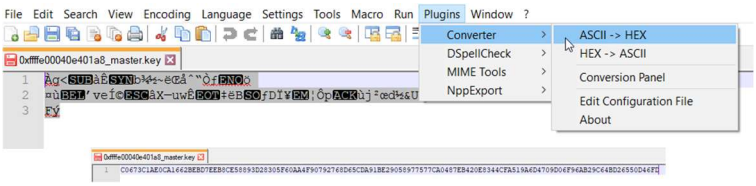
Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- Run TrueCryptmaster and dump the key

```
vol.py -f c:\Win8.1x64.vmem --profile=Win8SP1x64 truecryptmaster
-D c:\temp
```

- Open Key file in Notepad++
  - Select all, then click Plugins–Converter–Ascii to Hex
  - Concatenate the key into one long line



Alternatively you can output the file to a text file:
vol.py TrueCryptmaster –f c:\Win8.1x64.vmem --profile=Win8SP1x64 > c:\temp\master.key

Then do the following

Use Alt+select to select vertically
Use notepad++ to push it into a single string – select all text, go Edit – Line operations – Join lines
Find and replace spaces with nothing

Copyright 2021 – CSITech Ltd

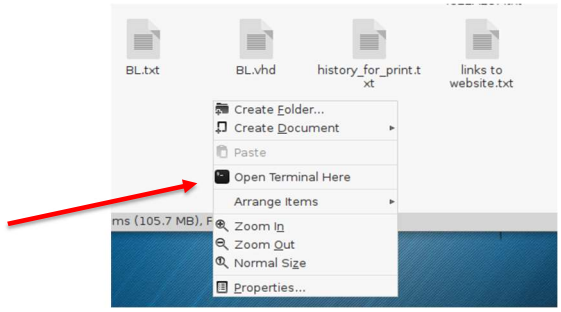
Volatility - TrueCrypt

- Install VMPlayer (unless you have VMware software installed already) in VM folder
- Unzip Kali-Linux-Light-2016.2-vm-amd64
- Open the Kali-Linux-Light-2016.2-vm-amd64.**vmx**
- Log in using username: **root** password: **toor**
- Copy concatenated AES key to MKDecrypt-master folder on linux desktop - should already be in MKDecrypt-master folder
- tctest.tc (TC container) - should already be in MKDecrypt-master folder

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

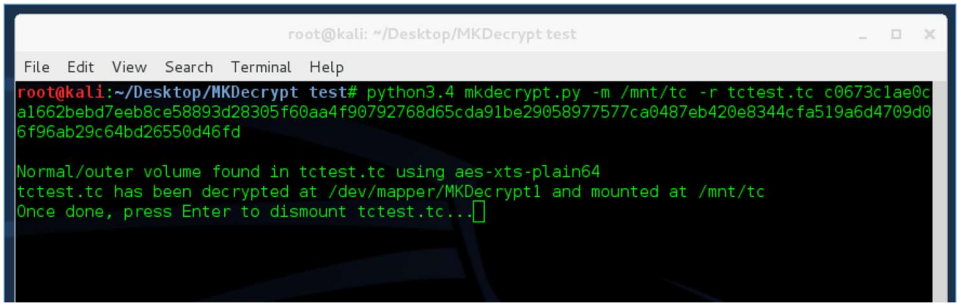
- We need to open terminal in MKDecrypt-master folder that is already on desktop
- Double click folder
- Right click (empty space) in side and select “Open Terminal here”



Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- Uses Python 3.5, you have to invoke the right version
  - `mkdir /mnt/tc`
  - `python3 mkdecrypt.py –m /mnt/tc –r tctest.tc <master key>`

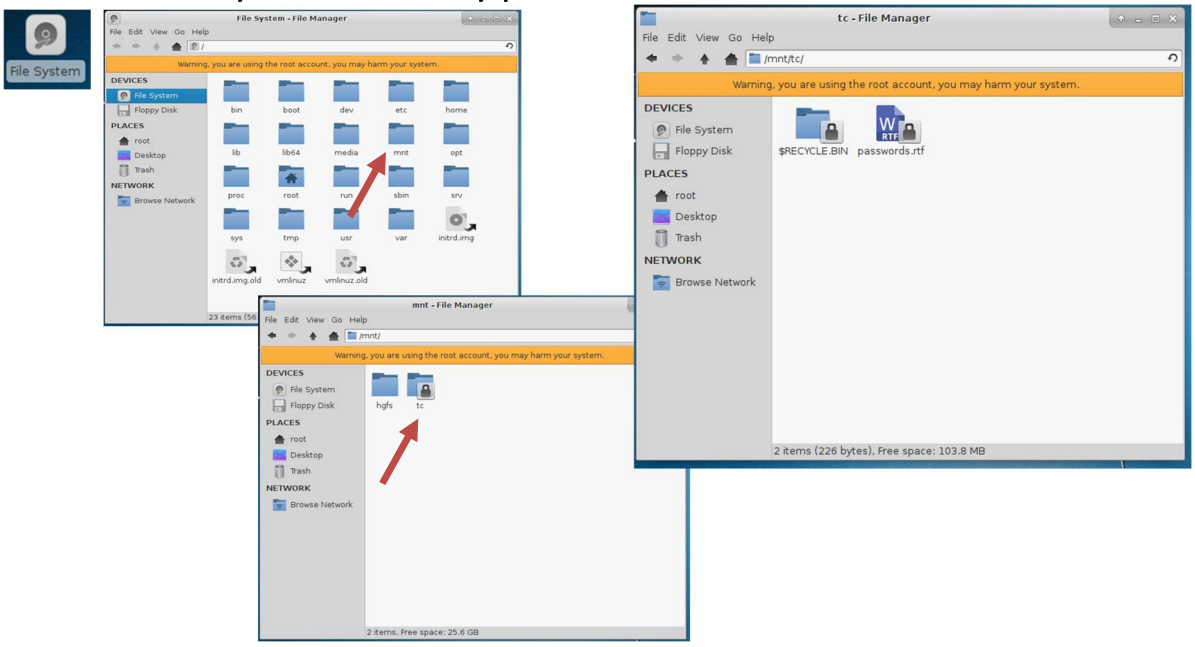


./MKDecrypt – Python Script used for mounting TrueCrypt
-m /mnt/tc (Mountpoint) – need to create mount point
-r (read only mode)
Master key
NOTE: this is the string of Hex characters. NOT the file

To change the language in Kali, open terminal and type setxkbmap XXX where XXX is your language. Eg. “se” for Swedish, fi for Finnish or en_US for English (Case Sensitve)

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt



Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- Remember dumpfiles command?
- Remember TrueCryptsummary output?

- Can we pull the passwords.rtf file out of the TrueCrypt process space?

- Hint:
  -Q to specify memory offset
  -p to specify PID
  -n to include file name

-n include filename with extension during output

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

- Using **dumpfiles** we can extract the rtf file at offset 0x6b40f6f0
- We need to use the –Q switch to specify the offset

```
vol.py -f c:\Win8.1x64.vmem --profile=Win8SP1x64 dumpfiles –n
-p 3604 -Q 0x6b40f6f0 –D c:\temp\dump1
```

- Open in WordPad

-n include filename with extension during output

Copyright 2021 – CSITech Ltd

Volatility - VeraCrypt

- Procedure is the same as with TrueCrypt
- Just need VeraCrypt plugins:
  - key\Volatility setup\Python plugins\VeraCrypt
- Copy two files to:
  - “volatility-master\volatility\plugins” folder

Copyright 2021 – CSITech Ltd

Volatility - TrueCrypt

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Decrypting Bitlocker encrypted drives can be achieved if the RAM is acquired
- Relatively easy to do
- Linux required
- Multiple Win versions supported
  - May change with Win 11 updates ☹

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Copy the plugin from “Volatility setup\PythonPlugins\BitlockerPlugin\bitlocker.py” into your plugins folder “volatility-master\volatility\plugins”
  - https://github.com/elceef/bitlocker
- Tutorial
  - https://tribalchicken.com.au/technical/recovering-bitlocker-keys-on-windows-8-1-and-10/
  - https://github.com/elceef/bitlocker

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Need:
  - Raw disk image (Note RAW, not a compressed image or series of images)
    - Our example – BL.vhd (KEY\Bitlocker\BL.vhd)
  - RAM dump
    - Our example – Bitlocker.mem
  - Linux
    - Installed sleuthkit
    - Installed bdemount
  - Bitlocker plugin for Volatility

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Run BitLocker Volatility plugin against the RAM dump
  - `vol.py -f **c:\Bitlocker.mem --profile=Win7SP1x64** bitlocker > c:\temp\bitlocker.txt`
- Should get the output:

```
Address : 0xfa80018c6c10
Cipher  : AES-128
FVEK    : 2a7317c48414016d431868a0e89c8e92
TWEAK   : ae5206e75d3e301691f5c43537829992
```

- FVEK and TWEAK are keys we'll use to decrypt the drive

BitLocker encrypts data with either 128-bit or 256-bit AES and optionally using a diffuser algorithm called Elephant. The key used to do the encryption, the Full Volume Encryption Key (FVEK) and/or TWEAK key, is stored in the BitLocker metadata on the protected volume. The FVEK and/or TWEAK keys are encrypted using another key, namely the Volume Master Key (VMK). Several copies of the VMK are also stored in the metadata. Each copy of the VMK is encrypted using another key, also know as key-protector key

Copyright 2021 – CSITech Ltd

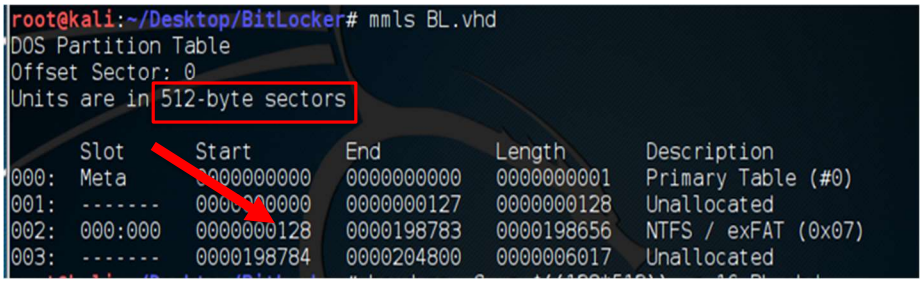
Volatility - Bitlocker

- Next, start Kali linux
  - We need to figure out the partition layout
  - We'll use mmls command from sleuthkit
  - Start terminal in "Bitlocker" folder on desktop
  - Run mmls against our image:
    - `mmls BL.vhd`

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Note the start of the NTFS partition
- We'll need to verify if that partition is Bitlocker encrypted

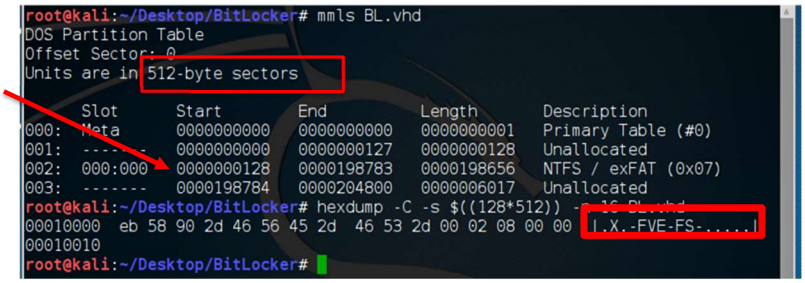


Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Remember the filesystem headers for encryption?
  - Slide 25
- Run:

```
hexdump -C -s $((128*512)) -n 16 BL.vhd
```



We will use hexdump command to read out the start of the potential Bitlocked partition. This is easy since we have only one partition but if multiple are present we have to know specifically which one is encrypted.
-C is HEX + ASCII dispy
-s is skip offset bytes from the beginning of the input
-n is interpret only n bytes of input
-$(()) – start offset * block size
More info on: https://www.freebsd.org/cgi/man.cgi?query=hexdump&sektion=1

- * is shift + 8
- $ is shift + 4

Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- In Kali linux run commands:
  - Create mount point for BL container
    - `mkdir /mnt/bt1`
  - Mount BitLocker drive with: FVEK and TWEAK keys found in RAM dump, by specifying the right partition we found earlier and mount point created in previous step

```
bdemount -k <FVEK key>:<TWEAK key> -o $((128*512)) BL.vhd /mnt/bt1
```


- * is shift + 8
- $ is shift + 4

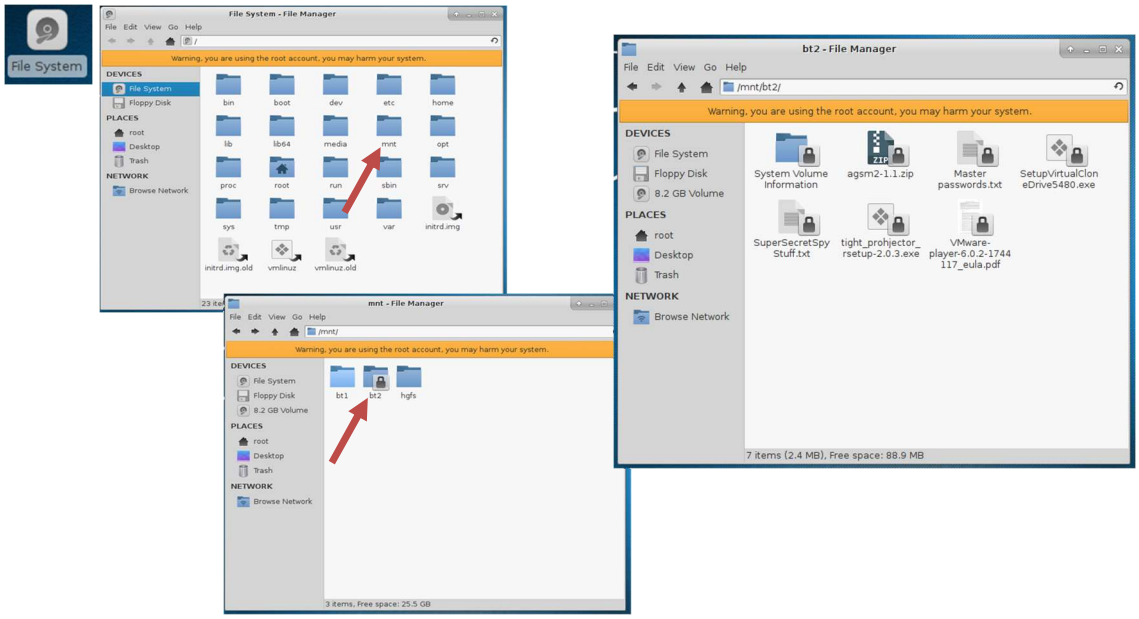
Copyright 2021 – CSITech Ltd

Volatility - Bitlocker

- Create second mount point for mounting the decrypted partition
  - `mkdir /mnt/bt2`
- Mount the decrypted partition in read only mode
  - `mount -o loop,ro /mnt/bt1/bde1 /mnt/bt2`

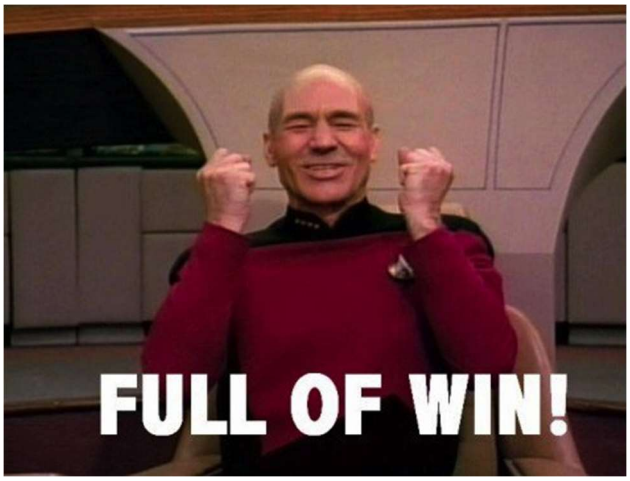
Copyright 2021 – CSITech Ltd

Volatility - Bitlocker



Copyright 2021 – CSITech Ltd

Volatility - Bitlocker



Copyright 2021 – CSITech Ltd

Volatility - hashdump

- Extracts hashed passwords from the SAM hive
- Unzip Windows7_Final.vmem to the root of C:\ and run

```
vol.py hashdump -f c:\Windows7_final.vmem --profile=Win7SP1x86 >
c:\temp\hashes.txt
```


This command will enable you to dump the user names and password hashes – awesome!

Copyright 2021 – CSITech Ltd

Volatility - hashdump

```
Volatility Foundation Volatility Framework 2.6.1
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
nickfx:1000:aad3b435b51404eeaad3b435b51404ee:3dbde697d71690a769204beb12283678:::
```

How cool is that!

Aad3b and 31d6c – null password but it's a short password so we need to look at the second part

Copyright 2021 – CSITech Ltd

Volatility - hashdump

- Then use:
  - Cain and Able
  - SAMinside
  - Ophcrack
- Crackstation.net
  - Copy and paste values
  - Enter the Captcha
- https://hashkiller.io/
- hashmob.net

There are lots of ways to crack the hashes from a Windows system. The most effective is to use Rainbow Tables, pre-computed tables that enable a very fast crack. As a commercial tool I favour Ophcrack. http://ophcrack.sourceforge.net/. There is a free version but the paid-for tables are excellent and will break almost any Windows password.

Copyright 2021 – CSITech Ltd

Volatility 3 and encryption

- View help file of Vol3, note the following commands:
  - Windows.hashdump
    - Dumps user hashes from memory
  - Windows.lsadump
    - Dumps lsa secrets from memory
  - Windows.cachedump
    - Dumps lsa secrets from memory

Exploiting the Registry

Copyright 2021 – CSITech Ltd

Volatility - Registry

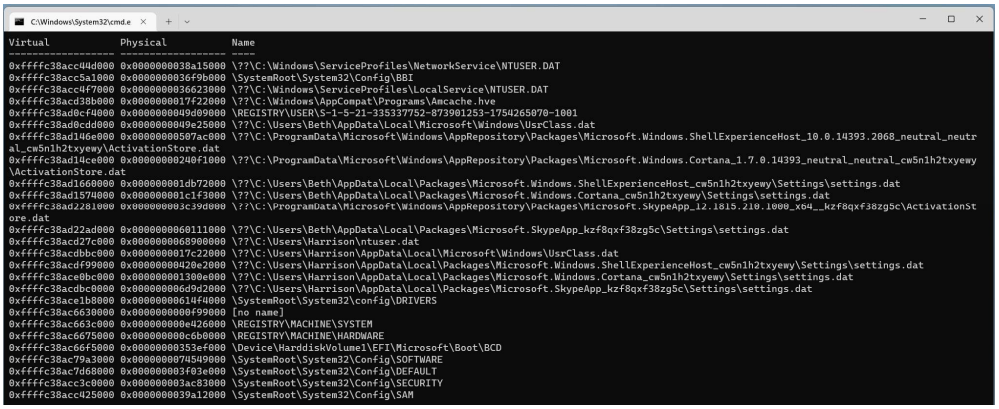
- All registry hives are loaded into RAM at boot time
- Some fantastic tools for registry parsing
- Huge amounts of information can be extracted

The Windows registry is the engine room of the Win OS. There are a significant number of registry keys can that be useful to an investigator. Incredibly the registry hives on disk are really a back up to what happens in RAM. At boot time the entire registry is loaded into RAM and can be extracted if we know how to reach in a get it.

Copyright 2021 – CSITech Ltd

Volatility - hivelist

- To parse out the top hives
  - `vol.py -f <path2image> --profile=<profile> hivelist`



This is always the place to start. Hivelist will give us the addresses of the top level hives. It is the VIRTUAL addresses you are interested in. Later on sometimes we will need to specify this virtual address to dump a specific registry hive or just to reference it.

Copyright 2021 – CSITech Ltd

Volatility - printkey

- We have parsed out the ‘top’ directories of the registry hives
- To explore a Subkey use –K to specify a specific key path
- Start with the SAM key

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 printkey –K SAM
```


Copyright 2021 – CSITech Ltd

Volatility - printkey

- SAM has a subkey called ‘Domains’
- This Subkey can then be explored
  - `-K SAM\Domains`
- Continue exploring until you find the named users

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 printkey
-K SAM\Domains
```


Copyright 2021 – CSITech Ltd

Volatility - Registry advanced

- Some other things that can be found in this Image:

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 printkey
-K "SOFTWARE\Microsoft\Internet Explorer\TypedURLs"
```

  - *There is a space in the path, it MUST go into double quotations!

Use the same process. Lots of good stuff here.

Copyright 2021 – CSITech Ltd

Volatility - Registry advanced

- What about the OEM?
  - Original Equipment Manufacturer?

```
vol.py -f c:\Win10.img --profile=Win10x64_14393 printkey
-K Microsoft\Windows\CurrentVersion\Store\Configuration
```

  - *There is no space in Configuration

Use the same process. Lots of good stuff here.

Copyright 2021 – CSITech Ltd

Volatility – dumpregistry

- Can dump all registry files and open in Registry Viewer

```
vol.py –f c:\Win10.img --profile=Win10x64_14393 dumpregistry
-D c:\temp
```

- Registry viewer on your key (demo mode)
- Quick wins: Regripper or Registry report (on your key in root folder)
- Excellent registry viewer by Eric Zimmerman
  - Registry Explorer/RECmd

If you want to dump only a specific reg key, or a key that belongs to a specific user, we have to do hivelist first, get the offset and in our dumpregistry command specify the virtual offset with –o flag.

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Easy and convenient way of accessing physical memory as files a virtual file system - point and click memory forensics
- Running multiple volatility commands over a single RAM image in a second
- Features:
  - Analyze Windows memory dump files
  - **Live memory** via DumpIt or WinPMEM
  - **Live memory in read-write mode** via linked PCILeech and PCILeech-FPGA devices!
  - Python plugins
  - Connect to a remote LeechAgent memory acquisition agent over a secured connection - allowing for remote live memory incident response - even over higher latency low band-width connections!

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Created by Ulf Frisk - Author of PCILeech
- Check out his talk from Disobey 2020 - Live Memory Attacks and Forensics
- https://github.com/ufrisk/MemProcFS
- Needs:
  - **Internet!** MS symbols as Vol3 for interpretation
  - Actual project https://github.com/ufrisk/MemProcFS/releases/latest
  - Mounting the file system requires the **Dokany file system library** https://github.com/dokan-dev/dokany/releases
    - Install from “General tools\MemProcFS” folder
  - Python 3.6 or later
    - MAKE SURE that during the installation you select **ADD TO PATH** as it will make thing much easier later on…
    - We already installed it while doing Volatility 3

Install latest 64bit version of Python 3
MAKE SURE that during the installation you select ADD TO PATH as it will make thing much easier later on….

Copyright 2021 – CSITech Ltd

The Memory Process File System

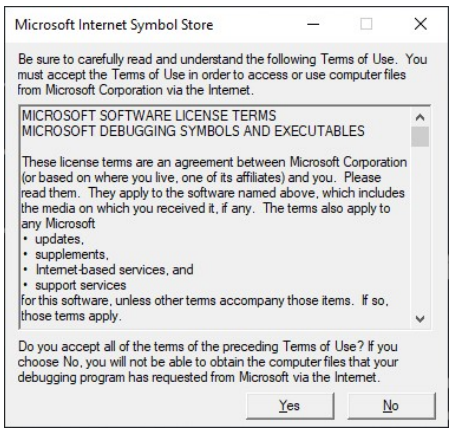
- Basic usage – unzip, start CMD in folder:
  - mount the memory dump file as default M:
    - `memprocfs.exe -device <path2ram>`
  - mount the memory dump file as default M: with extra verbosity:
    - `memprocfs.exe -device c:\temp\win10x64-dump.raw -v`
  - mount the memory dump file as default M: with extra extra verbosity:
    - `memprocfs.exe -device c:\temp\win10x64-dump.raw -v -vv`
  - mount the memory dump file as S:
    - `memprocfs.exe -mount s -device c:\temp\win10x64-dump.raw`

*It mounts to M because M is for Magic 😎

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Mount the memory dump file as default M:
  - `memprocfs.exe -device c:\Win10.img`
- You should get a pop up as below – NEED to select Yes
  - This is need for correct interpretation of the data in the memory, similar to Volatility 3



Please not that the FIRST time you start the memprocFS it will give you a pop-up for using Microsoft Symbols that need to be downloaded from the Internet!
If you don't accept, thigs will not work properly later on.

Copyright 2021 – CSITech Ltd

The Memory Process File System

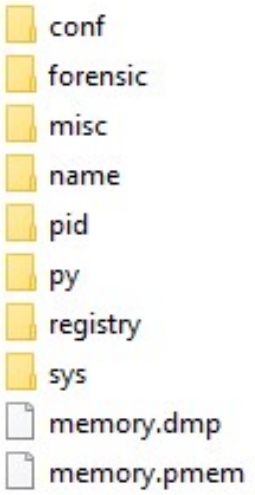
- Mount the memory dump file as default M:
  - `memprocfs.exe -device c:\Win10.img`
- If all is good, you should see it mounted in Windows explorer

```
C:\Users\test\Downloads\MemProcFS_files_and_binaries_v3.1-20200201>MemProcFS.exe -device c:\Win8.1x64.vmem
Initialized 64-bit Windows 6.3.9600
MOUNTING THE MEMORY PROCESS FILE SYSTEM
================================================================================
The Memory Process File System is mounted as: M:\
Loaded VmmDll Version: 3.1.0
Memory from dump files or PCILeech supported devices are analyzed to provide
a convenient process file system for analysis purposes.
 - File system is read-only when dump files are used.
 - File system is read-write when FPGA hardware acquisition devices are used.
 - Full support exists for Windows XP to Windows 10 (x86 and x64).
 - Limited support for other x64 operating systems.
 - Memory Process File System: https://github.com/ufrisk/MemProcFS
 - File system by: Ulf Frisk - pcileech@frizk.net - https://frizk.net
================================================================================
```


Copyright 2021 – CSITech Ltd

The Memory Process File System

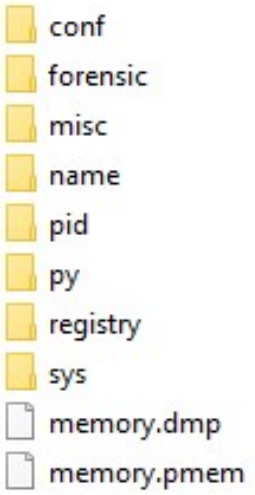
- The structure:
  - Root
    - Multiple directories and files which contains the physical memory of the target
    - *.pmem - contains the raw physical memory being analyzed
    - *.mem – chrash dump memory – WinDbg compatible
  - Conf
    - Global Memory Process File System status and configuration settings
  - Forensics
    - Collection of more thorough batch-oriented analysis
  - Misc
    - Various miscellaneous analysis tasks related to the global context
  - Name
    - Processes by name with their files – exe, files, dll…



Specific details and descriptions of each folder can be found here:
https://github.com/ufrisk/MemProcFS/wiki

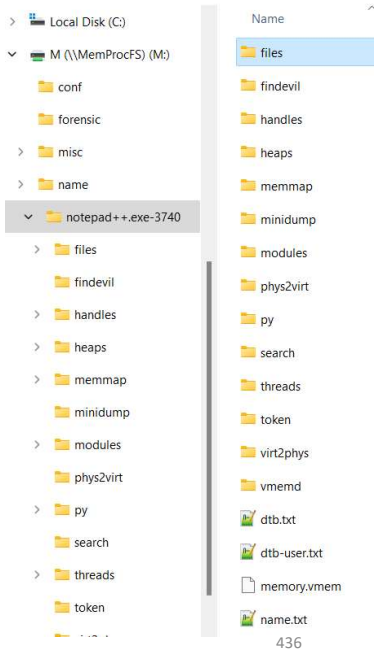
The Memory Process File System

- The structure:
  - Pid
    - Process by PID, similar to Name
  - Py
    - Placeholder for additional plugins
  - Registry
    - Registry files – by hive or by browsable folders
  - Sys
    - Directories and files displaying various system information
    - OS version, build no
    - Process tree + verbose output
    - Netstat information



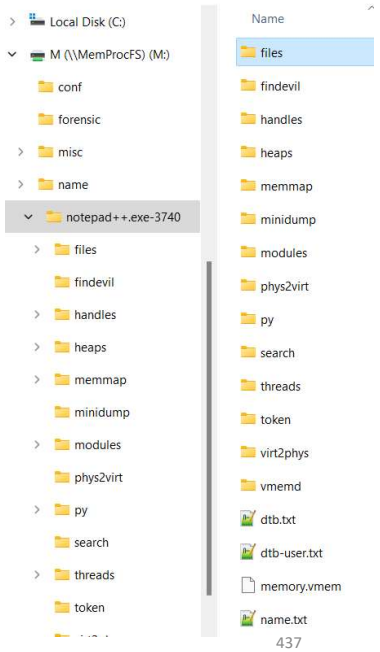
The Memory Process File System

- The structure - Name and PID
  - All processes that are identified are listed in an individual directory shown by name in the /name/ directory and by PID in the /pid/ directory
  - **Files** - contains three sub-directories named handles, modules and vads
    - Files reconstructed from specific process
  - **Findevil -** locates signs of malware by analyzing select indicators of evil
  - **Handles** - open process handles, such as files, mutexes, registry keys and more
  - **Heaps** - sub-directory for each heap as well as info files about the heaps



The Memory Process File System

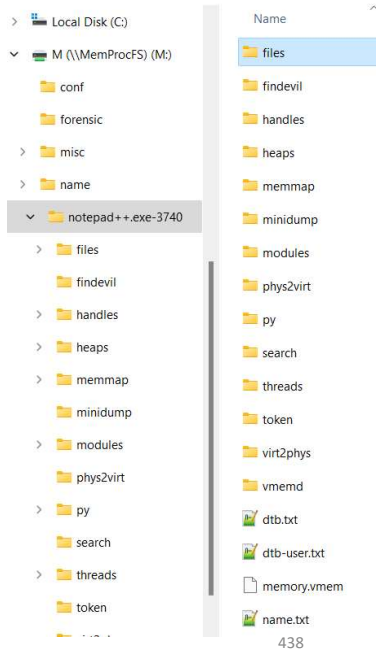
- The structure - Name and PID
  - **Memmap** – two mem maps – page table and vad generated. Also contains a sub-directory for verbose information about the VAD map
  - **Minidump -** WinDbg compatible full process minidump.dmp, for debugging
  - **Modules** - contains one sub-directory for each loaded module (.DLLs and EXEs), for kernel process will also have loaded drivers
  - **Phys2virt** – *for **writing*** into physical address
  - **Py** - contains per-process plugins coded in Python



For more details about the Modules, please see https://github.com/ufrisk/MemProcFS/wiki/FS_Process_Modules

The Memory Process File System

- The structure - Name and PID
  - **Search-** flexible efficient searching in process virtual memory and/or physical memory
  - **Threads -** contains one sub-directory for each thread named after its Thread ID
  - **Token** – Win user related info form Token process – SID, name, session…
  - **Virt2phy -** ***write*** into a virtual address
  - **Vmemd** - files of virtual memory according to the process memory map. Primarily entries from the virtual address descriptor (VAD) memory map are used, but also hardware page table map (PTE)



The Memory Process File System

- New **forensic mode** available with version 3.3 and up
- Collection of more thorough batch-oriented analysis tasks that may be undertaken on memory dumps
  - **NTFS**
    - Directory contains a best-effort reconstructed file system reconstructed from NTFS MFT entries located in physical memory
    - Automatically reconstructs the embedded MFT resident files
    - **All files will have correct file properties in sense of MAC times**
  - **Timeline**
    - Contains various system artifacts ordered by time stamp in timelines
  - **CSV**
    - Various results in csv format – processes, tasks, services,
  - **Findevil**
    - tries to identify and discover signs of malware infection
  - **Json**
    - Files for exporting data to elastic search and kibana

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Can be started with command line
  - Add `-forensics <number>`
  - Editing the `forensic_enable.txt` file
- The files in the *forensic* directory are listed in the table below:

File	Access	Description
database.txt	Read	Path and name of sqlite database (if exists).
forensic_enable.txt	Read/Write	Forensic mode number (see table below)
readme.txt	Read	Readme file.

- If set, will create permanent or temporary SQLITE database

Copyright 2021 – CSITech Ltd

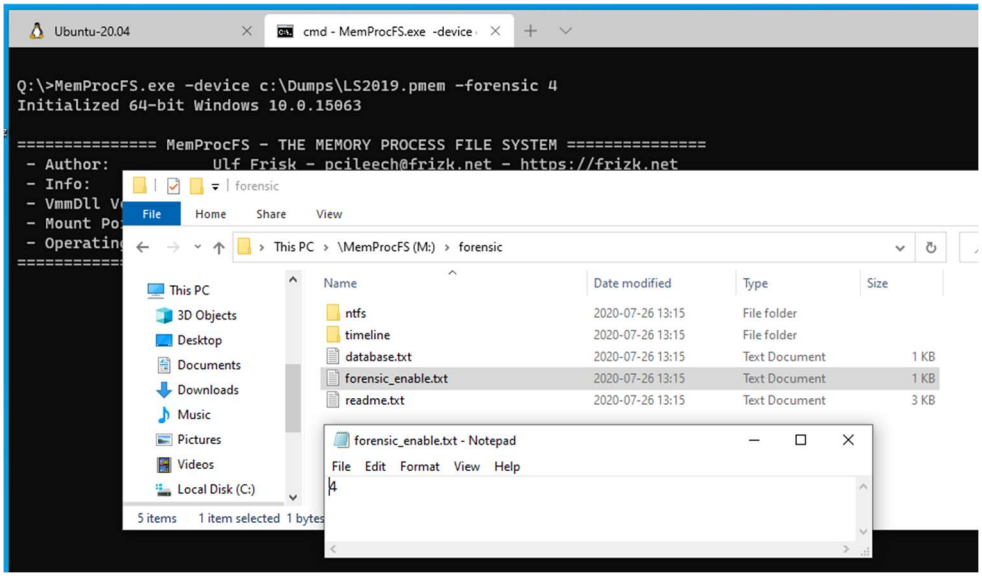
The Memory Process File System

- The values of the `forensic_enable.txt` file and `-forensic` start up parameter are listed in the table below:

Value	Description
0	Forensic mode not started or failed to start.
1	Forensic mode with in-memory only SQLITE database.
2	Forensic mode with temporary SQLITE database deleted upon MemProcFS exit.
3	Forensic mode with temporary SQLITE database kept upon MemProcFS exit.
4	Forensic mode with well known SQLITE database kept upon MemProcFS exit.

Copyright 2021 – CSITech Ltd

The Memory Process File System



USE NP++

Copyright 2021 – CSITech Ltd

The Memory Process File System

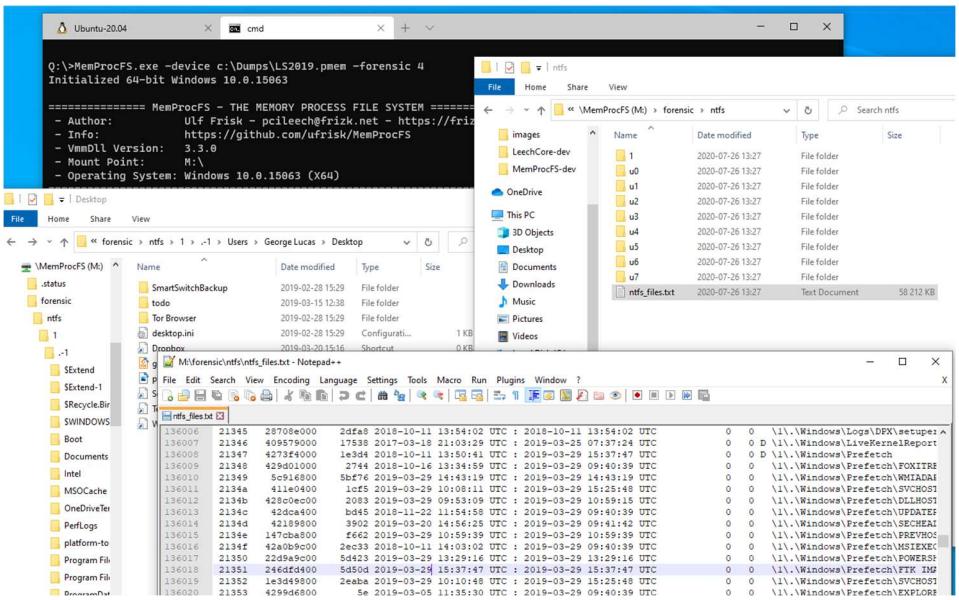
- NTFS
  - Best-effort reconstructed file system reconstructed from NTFS MFT entries
  - Resident MFT files will be reconstructed automatically
  - `ntfs_files.txt` contains a complete record of the recovered NTFS MFT entries and their path
  - **NOTE** - reconstruction is a best-effort reconstruction and may contain errors as well as missing files and folders
  - File output file has the following format:

```
     #  PHYS_ADDR RECID TIME_CREATE                    TIME_MODIFY                 SIZE AVAIL DIR PATH
====== ========= ===== ============================================= ==== ===== === ============
 21348 4273f4000 1e3d4 2018-10-11 13:50:41 UTC : 2019-03-29 15:37:47 UTC    0     0 D   \1\.\Windows\Prefetch
 21349  5c916800 5bf76 2019-03-29 14:43:19 UTC : 2019-03-29 14:43:19 UTC    0     0     \1\.\Windows\Prefetch\WMIADAP.E
 2134a  411e0400  1cf5 2019-03-29 10:08:11 UTC : 2019-03-29 15:25:48 UTC    0     0     \1\.\Windows\Prefetch\SVCHOST.E
```


Copyright 2021 – CSITech Ltd

The Memory Process File System

- NTFS



Copyright 2021 – CSITech Ltd

The Memory Process File System

- Timeline
  - Contains various system artifacts ordered by time stamp in timelines
  - Files in the **forensic/timeline** directory are listed in the table below:

File	Description
timeline/timeline_all.txt	All timelining information bundled into a single file.
timeline/timeline_net.txt	Network connection timeline.
timeline/timeline_ntfs.txt	NTFS MFT timeline.
timeline/timeline_process.txt	Process timeline.
timeline/timeline_registry.txt	Registry timeline.
timeline/timeline_thread.txt	Thread timeline.
timeline/timeline_[...].txt	Additional timelines generated by plugins.

Copyright 2021 – CSITech Ltd

The Memory Process File System

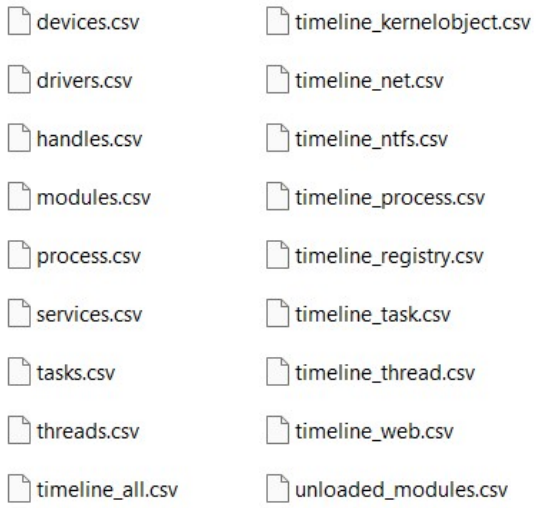
- Timeline
  - Files in the **forensic/timeline** directory are read-only!
  - The timeline files have the following format:

```
TIMESTAMP               TYPE   ACTION   PID  ADDRESS          TEXT
======================  ====   ======   ==== ================ ================================
2019-03-29 15:42:08 UTC  PROC   DEL      4376 ffffcf00cbad7080 backgroundTask [*] backgroundTask
2019-03-29 15:40:38 UTC  PROC   CRE      5968 ffffcf00cec0e080 svchost.exe [*LOCAL SERVICE] \Device\HarddiskVolume2
2019-03-29 15:40:38 UTC  PROC   CRE      4376 ffffcf00cbad7080 backgroundTask [*] backgroundTask
```


Copyright 2021 – CSITech Ltd

The Memory Process File System

- CSV
  - Contains a comma separated (csv) files that may be used to import into Excel or Timeline Explorer
  - Timestamps are in UTC



Copyright 2021 – CSITech Ltd

The Memory Process File System

- We can also utilize automatic plugin execution
  - https://github.com/ufrisk/MemProcFS-Plugins
- Currently developed
  - Pypykatz
    - Exposes mimikatz functionality in the folder /py/secrets/
    - Retrieval of hashes, passwords, kerberos tickets and various other credentials
  - pypykatz regsecrets
    - Retrieval NTLM hashes for local accounts amongst other things

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Executing the plugins
  - Have installed Python 3.6 or up
  - Install *pypykatz* and *aiowinreg* for Python 3.X
    - `py -3 -m pip install pypykatz`
  - Copy the plugins to the MemProcFS folder
    - Copy the *plugins* folder from *MemProcFS-plugins-master\files* folder to the *"MemProcFS_files_and_binaries"* folder

The pypykatz can also be installed by unzipping the "General tools\MemProcFS\pypykatz-master.zip", opening CMD and running the command *py -3 setup.py install*

Copyright 2021 – CSITech Ltd

The Memory Process File System

- If you run into issues…

  Specify the python install directory while running MemProcFS

  ```
  memprocfs.exe -device <path2ram> -pythonpath
  C:\Users\<username>\AppData\Local\Programs\Python\
  Python<version no.>
  ```

  - Disable Windows defender!
  - Once mounted, simply browser to the "*py\secrets folder*" of mounted memory file

The pypykatz can also be installed by unzipping the "General tools\MemProcFS\pypykatz-master.zip", opening CMD and running the command *py -3 setup.py install*

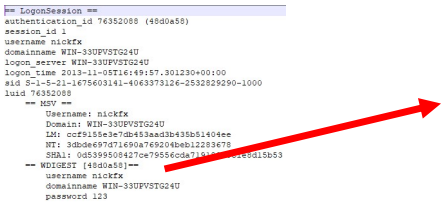
Copyright 2021 – CSITech Ltd

The Memory Process File System

- Executing the plugins
  - Mount the Windows7_final.vmem RAM dump
    - From MemProcFS folder in CMD run

```
MemProcFS.exe -device c:\Temp\Windows7_final.vmem
```

    - Browse to:"M:\py\secrets\by_domain\WIN-33UPVSTG24U\nickfx_76352088.txt"




```
username nickfx
domainname WIN-33UPVSTG24U
password 123
```


If you have any errors or hands, double check that you are have disabled Windows defender

Copyright 2021 – CSITech Ltd

The Memory Process File System

- Practical – Win10 image
  - How many files are on Harrisons Desktop? (Forensics)
  - What Time zone is the system set to? (Sysinfo)
  - In the Bitcoin-qt process – files, how many wallets are referenced? (Name)
  - Which User started the CMD shell?
  - What time was the CMD shell started?
  - In Sysinfo – Net, how many IP connections were open for Tor?
  - What's the Modified Date for Veracrypt (Forensics)?
  - In Timeline Processes what time did Electrum start?
  - Find the eventlogs files – svchost is the responsible program

Copyright 2021 – CSITech Ltd

Can we use this to create volatility2 profile?

- **YES!!!!**
- Go to the *"MemProcFS_files_and_binaries_vXXX-win_x64-XXXXX\Symbols\ntkrnlmp.pdb" after you mounted a RAM image*
- You will find there under a strange hash-ed named folder the ntkrnlmp.pdb
- Follow the same steps we used during Vol2 for creating the profile
- Just start with step:

```
pdb_tpi_vtypes.py ntkrnlmp.pdb > dump.txt
```

Linux Analysis

Copyright 2021 – CSITech Ltd

Linux

- Linux is increasingly popular
- Hobbyist to serious server applications
- 65% of worlds internet servers are Linux/Unix

It is easy to ignore Linux as the vast majority of investigations that come through out doors are Windows based. It is worth remembering that the majority of the worlds internet servers and most serious hackers, are based around Linux. If you are not au fait with Linux then I strongly suggest getting comfortable with it as an OS. Even just installing it as a VM and learning how to install programs, use the command shell and get online would be getter than nothing. Next, learning the standard processes that run the OS and what they do will help in an investigation.

Copyright 2021 – CSITech Ltd

Linux

- Imaging a Linux machine is currently difficult
- Used to simply dd – kmem and mem.
- Modern kernels do not allow it

- LIME - http://code.google.com/p/lime-forensics/
- AVML - https://github.com/microsoft/avml
- Second Look - https://secondlookforensics.com
- F-Response remote imager

Copyright 2021 – CSITech Ltd

Modern Acquisition

- **fmem**
  - Loads a kernel driver that creates a character device /dev/fmem
  - Advantages over /dev/mem:
    - It checks if physical pages reside in main memory before accessing them
    - It can access physical pages above 896MB
- **/proc/kcore**
  - Exports the kernel's virtual address space to userland in the form of a core dump (ELF) file
    - 32-bit systems – limited to the first 896MB of memory
    - 64-bit systems – the usability changes as the kernel keeps a static virtual map of all RAM
- **Linux Memory Extractor (LiME)**

Fmem operates by loading a kernel driver that creates a character device /dev/fmem. It exports physical memory for other programs to access. Advantage over /dev/mem is that it checks if physical pages reside in main memory before accessing them. This prevents investigators from accidentally reading device memory or unmapped physical addresses, which causes stability issues. Another advantage is that it can access physical pages above the 896MB boundary, unlike /dev/mem.

The /proc/kcore file exports the kernel's virtual address space to userland in the form of a core dump (ELF) file.
On 32-bit systems, this limits acquisition to the first 896MB of memory. On 64-bit systems, the usability of /proc/kcore changes as the kernel keeps a static virtual map of all RAM.
Unfortuntely, the steps to acquire memory from /proc/kcore are tedious, and often recquire a specialized tool.

Linux Memory Extractor, otherwise known as LiME, is the latest Linux memory acquisition tool. It fixes the issues and challenges involved with the previously discussed tools and techniques.

LiME (Linux Memory Extractor)

- The latest Linux memory acquisition tool
- Operates by loading a kernel driver
- Automatically determines the address ranges that contain main memory
- Multiple acquisition formats (recommended: lime)
- Features:
  - Full Android memory acquisition
  - Acquisition over network interface
  - Minimal process footprint
  - Hash of dumped memory

Linux Memory Extractor, otherwise known as LiME, is the latest Linux memory acquisition tool. It fixes the issues and challenges involved with the previously discussed tools and techniques.
LiME operates by loading a kernel driver, but instead of creating a character device that userland can access, all acquisition is done within the kernel. This significantly enhances the accuracy of the resulting sample because there are no context switches between userland, and the kernel required to transfer data.
When acquiring memory with LiME, you can choose between multiple acquisition formats. When the recommended **lime** format is used, a structured file is produced that removes the need to create a zero-padded file. The structured format contains metadata that describes from which physical offset each section came, which allows the memory analysis tool to dynamically reconstruct the original memory layout.

LiME

- Link: https://github.com/504ensicslabs/lime
  - Install with the command: `sudo git clone https://github.com/504ensicslabs/lime`
- Dump memory to either the local disk or over the network in the following formats:
  - raw
  - padded
  - lime
- Warning: unless necessary, never compile LiME on a system that you plan to investigate
  - Never save evidence to the local disk

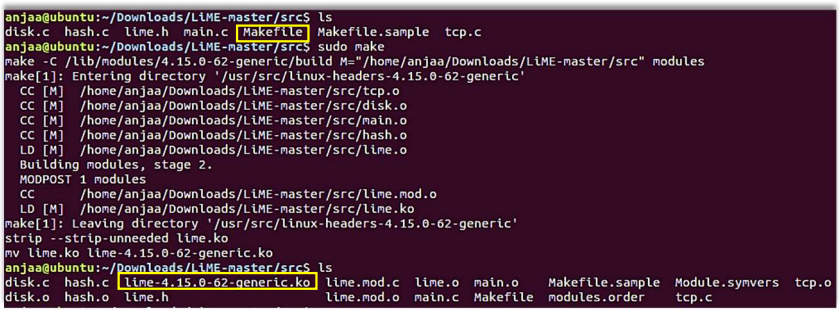
To use LiME, you must first compile the kernel module for the kernel version you want to analyze.

After you have compiled the module, you will have a file named lime-<kernel version>.ko that you can load on the target system.

Unless absolutely necessary, you should NEVER compile LiME (or any software, for that matter) on a system that you plan to investigate. Compilation requires the installation of source code and other supporting files, not to mention that it creates a number of temporary files during the build process that might overwrite evidence in slack space.

LiME (Forensic Memory Acquisition)

- When inside **/lime/src** directory, command `sudo make` will create **lime-…-generic.ko** file



- Command line: `sudo insmod ./lime-[VERSION]-generic.ko „path=../Linux64.mem format=raw”`
- **.mem** file is created and ready for further analysis

Video with steps on creating .mem file: https://www.youtube.com/watch?v=_7Tq8dcmP0k

LiME was successfully tested on:
Ubuntu (18.04.) – kernel 4.15.0-65-generic
Kali (2019.4) – kernel 5.2.0-kali2-amd64
openSUSE (Leap 15.1 & Tumbleweed 20190930) – kernel 5.3.1-1-default
~~Tails (3.16) – kernel 4.19.0-5-amd64~~
~~Red Hat (8.0) – kernel 4.18.0-80.el8.x86_64~~

AVML (Acquire Volatile Memory for Linux)

- Used to acquire memory without knowing the target OS distribution or kernel
- Memory sources:
  - /dev/crash
  - /proc/kcore
  - /dev/mem
- Download AVML:
  - `git clone https://github.com/microsoft/avml`

498

One of the problems we've experienced over the years with Linux memory forensics was the difficulty of obtaining a memory dump. This is because most acquisition tools require a loadable kernel module to be built on the exact same kernel version as the target. Building the kernel module on the target itself has several problems:

- It isn't forensically sound (we shouldn't be building our collection software on the target)
- It is also isn't stealthy – Ray Charles can see what's coming next if you're building forensic acquisition software
- Kernel headers must be installed on the target system (not a given)

Microsoft aims to change this with the release of avml. The avml tool is written in Rust and can capture memory without building a loadable kernel module. There's no requirement that rust libraries be present on the target system either. The avml tool is built statically linked, so there shouldn't be any additional libraries required on the target machine. Obviously, you must be root to acquire memory.

The avml tool attempts to capture from one of the following devices (in this order):

/proc/kcore
/dev/crash
/dev/mem

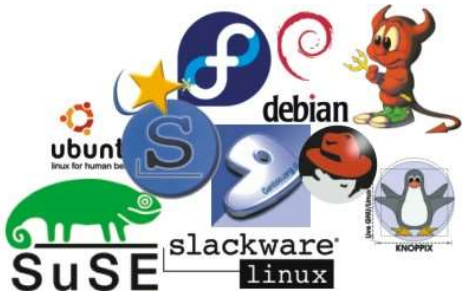
Why AVML?

- **No installation necessary, can be used as is!**
- If needed, there is a procedure for building
- Captures memory without building a loadable kernel module
- Does not require any additional libraries on the target machine

- Successfully tested on:
  - **Ubuntu** (18.04.) – kernel 4.15.0-65-generic
  - **Kali** (2016.2 light) – kernel 4.6.0-kali1-amd64
  - **Kali** (2019.2) – kernel 4.19.0-kali4-amd64
  - **Kali** (2019.4) – kernel 5.2.0-kali2-amd64
  - **openSUSE** (Leap 15.1 & Tumbleweed 20190930) – kernel 5.3.1-1-default
  - **Tails** (3.16) – kernel 4.19.0-5-amd64
  - **Red Hat** (8.0) – kernel 4.18.0-80.el8.x86_64

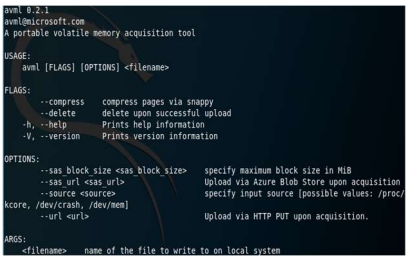
Why AVML?

- AVML should work as is on the following systems:
  - Ubuntu: 12.04, 14.04, 16.04, 18.04, 18.10, 19.04, 19.10
  - Centos: 6.5, 6.6, 6.7, 6.8, 6.9, 6.10, 7.0, 7.1, 7.2, 7.3, 7.4, 7.5, 7.6
  - RHEL: 6.7, 6.8, 6.9, 7.0, 7.2, 7.3, 7.4, 7.5, 8
  - Debian: 8, 9
  - Oracle Linux: 6.8, 6.9, 7.3, 7.4, 7.5, 7.6



AVML – How to

- Download latest release from GitHub
  - You just need the avml file – note no file extension
- In Linux, change the file to be executable
  - Open terminal in the same folder as avml file and type
  - `Chmod 755 avml`
- Check if avml works by looking at the help
  - `./avml –help`
- If all is good, start the acquisition with the command
  - `sudo ./avml <path2mem output>`
  - Eg: `sudo ./avml linux_mem.mem`



501

*AVML - Building on Ubuntu

- Install MUSL
  - `sudo apt-get install musl-dev musl-tools musl`
- Install Rust via rustup
  - `curl https://sh.rustup.rs -sSf | sh -s -- -y`
  - `source $HOME/.cargo/env` **(restart or log out afterwards for changes to apply)**
- Add the MUSL target for Rust
  - `rustup target add x86_64-unknown-linux-musl`
- Build (run command from the directory that contains Cargo.toml file)
  - `cargo build --release --target x86_64-unknown-linux-musl`
- Build without upload functionality (run command from the directory that contains Cargo.toml file)
  - `cargo build --release --target x86_64-unknown-linux-musl --no-default-features`

502

*AVML (Acquiring memory)

- After finishing the build, binary should be reachable at:
  - ./target/x86_64-unknown-linux-musl/release/
- To acquire the image of the system of interest:
  - `sudo ./avml /[PATH TO FILE]/[FILENAME].mem`
- You can find the prebuild AVML in USB drive Linux imaging\avml-release
  - Just copy the folder on to your imaging USB drive or SSD
  - Make sure that is properly formatted - **exFAT** or similar LINUX compatible file system
  - Don’t use **NTFS**!!!

AVML (Acquiring memory)

- Practical
  - Copy the “avml-release” folder on to your Kali linux
  - Try and image the RAM

504

Building Volatility profile for Linux

- Note, you will need to be ONLINE for this!
  - `git clone https://github.com/volatilityfoundation/volatility.git`
- Go to the Linux tools available to build the profile here:
  - `cd volatility/tools/linux/`
- Here we need to run `Make` but it has binary dependency we might have to install - dwarf

```
[demo@ubuntu linux]$ make
make -C //lib/modules/5.3.0-46-generic/build CONFIG_DEBUG_INFO=y M="/home/demo/Desktop/volatility/tools/linux" modules
make[1]: Entering directory '/usr/src/linux-headers-5.3.0-46-generic'
  CC [M]  /home/demo/Desktop/volatility/tools/linux/module.o
  Building modules, stage 2.
  MODPOST 1 modules
WARNING: modpost: missing MODULE_LICENSE() in /home/demo/Desktop/volatility/tools/linux/module.o
see include/linux/module.h for more information
  CC      /home/demo/Desktop/volatility/tools/linux/module.mod.o
  LD [M]  /home/demo/Desktop/volatility/tools/linux/module.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.3.0-46-generic'
/bin/sh: 1: dwarfdump: not found
Makefile:10: recipe for target 'dwarf' failed
make: *** [dwarf] Error 127
```

  - Run: `sudo apt install dwarfdump`

Building Volatility profile for Linux

- Run the make command again:
  - `make`

```
[demo@ubuntu linux]$ make
make -C //lib/modules/5.3.0-46-generic/build CONFIG_DEBUG_INFO=y M="/home/demo/Desktop/volatility/tools/linux" modules
make[1]: Entering directory '/usr/src/linux-headers-5.3.0-46-generic'
  Building modules, stage 2.
  MODPOST 1 modules
WARNING: modpost: missing MODULE_LICENSE() in /home/demo/Desktop/volatility/tools/linux/module.o
see include/linux/module.h for more information
make[1]: Leaving directory '/usr/src/linux-headers-5.3.0-46-generic'
dwarfdump -di module.ko > module.dwarf
make -C //lib/modules/5.3.0-46-generic/build M="/home/demo/Desktop/volatility/tools/linux" clean
make[1]: Entering directory '/usr/src/linux-headers-5.3.0-46-generic'
  CLEAN   /home/demo/Desktop/volatility/tools/linux/Module.symvers
make[1]: Leaving directory '/usr/src/linux-headers-5.3.0-46-generic'
```

- Now we have to find out the exact kernel version of our target system
  - `uname –a`

```
[demo@ubuntu Desktop]$ uname -a
Linux ubuntu 5.3.0-46-generic #38~18.04.1-Ubuntu SMP Tue Mar 31 04:17:56 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
```

  - Note its 5.4.0-46-generic, check your output!!

Building Volatility profile for Linux

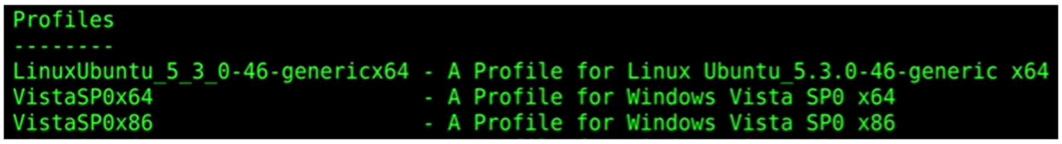
- Next, we create a zip archive of the Volatility profile for that EXACT kernel version
- The zip fie will contain:
  - Created dwarf file from before
  - Symbols map of the kernel in question
- To do this in one go, type:
  - `sudo zip <kernel version>.zip ./volatility/tools/linux/modules/module.dwarf /boot/System.map<kernel version>`

```
[demo@ubuntu Desktop]$ sudo zip Ubuntu_5.3.0-46-generic.zip ./volatility/tools/linux/module.dwarf /boot/System.map-5.3.0-46-generic
updating: volatility/tools/linux/module.dwarf (deflated 91%)
  adding: boot/System.map-5.3.0-46-generic (deflated 79%)
```

- On the desktop, there should be a zip file with the profile for that exact Linux kernel

Building Volatility profile for Linux

- Now that we have the profile for the RAM image, we use it as with windows profiles - copy the *.zip file to the volatility linux profiles
  - `volatility-master\volatility\plugins\overlays\linux`
- In linux use the command:
  - `cp <ProfileName>.zip volatiltiy/volatility/plugins/overlays/linux/`
- If we run volatility as in windows and list the profiles, the newly created one should be on top:
  - cd to volatility folder
  - `Python vol.py --info | more`



https://mirrors.edge.kernel.org/pub/linux/kernel/ - databases of many kernels that could be utilized for this purpouse

Back in Windows….

- Volatility 2 has a number of commands ready to go
  - `vol.py --info | findstr /i linux`

- There are also some prebuilt linux profiles available to download
- https://github.com/volatilityfoundation/profiles

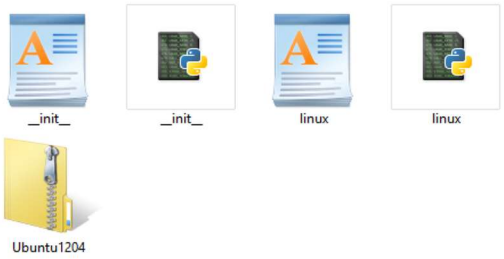
Distribution and Kernel	Bash history_list	Profile
CentOS (DFRWS 2008 Challenge)	0x80fab1c	dfrws-profile.zip
OpenSuSE 12.1 x86 (3.1.0-1.2-desktop)	N/A (shared lib/ASLR)	OpenSuSE12.zip
Fedora 17 x64 (3.5.2-3.fc17.x86_x64)	0x6e74e0	Fedora17.zip
Ubuntu 12.04 LTS x64 (3.2.0-23-generic)	0x6ed4a0	Ubuntu1204.zip
Mandriva 2011 x64 (2.6.38.7-desktop-lmnb2)	0x6cbf80	Mandriva2011.zip
CentOS 6.3 x64 (2.6.32-279.el6.x86_x64)	0x6e0950	CentOS63.zip

Doing –info will list all the commands, however just doing the command above will isolate the Linux commands. You could '>>' to a text file to keep a list handy.
Huge linux profile repo here: https://isf-server.techanarchy.net/

Copyright 2021 – CSITech Ltd

Linux

- The profile ends up as a .zip file as we already found out
- Just copy the zip file from Volatility setup/Profiles into the folder:
  - ~volatility/plugins/overlays/linux/
  - DO NOT UNZIP – just copy



Copyright 2021 – CSITech Ltd

Linux

- From your key Volatility setup/Profiles copy the profile Ubuntu1204.zip to:
  - …volatility/plugins/overlays/linux/
- Unzip RAM DUMP - ubuntu1204mem.zip

Copyright 2021 – CSITech Ltd

Linux

- Command has to include the profile so…
  - vol.py linux_pslist --profile=<profilename> -f <RamDump>
- To see what profiles are available to you:
  - `vol.py --info | findstr /i linux`

```
C:\Users\4N6\Desktop\volatility-master>vol.py --info | findstr /i linux
Volatility Foundation Volatility Framework 2.6.1
LinuxAMD64PagedMemory          - Linux-specific AMD 64-bit address space.
linux_apihooks              - Checks for userland apihooks
linux_arp                   - Print the ARP table
linux_aslr_shift            - Automatically detect the Linux ASLR shift
linux_banner                - Prints the Linux banner information
linux_bash                  - Recover bash history from bash process memory
linux_bash_env              - Recover a process' dynamic environment variables
linux_bash_hash             - Recover bash hash table from bash process memory
linux_check_afinfo          - Verifies the operation function pointers of network protocols
linux_check_creds           - Checks if any processes are sharing credential structures
linux_check_evt_arm         - Checks the Exception Vector Table to look for syscall table hooking
linux_check_fop             - Check file operation structures for rootkit modifications
linux_check_idt             - Checks if the IDT has been altered
linux_check_inline_kernel   - Check for inline kernel hooks
linux_check_modules         - Compares module list to sysfs info, if available
linux_check_syscall         - Checks if the system call table has been altered
linux_check_syscall_arm     - Checks if the system call table has been altered
```


Copyright 2021 – CSITech Ltd

Linux

- Many commands in the list for Volatility 2
- Volatility 3 has just few commands
  - `py -3 vol.py -h | findstr /i linux`

```
C:\Users\4N6\Desktop\volatility3-develop>py -3 vol.py -h | findstr /i linux
    banners.Banners      Attempts to identify potential linux banners in an
    linux.bash.Bash      Recovers bash command history from memory.
    linux.check_afinfo.Check_afinfo
    linux.check_creds.Check_creds
    linux.check_idt.Check_idt
    linux.check_modules.Check_modules
    linux.check_syscall.Check_syscall
    linux.elfs.Elfs      Lists all memory mapped ELF files for all processes.
    linux.keyboard_notifiers.Keyboard_notifiers
    linux.lsmod.Lsmod    Lists loaded kernel modules.
    linux.lsof.Lsof      Lists all memory maps for all processes.
    linux.malfind.Malfind
    linux.proc.Maps      Lists all memory maps for all processes.
    linux.pslist.PsList
                         Lists the processes present in a particular linux
    linux.pstree.PsTree
    linux.tty_check.tty_check
```

- We will cover the key commands on a clean Ubuntu install

Copyright 2021 – CSITech Ltd

Linux - pslist

- Just like the Windows command
- Creates a list of running processes
  - `vol.py linux_pslist --profile=LinuxUbuntu1204x64 -f c:\ubuntu1204.mem`

You will remember Pslist from the Windows command. It does exactly the same.

Copyright 2021 – CSITech Ltd

Linux - pslist

Process ID User ID

```
Offset             Name                 Pid             Uid             Start Time
------------------ -------------------- --------------- --------------- ---------
0x000088003d6a0000 init                 1               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a16f0 kthreadd             2               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a2de0 ksoftirqd/0          3               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a44d0 kworker/0:0          4               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a5bc0 kworker/u:0          5               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d0000 migration/0          6               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d16f0 watchdog/0           7               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d2de0 cpuset               8               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d44d0 khelper              9               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d5bc0 kdevtmpfs            10              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6f8000 netns                11              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6f96f0 sync_supers          12              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fade0 bdi-default          13              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fc4d0 kintegrityd          14              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fdbc0 kblockd              15              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d0d8000 ata_sff              16              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d0d96f0 khubd                17              0               Fri, 04 May 2012 19:23:59 +0000
```


The output, however, if very different.

Copyright 2021 – CSITech Ltd

Linux - pslist

- Important to learn what a‘normal’Linux boot process looks like
- What are ‘normal’ processes
- Google is your friend!

```
Offset             Name                 Pid             Uid             Start Time
------------------ -------------------- --------------- --------------- ----------
0x000088003d6a0000 init                 1               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a16f0 kthreadd             2               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a2de0 ksoftirqd/0          3               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a44d0 kworker/0:0          4               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6a5bc0 kworker/u:0          5               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d0000 migration/0          6               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d16f0 watchdog/0           7               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d2de0 cpuset               8               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d44d0 khelper              9               0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6d5bc0 kdevtmpfs            10              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6f8000 netns                11              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6f96f0 sync_supers          12              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fade0 bdi-default          13              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fc4d0 kintegrityd          14              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d6fdbc0 kblockd              15              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d0d8000 ata_sff              16              0               Fri, 04 May 2012 19:23:59 +0000
0x000088003d0d96f0 khubd                17              0               Fri, 04 May 2012 19:23:59 +0000
```


Copyright 2021 – CSITech Ltd

Linux - pstree

- Slightly different to pslist
- Provides a child/parent relationship to each process
- What processes are related to others

  - `vol.py linux_pstree --profile=LinuxUbuntu1204x64 -f c:\ubuntu1204.mem`

Pstree is available for the Windows OS but this is still a nice command for seeing how the OS is structured.

Copyright 2021 – CSITech Ltd

Linux - pstree

```
.lightdm              838             0
..Xorg                873             0
..lightdm             1019            0
...gnome-session      1120            1000
....ssh-agent         1156            1000
....gnome-settings-   1172            1000
....metacity          1350            1000
....unity-2d-panel    1365            1000
....unity-2d-shell    1366            1000
....gnome-fallback-   1380            1000
....nautilus          1381            1000
....bluetooth-apple   1382            1000
....nm-applet         1383            1000
....polkit-gnome-au   1384            1000
....gdu-notificatio   1667            1000
....telepathy-indic   1670            1000
....update-notifier   1754            1000
.getty                1013            0
```


Copyright 2021 – CSITech Ltd

Linux - psxview

- This provides a list of processes from 3 different system areas
- Can help to ID malware attempting to be hidden from standard system lists

- Pslist
- Pid hash
- Kmem_cache

Similar to the process lists maintained by Windows we have 3 different lists of processes for Linux, this provides them all.. It is possible in Linux for malware to modify the System Call Table and effectively hide a complete operating system,

Copyright 2021 – CSITech Ltd

Linux - psxview

```
vol.py linux_psxview --profile=LinuxUbuntu1204x64 -f
c:\ubuntu1204.mem
```

```
0x00008800373b96f0 dconf worker          1521 False  True     False
0x000088002718c4d0 dconf worker          1377 False  True     False
0x0000880036d1c028 ?                       -1 False  True     False
0x00008800296bdbc0 console-kit-dae       1089 False  True     False
0x00008800200d16f0 gdbus                 1466 False  True     False
0x000088002593c4d0 gvfs-afc-volume       1411 False  True     False
0x000088002807ade0 gdbus                 1356 False  True     False
0x00008800296744d0 console-kit-dae       1068 False  True     False
0x0000880025845bc0 dconf worker          1390 False  True     False
0x00008800296f2de0 console-kit-dae       1102 False  True     False
0x000088003b185bc0 console-kit-dae       1047 False  True     False
0x000088003e00f928                          0 False  True     False
0x00008800296a96f0 console-kit-dae       1081 False  True     False
0x00008800280f16f0 gnome-keyring-d       1170 False  True     False
```


Copyright 2021 – CSITech Ltd

Linux - psaux

- Very useful view which shows each process with its full command path
- Useful for seeing where process binaries exist on the drive
- Useful for seeing command paths and arguments

Psaux can help us to see where the command is running from. In a similar way to Windows we can check that a process is running from the right system location.

Copyright 2021 – CSITech Ltd

Linux - psaux

```
vol.py linux_psaux --profile=LinuxUbuntu1204x64
-f c:\ubuntu1204.mem
```

```
1514   1000   /usr/lib/unity-lens-files/unity-files-daemon
1515   1000   /usr/lib/unity-lens-applications/unity-applications-daemon
1518   0      [kworker/0:3]
1519   1000   /usr/lib/unity-lens-music/unity-music-daemon
1520   1000   /usr/bin/python /usr/lib/unity-lens-video/unity-lens-video
1555   1000   /usr/bin/zeitgeist-daemon
```


Copyright 2021 – CSITech Ltd

Linux - ifconfig

- Same as Ipconfig in Windows
- Lists all network adapters, IP addresses and Mac addresses
- You should be able to run it now ☺
- Linux_ifconfig
  - Not much to see on this RAM dump

Interface	IP Address	MAC Address	Promiscous Mode
lo	127.0.0.1	00:00:00:00:00:00	False

Copyright 2021 – CSITech Ltd

Linux - mount

- Enumerates all mounted drives
- Nothing to see on this RAM dump
- Very useful to see what drives are currently mounted on the system
- Includes encrypted drives

- Linux_mount

This is a great command to know what was mounted on the system when the RAM was imaged. You can see information on all drives including mounted encrypted drives.

Copyright 2021 – CSITech Ltd

Linux - Netstat

- Same as Windows version
- Lists all open sockets
  - (Errors on this RAM Dump)

- Linux_netstat

```
Volatile Systems Volatility Framework 2.2_rc1
TCP      192.168.110.150:13377 192.168.110.140:41744 CLOSE_WAIT           _h4x_bd/2777
TCP      0.0.0.0:13377         0.0.0.0:0             LISTEN                      _h4x_bd/2777
TCP      192.168.110.150:13377 192.168.110.140:41745 ESTABLISHED          _h4x_bd/2777
[snip]
```


Good old Netstat! Pops up everywhere. Does the same job as the Windows Volatility command Connscan or Netscan. It extracts all network connection objects and provides the IP addresses, local and remote and also the state of the connection.

Copyright 2021 – CSITech Ltd

Linux - LSOF

- List of Open Files
- Shows all files open on the system
- Very useful in Malware investigations and seeing what has been in use by a user
- Very verbose output

- Linux_lsof

LSOF will be familiar to Linux people. Standing for List of Open Files, it provides exactly that, a list of all the currently open files on the system. It can be rather verbose!

Copyright 2021 – CSITech Ltd

Linux - LSOF

```
1555        5 /
1555        6 /[eventfd]
1555        7 /[eventfd]
1555        8 /home/user/.local/share/zeitgeist/activity.sqlite
1555        9 /home/user/.local/share/zeitgeist/activity.sqlite-wal
1555       10 /home/user/.local/share/zeitgeist/activity.sqlite-shm
1555       11 /
1555       12 /[eventfd]
1555       13 /
1561        0 /dev/null
1561        1 /dev/null
1561        2 /dev/null
1561        3 /[eventfd]
1561        4 /dev/null
1561        5 /
1561        6 /[eventfd]
1561        7 /home/user/.local/share/zeitgeist/activity.sqlite
1561        8 /[eventfd]
1561        9 /home/user/.local/share/zeitgeist/activity.sqlite-wal
1561       10 /home/user/.local/share/zeitgeist/activity.sqlite-shm
1561       11 /home/user/.local/share/zeitgeist/fts.index/record.DB
1561       12 /
1561       13 /home/user/.local/share/zeitgeist/fts.index/termlist.DB
1561       14 /home/user/.local/share/zeitgeist/fts.index/position.DB
1561       15 /home/user/.local/share/zeitgeist/fts.index/postlist.DB
1563        0 /dev/null
1563        1 /dev/null
1563        2 /dev/null
1563        3 /[eventfd]
1563        4 /
```


Copyright 2021 – CSITech Ltd

Linux - DMESG

- Huge amount of information
- Boot messages
- Some error messages
- System messages
  - Such as USB's plugged in etc
- Linux_dmesg

Again, familiar to Linux people, DMESG, Driver or Display Message, prints the kernel messages. This can help in identifying how the machine booted, the connected drives and many more things. It's helpful to identify insertion or removal of USB devices and things like that.

Copyright 2021 – CSITech Ltd

Linux - DMESG

```
<6>[    0.914634] ehci_hcd: USB 2.0 'Enhanced' Host Controller (EHCI) Driver
<6>[    0.914675] ehci_hcd 0000:02:01.0: PCI INT A -> GSI 19 (level, low) -> IRQ 19
<6>[    0.914725] ehci_hcd 0000:02:01.0: EHCI Host Controller
<6>[    0.914760] ehci_hcd 0000:02:01.0: new USB bus registered, assigned bus number 1
<7>[    0.914913] ehci_hcd 0000:02:01.0: cache line size of 32 is not supported
<6>[    0.914937] ehci_hcd 0000:02:01.0: irq 19, io mem 0xc9000000
<6>[    0.925773] ehci_hcd 0000:02:01.0: USB 2.0 started, EHCI 1.00
<6>[    0.926141] hub 1-0:1.0: USB hub found
<6>[    0.926146] hub 1-0:1.0: 6 ports detected
<6>[    0.926326] ohci_hcd: USB 1.1 'Open' Host Controller (OHCI) Driver
<6>[    0.926340] uhci_hcd: USB Universal Host Controller Interface driver
<6>[    0.926382] uhci_hcd 0000:02:00.0: PCI INT A -> GSI 18 (level, low) -> IRQ 18
<6>[    0.926414] uhci_hcd 0000:02:00.0: UHCI Host Controller
<6>[    0.926469] uhci_hcd 0000:02:00.0: new USB bus registered, assigned bus number 2
<6>[    0.926744] uhci_hcd 0000:02:00.0: irq 18, io base 0x00002000
<6>[    0.926989] hub 2-0:1.0: USB hub found
<6>[    0.926992] hub 2-0:1.0: 2 ports detected
```


Copyright 2021 – CSITech Ltd

Linux - Cpuinfo

- Displays the make and type of the CPU
- Linux_cpuinfo

```
Processor    Vendor           Model
------------ ---------------- -----
0            GenuineIntel     Intel(R) Core(TM) i7 CPU       M 620  @ 2.67GHz
```


Does what it says on the tin

Copyright 2021 – CSITech Ltd

Practical

- Go to the Linux Practical

Copyright 2021 – CSITech Ltd

MacOS RAM forensics

Copyright 2021 – CSITech Ltd

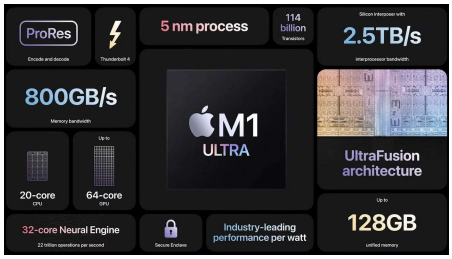
macOS RAM

- Hard, next to impossible without a password
- Mac RAM is protected by default
- Additionally, Kernel Extensions (more commonly known as KEXTs) are required to image the RAM
  - KEXT files are bundles that interact with the operating system's kernel, granting them elevated privileges that are required to image RAM on a Mac
- **RAM imaging currently impossible on live systems running M silicone due to Apple's notarization process**

Copyright 2021 – CSITech Ltd

macOS RAM

- Notarization
  - Apple requires software to be notarized to run on newer versions of macOS
  - Software that is not notarized by Apple will be labelled as malicious and prevented from being run permanently in future versions of macOS
  - Security restrictions prohibit the use of most KEXTs–including those utilized in RAM imagers.
  - Additional security by limiting the number of applications that use the kernel and have elevated permissions
- New Apple silicone makes it even more complicated!



Copyright 2021 – CSITech Ltd

macOS RAM

- Verry little information about it in the community
  - Matthieu Suiche
    - http://www.msuiche.net/con/BHDC2010_MacmacOS_PhysicalMemory.pdf
  - Sumuri
  - Cellebrite
  - SANS

Copyright 2021 – CSITech Ltd

macOS RAM

- MacMemoryReader released - free
- Execute Mac Memory Reader as root:
  - sudo ./MacMemoryReader <filename>
  - where <filename> is the path to write memory to, such as /Volumes/STORAGE/ram_dump.mach-o
- Only Mac OS X 10.4 through 10.7

Copyright 2021 – CSITech Ltd

macOS RAM

- Rekall OSXpmem
- Again pretty outdated – last version form 2017
- Supports up to El Capitan

Copyright 2021 – CSITech Ltd

macOS RAM

- FireEye Memoryze for the Mac - free
- Current Version: Memoryze for the Mac 1.1
- Release Date: November 5, 2012
- Supported Operating Systems:
  - Mac OS X Snow Leopard (10.6) 32/64-bit, Mac OS X Lion (10.7) 32/64-bit, Mac OS X Mountain Lion (10.8) 64-bit
- OSXpmem – again a bit outdated

Copyright 2021 – CSITech Ltd

RAM Imaging MacOS - Recon ITR

- Mac OS tool
- Boot environment allows you to image Mac RAM with no password – soft reboot
- Allows you to customize what is imaged by selecting whatever files or folders that are of interest to you
  - Ability to automatically collect macOS artefacts for all user accounts or just one
  - Live RAM imaging
- No support for M silicone

Copyright 2021 – CSITech Ltd

RAM Imaging MacOS – Cellebrite Digital Collector

- Mac OS tool
- Almost same functionality as RECON ITR
- Has it's own custom environment for preboot RAM imaging – requires admin password
- Live imaging
  - Target and forensically acquire files, folders, and user directories while avoiding known system files and other unneeded data
  - Live RAM imaging
- No support for M silicone

Copyright 2021 – CSITech Ltd

macOS RAM Analysis

- Works very well
  - Strings and Foremost tested
  - Photorec tested
- Volatility 2.6.1 includes a number of macOS commands

```
C:\Users\4N6\Desktop\volatility-master>vol.py --info | findstr /i mac
Volatility Foundation Volatility Framework 2.6.1
MachOAddressSpace              - Address space for mach-o files to support atc-ny memory reader
linux_slabinfo             - Mimics /proc/slabinfo on a running machine
mac_adium                  - Lists Adium messages
mac_apihooks               - Checks for API hooks in processes
mac_apihooks_kernel        - Checks to see if system call and kernel functions are hooked
mac_arp                    - Prints the arp table
mac_bash                   - Recover bash history from bash process memory
mac_bash_env               - Recover bash's environment variables
mac_bash_hash              - Recover bash hash table from bash process memory
mac_calendar               - Gets calendar events from Calendar.app
mac_check_fop              - Validate File Operation Pointers
mac_check_mig_table        - Lists entires in the kernel's MIG table
mac_check_syscall_shadow   - Looks for shadow system call tables
mac_check_syscalls         - Checks to see if system call table entries are hooked
mac_check_sysctl           - Checks for unknown sysctl handlers
mac_check_trap_table       - Checks to see if mach trap table entries are hooked
mac_compressed_swap        - Prints Mac OS X VM compressor stats and dumps all compressed pages
mac_contacts               - Gets contact names from Contacts.app
mac_dead_procs             - Prints terminated/de-allocated processes
mac_dead_sockets           - Prints terminated/de-allocated network sockets
mac_dead_vnodes            - Lists freed vnode structures
```


Copyright 2021 – CSITech Ltd

macOS RAM

- Volatility 3, limited number of macOS commands
  - `py -3 vol.py -h | findstr /i mac`

```
C:\Users\4N6\Desktop\volatility3-develop>py -3 vol.py -h | findstr /i mac
    mac.bash.Bash       Recovers bash command history from memory.
    mac.check_syscall.Check_syscall
    mac.check_sysctl.Check_sysctl
    mac.check_trap_table.Check_trap_table
                        Check mach trap table for hooks.
    mac.ifconfig.Ifconfig
    mac.kauth_listeners.Kauth_listeners
    mac.kauth_scopes.Kauth_scopes
    mac.kevents.Kevents
    mac.list_files.List_Files
    mac.lsmod.Lsmod     Lists loaded kernel modules.
    mac.lsof.Lsof       Lists all open file descriptors for all processes.
    mac.malfind.Malfind
    mac.mount.Mount     A module containing a collection of plugins that
                        produce data typically foundin Mac's mount command
    mac.netstat.Netstat
    mac.proc_maps.Maps  Lists process memory ranges that potentially contain
    mac.psaux.Psaux     Recovers program command line arguments.
    mac.pslist.PsList   Lists the processes present in a particular mac memory
    mac.pstree.PsTree   Plugin for listing processes in a tree based on their
    mac.socket_filters.Socket_filters
    mac.timers.Timers   Check for malicious kernel timers.
    mac.trustedbsd.Trustedbsd
    mac.vfsevents.VFSevents
```


Copyright 2021 – CSITech Ltd

macOS RAM

- Commands are run in the same way as Windows
- Profiles need to be loaded into the path:
  - ~volatility/plugins/overlays/mac
- Profiles can be downloaded from:
  - https://github.com/volatilityfoundation/profiles
  - Note the limited availability for the new systems
    - v10.5 to v10.13
  - They are on your key!

Copyright 2021 – CSITech Ltd

Practicals

Don’t Panic!

Copyright 2021 – CSITech Ltd

The Malware Practical

Practical

Copyright 2021 – CSITech Ltd

Scenario

- Asked by a company to investigate unusual activity on an employee's bank account
- Employee received email from a colleague (seems it was spoofed) with a PDF attachment
- Opened the PDF but a few days later odd transfers happened from his bank account
- Company imaged the computers RAM

Copyright 2021 – CSITech Ltd

Scenario

- Unzip Bob.vmem.zip
- ID the operating system of the computer
- List the running processes
- Remembering that it was a PDF, what process is likely responsible?

Copyright 2021 – CSITech Ltd

Scenario

- List IP connections, are there suspicious processes with a remote IP
- What is the IP address, who owns it?

Copyright 2021 – CSITech Ltd

Scenario

- Extract all ‘HTTPS’ lines from the RAM dump
- Use Strings and Findstr
- Any odd lines?
- Hint – search for ‘Bank’

Copyright 2021 – CSITech Ltd

Scenario

- Dump the suspicious process space (vaddump)
- Use Strings command – filtering for http://
- Any interesting URLS (Latest at the top).

Copyright 2021 – CSITech Ltd

Scenario

- Dump the parent process space
- ‘Strings’ the parent process, filter with the name of the suspicious URL
  - (hint – the word ‘search’)
- What can you see?
- Google the URL – what do you find?

Copyright 2021 – CSITech Ltd

Scenario

- Dump the 2 processes (procdump)
- Upload the 2 process dmp files to Virus Total
- What virus is confirmed?

Copyright 2021 – CSITech Ltd

Scenario

- Using Photorec and Foremost (There will be differences) can you extract the original PDF from suspicious process space
  - (The biggest one!)
  - *Might want to rename the ram dump to .img or .dd for it to work with the carvers
  - For more advanced students make use of PDFid.py in Python Plugins to look for PDFs with integrated Javascript running which will help ID the culprit

Copyright 2021 – CSITech Ltd

Scenario

- What suspicious processes have sockets open
- Using ethscan can you extract any network packets
- Using Wireshark, filter the results to the suspicious IP address
- Has the remote server been communicating with our computer?

Full write up available here: https://www.honeynet.org/challenges/forensic-challenge-3-2010-banking-troubles/

Copyright 2021 – CSITech Ltd

The Linux Practical

Copyright 2021 – CSITech Ltd

What can you do with Linux?

- Open Ubuntu12.04practical.vmem
- Does Foremost work?
- Does Photorec work?
- Does Strings work?

Copyright 2021 – CSITech Ltd

What can you do with Linux?

- Run all Linux commands, what do you learn?:
  - Pslist
  - Pstree
  - Psxview
  - Psaux
  - Lsof
  - Ifconfig
  - Dmesg
  - cpuinfo

Copyright 2021 – CSITech Ltd

Index

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Introduction
- Typography
- Date and Time
  - date /T
  - time /T
- TCHunt
  - tchunt -d c:\
- Zeroview
- Helix Pro – Windows
- Tool Testing
- Building Tool Kit
- Memory Analysis Overview

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Ram Imaging
- Dumpit
- Belkasoft
  - ramcapture <filename>
- Rekal
  - winpmem_2.0.1.exe <filename> (Needs to be admin CMD)
- FTK
- Quick Wins
- Foremost
  - foremost -t all -v -i c:\Win10.img -o c:\foremost
- Photorec
- Bulk Extractor
- Strings
  - strings64.exe <input> -n <length>
- Internet Evidence Finder

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Volatility
  - vol.py --info
- Imageinfo
  - vol.py imageinfo -f <image file>
- Kdbgccan
  - vol.py kdbgscan -f <image file>
- Hibinfo
  - vol.py imagecopy -f <image file> -O <new file>
- Pslist
  - vol.py pslist -f <image> --profile=<profile>
- Findstr usage
  - | findstr [/?] [/I] [/V] [/X] <search string>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Pstree
  - vol.py pstree -f <image> --profile=<profile>
- Psscan
  - vol.py psscan -f <image> --profile=<profile>
- Netscan
  - vol.py netscan -f <image> --profile=<profile>
- Volatility – What can go wrong?
- Dlllist
  - vol.py dlllist -f <image> --profile=<profile> -p <PID>
- Handles
  - vol.py handles -f <image> --profile=<profile> -p <PID>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Filescan
  - vol.py handles -f <image> --profile=<profile> -p <PID>
- Procdump
  - vol.py procdump -f <image> --profile=<profile> -D <dump directory>
- Mftparser
  - vol.py mftparser -f <image> --profile=<profile>
  - mft2file.py
  - vol.py mftparser -f c:\xpsp2.img --output=body --output-file=c:\temp\mft-body
  - Mactime -b mft-body -d -y -z UTC > mfttimeline.csv

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Dumpfiles
  - vol.py dumpfiles -f <image> --profile=<profile> -n -p <PID> -D <dump directory>
- Sessions
  - vol.py sessions -f <image> --profile=<profile>
- Userassist
  - vol.py userassist -f <image> --profile=<profile>
- Screenshot
  - vol.py screenshot -f <image> --profile=<profile> -D <dump directory>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- More about malware
- Vadinfo
  - vol.py vadinfo -f <image> --profile=<profile> -p <PID>
- Vaddump
  - vol.py vaddump -f <image> --profile=<profile> -p <pid> -D <dump directory>
  - copy /b *.dmp <output>
- Psscan – graph output
  - vol.py psscan --output=dot --output-file=<output file> -f <image> --profile=<profile>
  - dot.exe -T pdf -o <newfile.pdf> <output of psscan>
- Volatility - ldrmodules
  - vol.py ldrmodules -f c:\<image> --profile=<profile>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Svcscan
  - vol.py svcscan -f <image> --profile=<image>
- Device tree
  - vol.py -f <image> --profile=<profile>
- Ethscan
  - vol.py ethscan -f <image> -D <dump directory> -C <filename.pcap>
- Bulkextractor – PCAP carving
  - bulk_extractor.exe -x all -e net -o <output dir> <path to image>
- Wireshark usage
- Veracryptsummary / Truecryptsummary
  - vol.py veracryptsummary -f <image> --profile=<image>
  - vol.py truecryptsummary -f <image> --profile=<image>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Truecryptmaster
  - vol.py truecryptmaster -f <image> --profile=<profile> -D <dump dir>
- MKDecrypt
  - mkdir /mnt/tc
  - python3.5 mkdecrypt.py -m /mnt/tc -r <container> <master key>
- Dumpfiles – extracting from specific offset
  - vol.py dumpfiles -f <image> --profile=<profile> -n -p <PID> -Q <offset> -D <dump dir>
- Bitlocker – see slides for decryption instructions
  - vol.py bitlocker -f <image> --profile=<profile>
- Volatility – hashdump
  - vol.py hashdump -f <image> --profile=<profile>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Decryption of macOS Keychain
  - vol.py bitlocker -f <image> --profile=<profile>
  - From volafox-0.9 folder: vol.py -i <image> -o keychaindump
  - chainbreaker.py -i login.keychain -k <Master key from volafox>
- Registry
- Hivelist
  - vol.py -f <image> --profile=<profile>
- Printkey
  - vol.py printkey -f c:\<image> --profile=<profile> -K <specific subkey>
- Getsids
  - vol.py getsids -f <image> --profile=<profile>
- Dumpregistry
  - vol.py dumpregistry -f <image> --profile=<profile> -D <dump dir>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Clipboard
  - vol.py clipboard -f <image> --profile=<profile>
- Live Memory Analysis
- Extracting Gmail/Yahoo data
  - strings64 <image> > c:\temp\gmailstrings.txt
  - pdgmail.py -f c:\temp\gmailstrings.txt > c:\temp\gmailout.txt
- Extracting Skype data
  - strings64 <image> > c:\temp\skypeout.txt
  - Open CMD in (Volatility setup\Skypeex) and type: skype-extract26.py

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Linux Analysis
- Linux commands and profiles
  - vol.py --info | findstr /i linux
- Linux_pslist
  - vol.py linux_pslist -f <image> --profile=<profile>
- Linux_pstree
  - vol.py linux_pstree -f <image> --profile=<profile>
- Linux_psxview
  - vol.py linux_psxview -f <image> --profile=<profile>
- Linux_psaux
  - vol.py linux_psaux -f <image> --profile=<profile>
- Linux_ifconfig
  - vol.py linux_ifconfig -f <image> --profile=<profile>

Copyright 2021 – CSITech Ltd

Index and Commands

Note – All cmd lines are one line. Where there are two dashes before a switch there is no space between them

- Linux_mount
  - vol.py linux_mount -f <image> --profile=<profile>
- Linux_netstat
  - vol.py linux_netstat -f <image> --profile=<profile>
- Linux_lsof
  - vol.py linux_lsof -f <image> --profile=<profile>
- Linux_dmesg
  - vol.py linux_dmesg -f <image> --profile=<profile>
- Linux_cpuinfo
  - vol.py linux_cpuinfo -f <image> --profile=<profile>
- Mac macOS RAM forensics

Copyright 2021 – CSITech Ltd

Live Data Forensics